

MASTER THESIS

Developing a benchmarking framework for synthetic temporal tabular data

vorgelegt von

Benjamin Pöhlmann

MIN-Fakultät

Fachbereich Informatik

Studiengang: Informatik

Matrikelnummer: 7664102

Abgabedatum: 29.06.2025

Erstgutachterin: Dr. Annett Ungethüm

Zweitgutachter: Prof. Dr. Fabian Panse

Contents

Contents	iii
List of Tables	v
List of Figures	vii
1. Introduction	1
1.1. Contributions	2
1.2. Methodology	2
1.3. Structure of the thesis	3
2. Background	5
2.1. Temporal tabular data	5
2.2. Temporal tabular data synthesis	9
2.3. Evaluating synthetic tabular data	17
3. Related work	19
3.1. Evaluation metrics	19
3.1.1. Statistical fidelity	19
3.1.2. Temporal fidelity	20
3.1.3. Downstream utility	21
3.2. Benchmarking and evaluation frameworks	21
4. TempoBench: A benchmarking framework for synthetic temporal tabular data	25
4.1. Conceptual overview	25
4.2. Transfermarkt soccer data set	27
4.2.1. Overview	28
4.2.2. Sampling	28
4.2.3. Preprocessing	29
4.2.4. Explanation of the features	30
4.2.5. Descriptive analysis	31
4.3. Evaluation Metrics	34
4.3.1. Categories	34
4.3.2. Sanity	35
4.3.3. Domain-specific	37
4.3.4. Statistical fidelity	40
4.3.5. Temporal fidelity	44

Contents

4.3.6. Downstream utility	48
4.3.7. Visualizations	50
5. Prototypical implementation	57
5.1. Requirements for the prototype	57
5.2. Architecture	58
5.3. Implemented evaluation metrics	61
5.4. Implementation details	65
6. Evaluation results and discussion	67
6.1. Experimental setup	67
6.1.1. Transfermarkt data set train/test split	67
6.1.2. Generative models evaluated	67
6.1.3. Pre-/ and postprocessing	68
6.1.4. Training the generative models	68
6.1.5. Synthesized datasets	70
6.2. TempoBench results	70
6.2.1. General	70
6.2.2. Sanity	71
6.2.3. Domain	72
6.2.4. Statistical fidelity	77
6.2.5. Temporal fidelity	84
6.2.6. Downstream utility	94
6.3. Discussion	95
7. Conclusion	101
7.1. Summary	101
7.2. Limitations	102
7.3. Future work	102
Bibliography	105
A. Appendix	121
A.1. Transfermarkt data set preprocessing steps	121
A.2. Transfermarkt feature explanation	124
A.3. TempoBench prototype interfaces	128
A.4. Thresholds for metric comparison	130

List of Tables

2.1. Exemplary tabular data with temporal validity	8
4.1. Reasons for entry creation and their descriptions	29
4.2. Preprocessing steps and their descriptions	30
4.3. Features and their meaning	30
5.1. Functional requirements	57
5.2. Non-functional requirements	58
5.3. TempoBench prototype input parameters	61
5.4. Overview of implemented evaluation metrics and visualizations	64
6.1. Dimensionality of train/test data split	67
6.2. Dimensionality of synthetic data	70
6.3. Comparison of duplicate rows	71
6.4. Comparison of datatype mismatch	71
6.5. Comparison of tuple validity overlap	72
6.6. Comparison of validity end before start	72
6.7. Real sequence excerpt	73
6.8. TVAE sequence excerpt	73
6.9. TabSyn sequence excerpt	74
6.10. PAR sequence excerpt	75
6.11. Comparison of monotonic increase violations for entities and rows	75
6.12. Comparison of excerpt for monotonic increase violation details	76
6.13. Comparison of static data remains unchanged	76
6.14. Comparison of timestamp minimum and maximum	77
6.15. Comparison of maximum validity time	77
6.16. Comparison of elementary categorical statistics	78
6.17. Comparison of elementary numerical statistics deviations with respect to the training data	78
6.18. Comparison of average Jensen-Shannon divergence	79
6.19. Comparison of correlation structure	82
6.20. Comparison of correlation structure averaged over positions	84
6.21. Comparison of incoming nearest neighbor distance	84
6.22. Comparison of average sequence length	85
6.23. Comparison of mean validity	87
6.24. Comparison of intra-class distance	87
6.25. Comparison of normalized 1-lag transition matrix distance	88

List of Tables

6.26. Comparison of TSTR-Regression	94
6.27. Overview evaluation result and comparison between models	99
A.1. Preprocessing steps and their descriptions	121
A.2. Features and their meaning	124
A.3. Thresholds for evaluation	130
A.4. Unit used for thresholds evaluation	131

List of Figures

1.1. Evaluation challenges	1
2.1. Examples for multidimensional synthesized data	12
2.2. Examples for regular and irregular sampled data	13
2.3. VAE architecture	15
4.1. Processing flow in <i>TempoBench</i>	27
4.2. Distribution of reason for sampled events	32
4.3. Distribution of leagues	32
4.4. Distribution of sequence length	33
4.5. Average market values per season and league	33
4.6. Market value marginal distribution in million € example	50
4.7. Column correlation ground-truth data	51
4.8. Sequence length distribution example	51
4.9. Confusion matrix example	52
4.10. Events per month example	53
4.11. Injuries per month example	53
4.12. Market value per season example	54
4.13. Transfer fee per season example	55
4.14. Transfer per month example	55
5.1. TempoBench prototype architecture	58
5.2. Data flow TempoBench prototype	60
6.1. Comparison of league played matches distribution	80
6.2. Comparison of market value distribution	80
6.3. Comparison of market value category distribution	81
6.4. Comparison of league distribution	82
6.5. Correlation Matrices	83
6.6. Comparison of sequence length distribution to length 10	85
6.7. Comparison of sequence length distribution	86
6.8. Comparison of events per month	89
6.9. Comparison of injuries per month	90
6.10. Comparison of market value per season	91
6.11. Comparison of transfer fees per season	92
6.12. Comparison of transfers per month	93
6.13. GSP results	95

1. Introduction

Temporal tabular data is everywhere. It plays a central role in industrial production, health-care, or the financial domain [139]. These data sets typically contain structured records with multiple entities that can evolve over time. Examples include patient records, Internet-of-Things device logs, or multi-parameter financial data.

Artificial intelligence (AI) technologies are becoming more and more relevant, with new models and applications appearing at an ever-increasing speed. Access to high-quality data is therefore critical for training these models. However, real-world data is often not available for various reasons. For example, it can be too expensive to collect the data manually, or there are legal reasons that prevent the free sharing of data sets. In this context, synthetic data, i.e. artificially generated data that approximates the real data, has emerged as a potential solution. This synthetic data can then be used to replace or augment the original data set. Among the different data types, temporal tabular data presents challenges for synthesis due to its complex nature. It can contain multiple entities, heterogeneous feature types such as numerical and categorical, and temporal dependencies that can play a role within each sequence and over the whole data set. While new synthesis models are emerging [165],[44],[2],[13],[92] evaluating the quality of synthetic temporal tabular data remains challenging. There exists a vast amount of metrics that are used for tabular data synthesis. However, most existing evaluation frameworks fail to account for temporal dependencies and especially between categorical features within these data sets. Figure 1.1 further illustrates the challenges associated with temporal data. Evaluation must not only account for metrics that hold at each snapshot of the database (1), but also for transitions between states (2) and long-term dependencies (3).

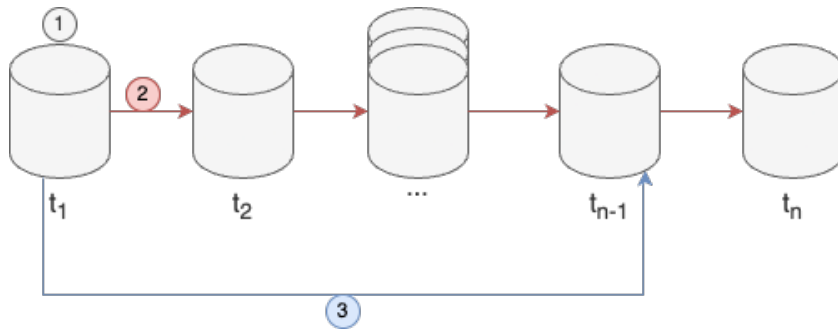


Figure 1.1.: Challenges include evaluating metrics for each snapshot (1), for transition between states (2) and long-term dependencies (3).

This thesis addresses this gap by introducing TempoBench, a benchmarking framework

1. Introduction

tailored to temporal tabular data. TempoBench incorporates different evaluation categories that examines the similarity between real and synthetic data sets along multiple dimensions, including, but not limited to, temporal fidelity, statistical fidelity, and downstream utility.

In the following sections, the contributions of this thesis are described, followed by a presentation of the methodology, and concludes with the structure of the thesis.

1.1. Contributions

This thesis makes several contributions to the field of synthetic data evaluation for temporal tabular data. First, it introduces a benchmarking framework specifically targeting temporal data sets that contain both numerical and categorical features, which is often overlooked in existing evaluation efforts. Second, a novel data set tailored to this use case is published along with this thesis. It contains soccer data sourced directly from the Transfermarkt GmbH & Co KG database. It has been preprocessed and documented, making it immediately usable by other researchers, also outside of this work and context of data synthesis. Third, a fully functional prototype of the proposed benchmarking framework is developed. Its modular design makes it easy to extend. This allows for standardized evaluation of generative models and makes it simpler to compare them with the same set of metrics. Fourth, the thesis introduces evaluation metrics that are not found in existing frameworks. This includes, for instance, a measure of update dependency between attributes or a transition matrix distance specifically designed to capture temporal relationships in categorical features. Finally, the prototype is used to assess the capabilities of existing generative tools, providing a first demonstration of the frameworks capabilities, and assesses the performance of these models. The research questions that have guided the development of the thesis are the following:

- How can we systematically quantify the quality of synthetic temporal tabular data with heterogeneous (numerical & categorical) features and integrate these metrics into an extensible, automated benchmarking framework for evaluation?
- What is the performance of existing generative methods in synthesizing temporal tabular data, as measured by the proposed metrics within the benchmarking framework?

1.2. Methodology

The methodology of this thesis proceeds in multiple stages. First, a literature review, using keywords such as “tabular data synthesis evaluation” and “synthetic temporal tabular data metrics” identifies existing benchmarking tools, frameworks and metrics used when evaluating tabular data, highlighting gaps in the assessment of temporal and categorical aspects. Second, metrics are presented and grouped into categories. Third, we translate these categories and metrics into a conceptual design for the TempoBench framework, outlining used categories and metrics as well as result visualization. Fourth, a Python prototype of the

TempoBench framework is implemented. Finally, we validate the framework by benchmarking three different generative models with the novel data set introduced in the TempoBench framework.

1.3. Structure of the thesis

This thesis is structured as follows. Chapter 2 introduces the relevant foundational concepts necessary to understand this work. It outlines temporal tabular data and motivates use of synthetic data in this context with its challenges. Additionally, it presents commonly used generative models for tabular data and temporal tabular data and concludes with a discussion on metrics and benchmarking. Chapter 3 presents related work, focusing on existing evaluation metrics and other benchmarking frameworks. These are analyzed in context of the challenges introduced in this thesis and positions TempoBench in relation to the existing frameworks. Chapter 4 then introduces the proposed TempoBench framework. First, a general overview will be given, and the used data set will be explored and evaluation metrics are presented. Chapter 5 then describes the prototypical implementation of the TempoBench framework, explaining the system architecture, processing pipeline and the integration of evaluation metrics. Chapter 6 demonstrates the application of the TempoBench prototype for three different generative models. It outlines the training setup and presents the evaluation results by analyzing them with the proposed metrics. Chapter 7 concludes this thesis by summarizing key findings, discussing limitations of the current work, and giving an outlook for future research in this area.

2. Background

This chapter introduces the foundational concepts necessary to understand the context and scope of this thesis. We first begin with an exploration of tabular and temporal data - two key data types widely used in domains such as finance, healthcare, and Internet of Things [139]. The unique characteristics of these data types with their structured format and the presence of temporal relationships present unique challenges for data synthesis, which are also explained.

Next, we introduce the concept of data synthesis, which refers to the generation of artificial data sets that aim to resemble the original data set. Various generative approaches and models that are used for tabular data will also be presented. These also highlight the diversity and complexity of this research field.

Finally, the chapter offers a brief overview of what benchmarking and evaluation metrics are used for. These foundational topics provide the basis for understanding the methods and contributions of this thesis.

2.1. Temporal tabular data

In this thesis, we evaluate synthetic temporal tabular data, where each row combines structured attributes with a temporal dimension. Understanding both static and dynamic dependencies is crucial for assessing data set quality. This section summarizes the defining characteristics of temporal tabular data and highlights the main challenges they pose for synthesis and evaluation.

Tabular data Tabular data consists of structured, spreadsheet-style records organized in rows and columns [49]. It has been a popular data storage format since the 1970s with the introduction of SQL [22]. As traditional relational databases power countless applications, a huge amount of data is stored in this format. “Heterogeneous tabular data are the most commonly used form of data and are essential for numerous critical and computationally demanding applications” [16]. In 2024, the data being created and used stands at 149 zettabytes (ZB) and is set to grow to 294 ZB in 2028 [137]. Since tabular data is prevalent in various domains such as healthcare, finance and cybersecurity, a significant portion of this data will be structured in tabular form [155], [43], [20]. Additionally, the tabular format is the most used format in Open Data portals [147].

A table with multiple columns¹ is referred to as multivariate data [115]. Moreover, when a data set includes different types of data, such as a combination of numerical and categorical features, it is termed a mixed-type data set [165]. In multivariate scenarios, tabular

¹For the remainder of this work, column, feature, and attribute are used interchangeably.

2. Background

data can exhibit dependencies within rows, for example, in the form of correlation between columns [130]. Such dependencies can also be functional, where the value of one column is determined by the value of another column within the same row [34]. A typical example is in a hierarchical context, for instance, if the city is 'Berlin', the country must be 'Germany'. Additionally, data sets can include custom constraints - rules the data must satisfy [57]. These constraints restrict the range of valid values for a column, ensuring consistency and accuracy of the data. Beyond structural constraints, one table represents a set of entities, resulting in a multi-entity data set [94]. Each entity is typically represented by a set of rows with the same identifier column, introducing additional structure and potential dependencies within or between entities [94].

At the table level, we can obtain additional statistical properties of the data [16]. For instance, one can analyze the distribution of individual columns or joint distributions across multiple columns. Beyond statistical analysis, data mining techniques, which are described as "discovering interesting, useful, and unexpected patterns in databases" by Fournier et al. [46], enables the identification of relationships in the data set. Those patterns can be extracted in an automated or semi-automated way, revealing regularities that may not be obvious simply by looking at the data. Such techniques help expose the underlying structure. Relations can vary in their degree of normalizations, which refers to the level of redundancy in the data [57]. Normalization involves splitting the data set into multiple smaller tables and explicitly defining the relationships between them, ensuring the efficiency of the database and allowing easy maintenance. In contrast to images or videos, tabular data is a well-structured data format that enables the quantifiable comparison of measures between tables, and it is more difficult to rely on intuition [139].

In the following, some of the challenges when working with tabular data are listed.

- **Missing data:** Missing data is the absence of a value for an observation in a data set [174]. There are various reasons for the missing data such as Missing Completely at Random, Missing at Random or Missing Not At Random [174]. On the other hand there are also different strategies how to deal with the missing data based on how the data set is used that include Regression Imputation, Mean Substitution or Median Imputation [113].
- **Anomaly detection and treatment:** Anomalies can hinder informational gain from analyzing data sets [141]. For example, extremely high or low values in columns may skew the statistical summaries, making data analysis more challenging.
- **Data integration from heterogeneous sources:** When consolidating data from multiple organizations or departments, inconsistencies in data formats, units and conventions can occur [157].
- **High dimensionality:** If there exist a large number of features, data analysis is more complex and computationally expensive [80].

There are also quality criteria that apply to tabular data. Accuracy measures how well the data represents world-world data, completeness measures if all required data is available, and

2.1. Temporal tabular data

consistency ensures the data adheres to the custom rules and constraints [76]. During data collection of tabular data, there can also be a multitude of reasons for quality issues. Sensor readings can be inaccurate or the software processing input before writing it to a database can contain bugs [144], [7]. Tabular data is also used to provide input to machine learning applications [16]. If the data is faulty or includes entities that do not accurately reflect real-world conditions, this can lead to machine learning models learning incorrect biases [5]. A tabular data set is considered static if every entity is deterministically described by a single row (there are no updates of an entities data), whereas introducing a temporal dimension transforms it into a dynamic data set, potentially forming a (multivariate) time series [115].

Temporal tabular data Temporal data refers to sequences of records that are typically numerical or categorical and can be multivariate that evolve over time [95]. When such temporal information is embedded within a tabular format, where each row is associated with a specific point or range in time, we refer to it as temporal tabular data. In some cases, timestamps may refer to individual attributes and therefore contain different timestamps for a single row [104].

A further refinement of this concept is multi-entity temporal tabular data, which is tabular data that contains different entities, each with their own sequence [140]. A distinction between the features of the sequences can be made. If features do not vary over time, they are considered time-invariant or static, whereas if they change over time, they are considered dynamic or time-varying [62]. These time-invariant features can include an identifier or meta information about the sequence. These sequences may be sampled in a regular or irregular interval. Regular sampling refers to observations collected at consistent time intervals such as every minute, whereas irregular sampling involves observations recorded at varying intervals or triggered by an event [70]. Consequently, this also results in sequences with varying length because they are not always sampled at the same time.

Time series data is often linked to only numerical data [138], [140], whereas temporal data in general can also contain other data types [91]. Furthermore, time series can be stationary or non-stationary. It is stationary, when its statistical properties, such as mean or variance, remain constant over time. In contrast, non-stationary time series exhibit changing statistical properties over time, for example the distribution of a feature may change depending on when it is observed [108]. A more formal definition for time series is given by Stenger et al. [139]:

“Let $X = \{\mathbf{x}_{t_1}, \mathbf{x}_{t_2}, \dots, \mathbf{x}_{t_l}\}$ be a time series with $\mathbf{x}_{t_i} \in \mathbb{R}^d$. Then we define the length l of X as the number of observations, $l = |X|$, and the dimensionality as d , the number of dimensions of the associated Euclidean space $\mathbb{R}^d$. Furthermore, if $d = 1$, we say X is univariate, and otherwise, that is, $d > 1$, multivariate”. Another type of temporal data is sequential data. It is ordered by a sequence index that can be a timestamp or just a plain numeric index [166]. These can then also contain multiple entities, leading to a multi-sequence data set [166].

Mitsa [104] makes the distinction between transaction time and valid time. Transaction time is the time the row is inserted into the database. Valid time is the time when the

2. Background

information is true in the real world. He furthermore distinguishes between an interval and an event. An interval has a beginning and ending time, while an event just has an occurrence time [104]. Temporal databases can also be categorized into the following types which include [104]:

- Snapshot database: Keeps only the latest version of the database.
- Rollback databases: Support only transaction time.
- Historical databases: Support valid time.
- Bitemporal databases: Supports valid and transaction times.

Table 2.1 depicts an exemplary table showing a historical database. It contains an entity identifier (id), start and end validity dates (start, end), static data for each entity (name), and columns that vary over time (goals, club). We can observe that the start and end dates order the tuples temporally. The goals can only remain the same or increase while the club is a categorical attribute that can also change over time. Furthermore, there exists a functional dependency between club and club_id.

id	start	end	name	goals	club	club_id
1	01-01-2020	31-12-2020	John Doe	5	FC Alpha	90
1	01-01-2021	31-12-2021	John Doe	5	FC Beta	13
1	01-01-2022	31-12-2022	John Doe	12	FC Beta	13
2	15-03-2019	14-03-2020	Alice Smith	3	United Club	22
2	15-03-2020	14-03-2021	Alice Smith	6	United Club	22
2	15-03-2021	14-03-2022	Alice Smith	10	City FC	78

Table 2.1.: Exemplary tabular data with temporal validity

Data with a temporal dimension also include a multitude of new possible characteristics that are absent in static data. One such characteristic is autocorrelation, where the current values of features are dependent on their past values [86]. Another important aspect is concept shift, which occurs when the underlying data distribution or semantics change over time [167].

Temporal data sets may also exhibit seasonality, reflecting recurring patterns at regular intervals, such as monthly fluctuations observed over years [70]. In the case of non-stationary data, long-term trends can be present, such as a consistent increase or decrease in feature values over time across sequences [70]. Moreover, temporal imbalance can arise when a set of classes dominate the data set temporally, skewing the distribution towards this class and possibly hindering downstream tasks, such as a machine learning task [168]. Similar to pattern mining in plain tabular data, the concept can be extended to temporal data. Temporal data mining can be defined as “the extraction of implicit, non trivial and potentially useful abstract information from large collection of temporal data” [95] or also as the “process of knowledge discovery in temporal databases that enumerates structures (temporal patterns or

models) over the temporal data, and any algorithm that enumerates temporal patterns from, or fits models to, temporal data is a temporal data mining algorithm" [91]. They include algorithms such as association rules [91].

The temporal dimension also introduces new challenges when working with this data. In addition to the errors that can occur in regular tabular data, Josko [71] also mentions data defects that can occur when working with a temporal dimension. This can be especially challenging when the data is supposed to be used for analytical processes such as data mining or predictive analysis like forecasting [71]. Such formalization of the defects is necessary to evaluate whether temporal data is useful in hindsight to the efficacy of downstream tasks [71]. The defects include the following,

- **Granularity Constraint Violation:** Where the attributes that define the temporal dimension can have different granularity, i.e. a difference where a data set uses an impermissible data format such as including hours in the timestamp. Some include only hours, whereas others include minutes and seconds.
- **Qualitative Constraint Violation:** Where rows of one entity need to adhere to a logical ordering like event A needs to happen before event B may occur.
- **Interval Domain Constraint Violation:** Where two attributes forming an interval do not adhere to the rules of the domain such as minimum time between consecutive events or minimum and maximum time.

Next, concepts related to synthesis of temporal tabular data are introduced.

2.2. Temporal tabular data synthesis

Data is the foundation for applying machine learning, enabling the training for increasingly complex models such as the latest Large Language Models (LLM) which power applications such as OpenAI's ChatGPT². However, real data is hard to obtain because of a range of factors: Data collection is expensive, can be error prone, or is simply not possible due to legal constraints [75]. In particular, privacy concerns limit access to sensitive data sets such as those in the medical domain, where even anonymization techniques may fail to prevent reidentification [124]. While techniques such as differential privacy can mitigate these risks [39], they come at the cost of reduced data utility [171].

The following examples demonstrate the limitations of anonymization:

- Researchers were able to retrieve exact records from aggregated data from a US census [68].
- Narayanan & Shmatikov [106] re-identify Netflix users based on their watch history and ratings.

²<https://openai.com/chatgpt/overview/>

2. Background

Data synthesis has emerged as a strategy to address the aforementioned limitations. “Synthetic data is artificially generated data: the data is not captured, measured, or recorded from the real world. Instead, algorithms or software were used to create or generate this data” [75]. It is a concept that goes back to the 1990s where Rubin [125] introduced the idea of not sharing the original data but creating synthetic data. However, using synthetic data alone does not guarantee privacy and prevent leaks of private information [68]. A general definition of synthetic data is given by Stenger et al. [139]:

“Let $D_r = \{x_1, \dots, x_n\} \subseteq \mathcal{X}$ be a dataset of individual data samples x_i from data space $\mathcal{X}$ collected by observing some real-world environment. D_r follows a distribution P_r , that is, $x_i \sim P_r$. A synthesis or generation of a new dataset $D_g = \{x_1, \dots, x_m\} \subseteq \mathcal{X}$ or stream means to apply an (unconditional) generator function $g_\theta : \mathcal{Z} \rightarrow \mathcal{X}$ m times. $\mathcal{Z}$ is some input that can trigger the generation of the desired samples and is typically just Gaussian noise. θ is a set of parameters that are learned using D_r . We call D_g synthetic or generated data.”

Tabular data synthesis refers to artificially created data that replicates real-world data stored in rows and columns, as described in the previous section [44]. When a temporal component is added, such as a temporal observation for each row, the data becomes temporal tabular data. Within this context, a key distinction arises: whether the goal is to generate additional data for the entities already present in the real data set (e.g., synthesizing data for an existing stock) or whether we can synthesize entirely new entities not present in the original data set [94].

Synthetic data is already widely adopted for training machine models across various domains. One of many examples is the Google Waymo autonomous car driving system [156]. Other areas include medical research [11], [110], healthcare [98], [60] or even the construction industry [107]. Generated data will also be more useful in the future as data-driven decisions are expected to become more important and critical [151]. In some cases, combining real data with synthetic data has shown to improve performance in comparison to training on real data alone because it forces the model to learn essential features [146]. Although synthetic data is unlikely to be a complete replacement for real data, its goal is to approximate the distribution of real data features as closely as possible [132].

Beyond model performance, other advantages of using data synthesis include [75], [138], [60], [33], [94]:

- **Enables data sharing** to make research more reproducible by eliminating the exposure of sensitive or confidential information.
- **Augment small datasets**, providing additional samples to allow more robust model training.
- **Enhances data set diversity** by introducing underrepresented classes or introducing new classes.
- **Facilitates the exploration of hypothetical scenarios** and test simulations that do not occur often, such as natural catastrophes that are difficult to capture in the real world.

- **Enables controlled data generation** with desired properties, such as targeted labels or attribute distribution.

Despite its advantages, working with synthetic tabular data also presents a number of challenges that must be addressed. “Tabular data poses challenges in collection, encoding, synthesis, and evaluation due to its heterogeneous nature, complex distribution, and lack of benchmarks.” [151]. For instance, the collection of real data can come from different sources that have different dimensions, such as different features or lengths, introducing inconsistencies [152]. Another challenge is the diverse set of domains that exist in tabular data formats that make it harder to apply transfer learning [152]. Furthermore, real data can contain an intrinsic dynamism that leads to ever changing interdependence and uncertainty, which can be difficult to replicate in synthetic data [75]. Moreover, modeling rare random events that happen in real life or are in the original data due to human error are particularly challenging to synthesize [75].

To be able to use models trained on synthetic data in the real world, synthetic data must share the same patterns, associations, and characteristics as the original data [75]. Kerim [75] mentions the domain gap between synthetic data and ground-truth data, where not all constraints and dependencies that exist in the real data are accurately portrayed in the synthetic counterpart. A well generalizing model should perform reliably on previously unseen data or domains [75]. Additional challenges arise from the structural and statistical characteristics of tabular data. For instance, sparsely filled columns and high dimensionality of the data are common problems [151]. Tabular data sets can contain categories that must be encoded into numerical formats for use in machine learning models [55]. This can be problematic for synthesizers that use machine learning, as encoding without bias and information loss is difficult and scalability issues can emerge when dealing with a large number of categories [21]. In contrast to image generation, synthetic tabular data is often not the end goal but an intermediate step in solving another downstream task [151]. As a result, generation techniques focus on different qualities, such as fidelity, utility, or privacy, depending on their downstream goal [151]. Also, unlike images, evaluating entire tables simply by visual inspection is not feasible [151]. Tabular data often also has a skewed distribution where a set of classes or labels occurs much more often than others, so called “long tail distributions” [170]. Zhao et al. [170] also state that “it is unclear if existing solutions can efficiently handle highly imbalanced categorical variables and skewed continuous variables”. The size of the data set is also key in cases where small datasets lead to incorrect correlation learning [160]. Integrity constraints such as functional dependencies present yet another challenge [33]. “To ensure the best possible utility of the generated data, a universal tool must fully capture and replicate several characteristics of the original data, including column types and distributions, correlations between columns, and integrity constraints, such as unique constraints or functional dependencies.”, state Davila et al. [33]. Alzantot et al. [3] also emphasize the difficulty in evaluating a generator for synthesized data due to a lack of standardized metrics to determine how realistic and novel it is. In contrast to image or speech data, tabular data has fewer correlations between features [16]. As Kadra et al. [73] express it, “tabular datasets are the last unconquered castle for deep learning”.

2. Background

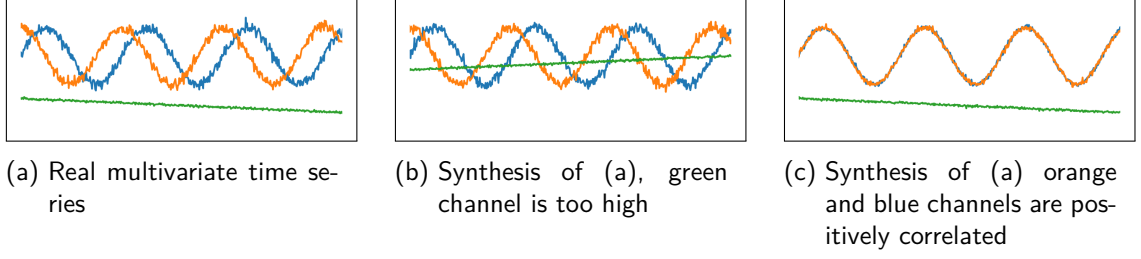


Figure 2.1.: Comparison of possible synthetic data for a multivariate time series where (a) is the real data, (b) and (c) are synthesized examples and every color represents a different feature.

The challenges become even more pronounced when a temporal dimension is added. Many current models struggle with the temporal dependencies and inter-dependence of features that happen over time [140]. These temporal dynamics are seen as more difficult than correlations between continuous or discrete features [140]. Yoon et al. [164] highlight that “a good generative model for time-series data should preserve temporal dynamics, in the sense that new sequences respect the original relationships between variables across time”. What makes temporal sequences even more challenging is that we need to capture distributions not only within a time step (row), but also across a temporal dimension [164]. For instance, they also introduce a conditional distribution for temporal transitions between feature values [164].

Figure 2.1 adapted from Stenger et al. [139] also illustrates the complexity introduced by multidimensional temporal data. The figure demonstrates that while the first synthetic example (b) closely approximates the real data (a) for the blue and orange channels, it fails to capture the green channel properly. Conversely, in the second synthetic example (c), the green and blue approximation is very good, but the orange channel is very poorly reproduced. This highlights a key challenge in that a model might be very good at learning and synthesizing a specific set of attributes and at the same time perform poorly for others. Other challenges include irregular sampling of the data set, which leads to key challenges when synthesizing data for the training of a model. They include variable gaps between observations within a sequence, variable number of observations for the sequences, and no guaranteed alignment between the sequences, as they can be sampled at different points in time [131]. Figure 2.2 adapted from Shukla & Marlin [131] illustrates the problems that can occur. Depending on when the data is sampled, different trajectories can occur for a column. This can be seen in where the valley of both variables is observed in the regularly sampled data (a), but not in the irregularly sampled data (b).

Another challenge with multi-entity data is how the entity identifier column is treated. If it is seen as just another categorical feature, it restricts the generative model to only sample seen entities and do not create entirely new entities [94].

These limitations highlight the need for flexible generative approaches, particularly those tailored to the intricacies of temporal tabular data. There exist a variety of models that specialize in synthesizing (temporal) tabular data.

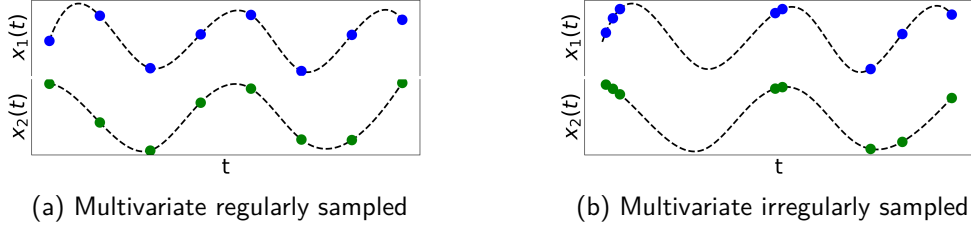


Figure 2.2.: Example showing how temporal data can seem different based on whether they are regularly sampled (a) or irregularly sampled (b).

Several key approaches will be introduced in the following.

Generation Techniques Machine learning in general enables computers to learn from experience, not from explicitly programming every possible outcome [127]. In recent decades, the amount of digital information has increased dramatically, making it possible to train vast models on them. The trend is expected to continue with the volume of digital information projected to grow even larger in the coming years [137]. Machine learning approaches can be broadly categorized into two paradigms: Model-centric and data-centric. Model-centric approaches focus on optimizing model performance, for instance, accuracy, by changing the architecture and hyperparameters with a fixed data set [84]. Data-centric machine learning has a fixed architecture and aims to improve data quality [84].

In the context of synthetic data generation, Generative AI (GenAI) plays a central role. However, alternative techniques such as rule engines, entity cloning, or data masking also exist [72]. Within GenAI, deep generative models are commonly employed and can be categorized into two types: Explicit and implicit density models. Explicit models like Variational Autoencoders (VAE) use the original data to train the model to approximate that distribution, while implicit models like Generative Adversarial Networks (GAN) do not try to approximate the data distribution explicitly but try to learn to generate samples that resemble the real data [151].

Some of the most prominent generator techniques used for tabular and temporal data are introduced below.

Generative Adversarial Networks Generative Adversarial Networks are a class of machine learning models introduced by Ian Goodfellow et al. [53] in 2014. GANs consist of two neural networks, a generator G and a discriminator D , which are trained simultaneously. The generator aims to produce synthetic data that resembles the real data, while the discriminator attempts to distinguish between real and synthetic data.

The generator maps a noise vector $z \sim p_z(z)$ from a distribution to the data space, that is synthetic data $G(z)$. The discriminator then tries to distinguish between real and synthetic samples and outputs the probability that sample x is real $D(x)$. The training then consists of a minimax between G and D represented by the following loss function where the generator

2. Background

tries to minimize and the discriminator tries to maximize it:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]. \quad (2.1)$$

The generator wants to achieve $D(G(z))$ close to 1, which means it can fool the discriminator. The first term in the sum is the average discriminator loss over the real samples, while the second term is the discriminator loss over the generated samples. A central challenge when training GANs includes mode collapse. This leads to the GAN generating samples from just a subset of the target distribution, failing to capture the full diversity of the data [136].

Beyond the original GAN architecture proposed by Goodfellow et al. [53] in 2014, there exist various adaptations that focus on different aspects of the data generation. Conditional GAN (CGAN) enables the introduction of own constraints that lead the generator to create samples with certain attributes or labels [103]. Xu et al. [159] introduced Conditional Tabular GAN (CTGAN), which focuses on tabular data with a mix of discrete and continuous columns. Yet another approach is Wasserstein GAN (WGAN), which tries to address the instability and mode collapse that occurs in standard GAN [9]. An additional type of GAN is CTAB-GAN that shows promising results in diverse data types, which also include continuous and categorical variables [170], [171].

GANs have also been extended to handle temporal data, such as with the adaptation of Recurrent GAN (RGAN) and Recurrent Conditional GAN (RCGAN) [43]. Another type of GAN focusing on time series is the Time-series GAN [164]. Zin et al. [93] proposed Doppel-GANger (DG) which also focuses on sequential data.

Additional notable GAN variants include:

- Time-series Transformer GAN which achieves high performance in five data sets when compared to other state-of-the-art models [135].
- Quant GAN for being able to deal with long-term dependencies [155].
- CMTGAN as it can generate multi table data [42].

Diffusion Models Diffusion models (DMs) are a class of generative models capable of learning complex data distributions by progressively denoising random noise. Originally proposed for high-quality image generation [61], [36], they have been adapted for other domains, including tabular and time series data [81], [149].

The diffusion process consists of two phases [61]:

- Forward process $q(x_{1:T} | x_0)$ (diffusion): This is a Markov chain that iteratively adds Gaussian noise to the data for T time steps. The original input data x_0 is corrupted until the final state x_T is almost pure noise.
- Backward process $p_\theta(x_{0:T})$: This is a Markov chain that aims to reverse the forward process. Starting from a x_T , the model iteratively denoises the sample through learned conditional distributions $p_\theta(x_{t-1} | x_t)$ to reconstruct data resembling the original input x_0 .

After training the model, the backward process can be used to generate a new element from random noise. While diffusion models have shown remarkable results in image generation, applying them to tabular data presents its own challenges. In contrast to image generation, which only contains continuous values for pixels, tabular data contains heterogeneous data types (categorical, numerical) and complex distributions [159]. Nonetheless, several works have demonstrated the potential of diffusion models in the domain of tabular data. For instance, STaSy, which shows promising results in sampling diverse samples [77], TabDDPM, which focuses on handling different feature types [81], TabCSDI, which focuses on imputing missing values [172] and SOS, which is designed for oversampling underrepresented classes [78]. Lee et al. [88] proposed CoDi, which is a model that tries to maintain the correlation between continuous and discrete variables by using two diffusion models simultaneously.

Variational Autoencoders Variational autoencoders are among the first synthetic data generators and were introduced by Kingma and Welling [79] in 2013. A standard autoencoder is made up of two neural networks: an encoder and decoder. The encoder compresses the original data into a lower-dimensional latent space and the decoder uses this representation of the latent space to attempt to decode it back to the original input data [45]. This leads to the same input always generating the same output as this process is deterministic once the parameters of the network have been learned. To allow more diversity, the architecture is modified to diversify the decoded output, as shown in Figure 2.3 adapted from Bahree [12]. Now instead of the encoder mapping to a fixed vector, it will map to a multivariate normal distribution made up of two vectors: The mean vector and the variation vector [12]. Afterwards, a point can be sampled from this distribution and passed through the decoder, resulting in a sample that resembles the original data [12]. Because there is a probability distribution that is being sampled, the same input can lead to different output from the decoder. To generate new data points, we can sample from a standard normal distribution [45].

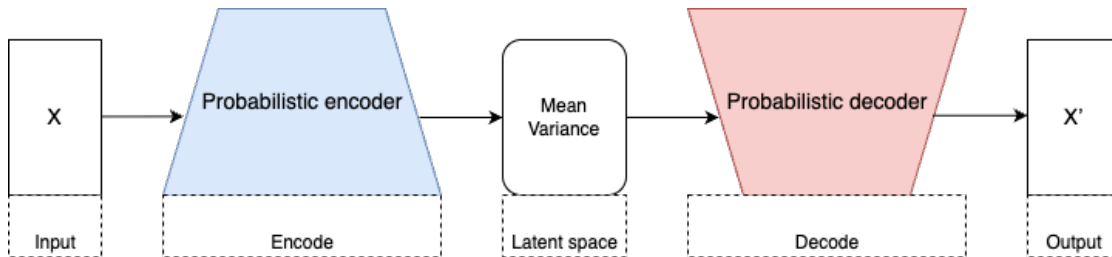


Figure 2.3.: A VAE architecture encodes the input into a mean and variance that parameterize a distribution, from which a sample is drawn and then decoded to reconstruct the original input.

One known limitation of VAEs is their handling of categorical attributes with a high dimension, which either require high computational cost or introduce more complexity [67]. Other challenges include the lack of diversity and the modeling of complex distributions [12].

2. Background

Training VAE models can also be unstable and lead to mode collapse [12]. Nevertheless, there exist various adaptations to generate tabular data with VAEs. Tazwar et al. [143] introduce TAB-VAE that focuses on categorical columns. Xu et al. [159] also focus on mixed-type tabular data and propose Tabular VAE (TVAE). Another approach by Akrami et al. [2] is called robust VAE (RTVAE) and focuses on outliers in tabular data. Apellániz et al. [8] created a model that focuses on tabular data that incorporates a Bayesian Gaussian Mixture model into the VAE architecture and shows promising results by outperforming TVAE and CTGAN. TimeVAE proposed by Desai et al. [35] focuses on multivariate time series generation and can approximate temporal dependencies such as periodicity.

Large Language Models Large Language Models (LLMs) are neural network-based statistical models that are pre-trained on large-scale text data to understand and generate human language [101]. They are transformer-based and contain billions of parameters and can learn new tasks from only a few examples [101]. Therefore, LLMs can also be used to create synthetic data. In the context of tabular data synthesis, it can be leveraged that LLMs do not differentiate between categorical or continuous columns because they are fully text-based [169]. Models like ChatGPT, Llama³, or Claude⁴ have demonstrated the ability to create data that is useful for training downstream machine learning models [48]. There are also approaches that leverage LLMs to synthesize tabular data as shown in [17], or [153] by fine-tuning an LLM to discover inter-row relationships and the format of the data which do not yet take into account a temporal dimension. An example of an approach is TabuLa that was proposed by Zhao et al. [169] and discards the pre-training of LLMs that focuses on natural language tasks and focuses entirely on tabular data training. Moreover, transformer-based models have shown promising results in synthesis of temporal data. Cholakov and Kolev [25] found that transformer-based models can be applied to time series generation.

Ensemble Methods Ensemble methods combine multiple models to generate synthetic data and have shown to produce good results in complex data sets. They try to capture the desired data distribution collectively [41]. For instance, Sun et al. [142] demonstrated the use of multiple models when focusing on privacy sensitive data sets. Furthermore, Wang et al. [151] have demonstrated that using ensemble methods show promising results in the generation of tabular data. Suh et al. [140] created TimeAutoDiff which combines an autoencoder and diffusion models to improve over state-of-the-art models in certain metrics. Lastly, Zhang et al. [165] proposed TabSyn, which also integrates a VAE with a diffusion model.

Further Approaches In addition to the techniques described above, a variety of other methods have been proposed for the generation of tabular data. These include Copulas [74], Synthetic Minority Over-Sampling Technique (SMOTE) [32] or synthpop [112]. The Synthetic Data Vault (SDV) probabilistic autoregressive model (PAR) is a neural network for the synthesis of mixed-type multivariate time series [166]. At each time step, PAR

³<https://www.llama.com/>

⁴<https://claude.ai/>

2.3. Evaluating synthetic tabular data

processes the entire history of the sequence along with a fixed context vector and predicts a probability distribution over the next value from which a sample is drawn. This sample is then appended to the history, and the process is repeated until the sequence is terminated. Beyond these, there exist graph neural networks (GNNs) to capture complex relational structures in tabular data, as well as normalizing flows (NF) [33].

Improving generation quality To improve the quality of the generation, there are various approaches. Diversity regularization can be used, which introduces a regularization term that penalizes models that generate similar examples. This can prevent mode collapse in GANs [96]. On the other hand, this can lead to the widening of the domain gap and as a result to obtaining data that is very diverse but far from real data samples [75]. External knowledge can also be incorporated to achieve a more fine-grained control of the generation process. An example of this is Directed Acyclic Tabular GANs (DATGAN) [87]. They integrate expert knowledge into deep learning models and allow for interaction between variables to be defined.

Another approach focuses on data quality. Preprocessing the data by cleaning, transforming, and ensuring representativeness can help to improve results [151]. However, another option after the synthesis itself is postprocessing. This includes rule filtering and knowledge integration [151]. We can then create custom rules and constraints that the synthesized data has to follow. If it does not follow those rules, we can filter these out or try to transform this data again [151].

2.3. Evaluating synthetic tabular data

To evaluate the performance of synthetic data generation models, a standardized comparison method is needed. Xie et al. [158] define a benchmark study as a “systematic comparison between computational methods, in which all of them are applied to a gold standard dataset and the success of their [...] predictions are summarized in terms of quantitative metrics [...]”. The data sets used can be simulated or real data [47]. The goal of the benchmark is to identify which model performs best under given conditions, such as a data set [47]. However, as previously discussed, evaluating synthetic temporal tabular data is not straightforward. In contrast to synthetic data generation for images, the intuitive evaluation is not directly possible with a human judge in temporal tabular data [139], [138]. That is because temporal tabular data is often high-dimensional and contains many time steps, which makes it hard to display and grasp visually [139].

A metric is a qualitative or quantitative scoring that assigns a numerical or categorical value of quality to synthetic data [139]. Theil et al. [145] state that “Just as choosing the right training method is important for achieving good performance in a given application, so is choosing the right evaluation metric for drawing the right conclusions”. They can be categorized in various ways, depending on the aspect of the data they are trying to measure. Further distinctions can be made about whether they work on a whole data set or just on single sequences when considering temporal data [139]. Further classification includes whether the measure is easy to obtain, that is, if a public implementation is available or

2. Background

whether the community has adapted them [139].

Each metric can be seen to measure different business requirements that an organization has [151]. An organization valuing privacy will have very different metrics than an organization focusing on simulating car crashes where the data itself is scarce. Inability to distinguish between real and synthetic data is yet another example criterion for evaluating the quality of synthetic data [3]. Visualizations are typically used for evaluating realism in non-tabular data, but one could also use classifiers where the goal is that synthesized data is not separable from the real data [3]. Current models also excel in different qualities, as Wang et al. [151] found in their study. Furthermore, Hernandez et al. [60] state that “there is no universal method or metric to evaluate and benchmark the performance of various approaches” when focusing on GANs for healthcare data.

As previously mentioned there are various categories that focus on different quality aspects of the data. They will be introduced in the following chapter alongside an excerpt of commonly used metrics.

3. Related work

This chapter provides an overview over similar works with regards to metrics and categories used in evaluating the quality of synthetic data and benchmarking this tabular and temporal data.

3.1. Evaluation metrics

The evaluation of models to synthesize temporal data is considered an active area of research and therefore there does not exist a set of metrics that is universally applicable to a wide range of datasets [139]. Marin [97] also calls for using a variety of metrics and not relying on single ones, because a single metric cannot reliably capture both distributional and structural similarity. Yan et al. [161] confirm this when benchmarking health records by saying the synthesized data set and the generation methods have to be evaluated in their specific domain context. As briefly described in the previous chapter, researchers use a variety of metrics and metric categories to evaluate the quality of synthetic tabular data. Many metrics focus only on single aspects of the data [33].

Commonly used categories and metrics from the respective category are introduced in the following.

3.1.1. Statistical fidelity

Statistical fidelity is a widely used category that includes how well the synthesized data is able to replicate the statistical characteristics the original data possesses [117], [31], [100]. For measuring the statistical approximation of the synthetic data to the original data there are various metrics. For measuring the difference between the marginal distributions between synthetic and real data, the Jensen-Shannon divergence or distance and the Kolmogorow-Smirnow test are often utilized [93], [18], [90]. Further metrics include the Kullback-Leibler divergence or the Wasserstein distance to measure differences in marginal distributions [52], [122]. For comparing categorical columns, Zhang et al. [165] use the Total Variation Distance (TVD) to quantify the densities of columns. For measuring the difference in joint distributions, the Jensen-Shannon divergence can also be used [139]. Apellániz et al. [8] use a Random Forest discriminator to measure joint distributions of real and synthetic datasets. Simpler statistical metrics include the comparison between mean and mode [172]. The minimum and maximum values can also be compared [33]. Computation of the correlation between the features in the synthetic and real data sets and comparing them afterwards is also used [81]. Elliker et al. [42] compute the distance between the correlation matrices. Distance to closest record (DCR) is also employed to capture the resemblance of single synthetic rows to real rows in the original data [116]. (Improved) Precision measures how many

3. Related work

synthetic samples lie in the support of the real data set distribution [126], [83]. (Improved) Recall measures how many real examples lie in the support of the synthetic data set distribution [126], [83]. Kim et al. [77] use a similar metric 'coverage', which evaluates whether a generated sample is close to the sphere of a real sample. Esteban et al. [43] and Wang et al. [151] apply the maximum mean discrepancy (MMD), which evaluates whether two sets of samples are of the same distribution. For this, statistics from the samples are computed and afterwards compared to determine if they are of the same distribution [43]. Train-on-Real-Test-on-Synthetic (TRTS) is also relied upon to evaluate the generator's ability to generate realistic samples [43].

For visualizing the statistical similarity, Principal Component Analysis (PCA) or t-distributed Stochastic Neighbor Embedding (t-SNE) is oftentimes used [90], [102], [135]. The correlation matrix can also be visualized to compare it qualitatively by visual inspection [81]. Distributions can also be plotted for marginal distributions [81] or joint distributions [17] to compare the synthesized to the real distributions by visual inspection.

3.1.2. Temporal fidelity

Temporal metrics are designed to capture dependencies related to the temporal dimension of the data. For example, they assess whether the synthetic data preserves patterns found in the real data [139].

Temporal metrics include comparing the autocorrelation utilized by Li et al. [92] and Wiese et al. [155] who call it autocorrelation function (ACF) score. The distribution of the time series length is another metric to look for when comparing temporal aspects in the tabular data [92]. Other metrics include the co-integration test that checks whether channels correlate in the long term [139]. But this assumes that for each synthetic time series, there is a matching time series in the real data set [139].

Measurement of the distance between time series is also common. For this, various distances such as Dynamic Time Warping (DTW) [119] or the Fréchet distance [37] can be used. These distances can be employed to compare individual entities or aggregate data over the whole data set. For example, incoming nearest neighbor (INN) computes the distance between a synthetic time series and its closest neighbor in the real data [10], which can then be averaged over the subset of samples used [139]. The intra-class distance (ICD) can be applied to measure the diversity of the synthesized data, where the average distance between sequences is calculated [89], [139]. Yoon et al. [164] and Srinivasan & Knottenbelt [135] engage a discriminative score by training a binary classifier to distinguish between real and synthetic sequences and evaluate the classification error.

Suh et al. [140] propose comparing the distributions of inter-row differences between the original and synthetic sequences to evaluate whether these temporal dependencies are integrated in the synthetic data. Boursin et al. [19] employ a similar metric that compares the mean squared error between time steps in the original and synthetic data sets. Kulkarni et al. [82] measure long-term dependencies with mutual information decay in mobility data. Wiese et al. [155] also measure long-term dependence within time series with dependence scores such as the autocorrelation or leverage effect. Most of the mentioned metrics are only

3.2. Benchmarking and evaluation frameworks

applicable to numerical features. Temporal visualizations include plotting of features in a temporal dimension [139].

In general, when measuring temporal fidelity, mining temporal patterns such as subsequences is oftentimes overlooked by researchers. The same applies to the temporal dynamics of categories over time in general, where there is a research gap.

3.1.3. Downstream utility

Stenger et al. [138] describe utility as follows: “Utility interprets the quality of synthetic data as its usefulness in a downstream (machine learning) task”.

Train-on-Synthetic-Test-on-Real (TSTR) is a widely used metric for evaluating the utility of a synthesized data set to train a model for a downstream task [43], [140], [159]. A model is trained on real data and synthetic data and afterwards the results are compared on the held-out real test data set. If the model trained on the synthesized data performs equally well, it “demonstrates the ability of the synthetic data to be used for real applications” [43]. Algorithm comparison is utilized by Lin et al. [93] where it is evaluated if algorithm rankings are preserved when training a set of classifiers on real and synthetic data. Predictive score is used by Yoon et al. [164] to evaluate how useful the synthetic data is in one-step predictions and captures conditional distributions over time. Similarity in feature importance when training a downstream model is also employed to evaluate the quality of synthetic data [151], [56].

Common visualizations include a confusion matrix when applying a classifier trained on real data to synthetic samples [111]. With this, classes that are poorly represented by the synthesizer can be made visible.

3.2. Benchmarking and evaluation frameworks

More benchmarking datasets for tabular data synthesis are needed in the research community [151]. Hernandez et al. [60] also call for reliable metrics and benchmarks to further research in this direction. Bauer et al. [13] refer to the need for “more common metrics and datasets”. Davila et al. [33] also state that there are no benchmarks to measure the utility of synthesizers for certain applications and their specific suitability in their respective domain. Alzantot et al. [3] also call for a standardization of metrics to evaluate the capabilities of a generator. Theis et al. [145] state that the evaluation of generative models heavily depends on the metrics used, resulting in different outcomes and possible conclusions depending on the metric. Apellániz et al. [8] call for a “multifaceted approach” when evaluating generative models due to a lack of standardized metrics.

Hudovernik et al. [63] introduce a benchmarking tool to evaluate relational data sets. They analyze multiple publicly available datasets and design metrics to evaluate the synthetic data. The focus is on statistical fidelity and utility. While their data sets contain multiple tables, they also do not specifically use metrics for tables with a temporal dimension. Categorical

3. Related work

attributes and their change over time are therefore not studied either.

Synthcity is a Python library for generating and evaluating synthetic tabular data [122]. It implements various models for generating data and also contains metrics to evaluate the synthesized results. It also contains a well-documented usage and architecture example. While it has metrics for statistical fidelity and downstream utility of the synthesized data, Synthcity does not explicitly incorporate any metrics for temporal data. However, it still includes models for generating time-series data.

The Synthetic Data Vault is an open-source Python library for generating and evaluating tabular data sets [117]. While generating sequential data is integrated into the tool, there are only few metrics for evaluating sequential data. At the time of writing, there are only two metrics for sequential data, "SequenceLengthSimilarity" and "StatisticMSAS". While "SequenceLengthSimilarity" compares the average length of the sequences between real and synthetic data, "StatisticMSAS" compares the changes between the sequence steps for the numerical and date data. They currently do not offer any metrics for categorical data or transition dependencies. They also do not compare the utility of the synthesized sequences for a downstream machine learning model utility.

Chundawat et al. [26] introduce TabSynDex, which is a single combined interpretable metric made up of different aspects and tries to be as general as possible. The paper shows utility in comparing different models with various data sets. It is therefore applicable to a wide range of data sets as it also covers different aspects such as statistical similarity, distributional similarity, or downstream machine learning utility. While TabSynDex can also handle mixed-type data such as numerical and categorical data, it does not handle temporal aspects nor multi-entity tables explicitly. As temporal aspects are not considered, irregular sampling is also not considered.

YData is a commercial data synthesis offering that can also evaluate synthesis results [65]. The focus lies on generating new synthetic data that also includes multi-entity multivariate time series [66]. YData's evaluation metrics focus on privacy, statistical similarity, and downstream utility. They therefore do not include metrics that focus specifically on temporal similarity in the sequences, let alone mixed-type data metrics for irregularly sampled temporal data.

Nikitin et al. [109] design Time Series Generative Modeling (TSGM), which can synthesize and evaluate time series data. Their evaluation metrics cover a wide range of metrics, including distance-based metrics and diversity. Visualizations are also included in the library. A gap in the framework is the lack of metrics that evaluate categorical data over time, such as category transitions. Moreover, there is no inherent support for irregular temporal data with mixed types. Nikitin et al. [109] state that "Although the codebase has been developed to support irregularly sampled time series data, TSGM has not yet been tested with such data and does not currently include state-of-the-art models in this domain".

Ang et al. [6] outline the Time Series Generation Benchmark (TSGBench), which focuses on time series and offers various metrics in the temporal domain. They propose usage of distance-based measures and feature-based measures, as well as visualizations. It also introduces metrics focusing on skewness or the difference in kurtosis. Yet again, TSGBench does not include metrics specific to categorical dimensions such as transition between categories,

3.2. Benchmarking and evaluation frameworks

nor update dependencies or common sequences in the time series.

Another commercial offering is Gretel.AI by Gretel Labs, which consists of generation models and also an evaluation benchmark offering [85]. The focus is also on the training time for models and only a few metrics that compare the data quality of the synthesized data. The metrics include univariate divergence with the Jensen-Shannon distance or column correlation in the data set. Again, metrics that evaluate categorical attributes in time series are missing.

Wang [152] proposes an evaluation framework TabEva that focuses on statistical fidelity and machine learning utility while also using visualization techniques. Specific quantitative metrics for temporal data are not provided.

Eigenschink et al. [40] design an evaluation framework focusing on sequential data but not only on data in tabular format. In their work, they sum up that there are domain-specific metrics and that each industry has special requirements that require an evaluation focus on different aspects of a data set depending on downstream usage [40].

Santaneglo et al. [128] propose Synthetic data Rank and Order (SynthRO) which is a tool to benchmark synthetic tabular data. They offer a user-friendly graphical user interface and dashboard to reduce the barrier for use. Although it contains various metrics for utility and statistical fidelity to the original data, only “future developments will include temporal data evaluation” [128]. Therefore, it also does not currently support evaluation metrics for temporal data, let alone irregular time series.

Hansen et al. [56] also present a benchmark focusing on Data-Centric AI. The categories used included statistical and data utility metrics. They did not include temporal data in their benchmark for tabular data.

Wang et al. [153] introduce HARMONIC that focuses on generating and evaluating tabular data using LLMs. They mainly focus on downstream utility and privacy and do not explicitly consider temporal data [153].

Du & Li [38] propose another framework that focuses on fidelity, privacy, and downstream utility. They do not include metrics for evaluating temporal data. The data sets and synthesizers evaluated also do not consider the temporal dimension at all.

Yang et al. [162] also describe an evaluation framework for synthetic tabular data. They also do not touch on temporal data and focus on statistical, privacy, and machine learning efficacy metrics.

Wang [152] also complains about the lack of details provided for implementation along with their framework. Thus, the practical usability is limited.

In summary, many of the tools and benchmarks mentioned propose using different metrics and have different focus. While most of them do not include support for irregular time series and temporal metrics, none support categorical data in specific. For example, the transition between categories or long term dependency is not inspected in any of the above mentioned tools. Stenger et al. [139] also state that there is no current benchmark with which generated time series can be objectively evaluated.

4. TempoBench: A benchmarking framework for synthetic temporal tabular data

This chapter introduces the benchmarking framework TempoBench for evaluating the synthesis of temporal tabular data developed as part of this thesis. It begins with a general overview, followed by a detailed description of the soccer dataset, including its dimensionality and associated constraints. Subsequently, the proposed evaluation metrics and the corresponding visualizations are presented.

4.1. Conceptual overview

TempoBench is a benchmarking framework for synthetic temporal tabular data. The scope of the benchmark is to evaluate the quality of synthesized data in various dimensionalities with respect to a ground-truth data set. Therefore, training a generative model and synthesizing a data set fall outside the scope of this framework. The framework focuses on single table data. These ground truth data sets are used to train a generative model whereafter a synthetic data set is sampled. TempoBench assumes that the data is formatted in the same way as the ground-truth data and does not perform preprocessing steps to ensure consistency across data sets. This data set is passed to TempoBench which will yield a detailed analysis of the synthetic data set's quality and where its strengths and weaknesses lie. The analysis consists of multiple dimensionalities that are grouped in categories. Then each category contains multiple metrics. In summary, TempoBench consists of the following components:

Data sets TempoBench supports the inclusion of multiple data sets. Each data set should contain a temporal dimension. This can be just a temporal ordering with an index, timestamps, or also validity intervals of tuples. Data sets should contain multiple entities and not just a single time series. TempoBench does not dictate whether sequences are sampled in regular or irregular intervals. Furthermore, in order to measure how well a generative model can approximate categorical features and their behavior over time, the data sets must contain categorical attributes. The categories and metrics are mostly data set agnostic, and therefore make it possible to choose from a variety of data sets fit for the use case of the synthesizer.

The initial data set integrated in TempoBench is introduced in Section 4.2.

4. *TempoBench: A benchmarking framework for synthetic temporal tabular data*

Categories Different categories capture distinct aspects of synthetic data evaluation. Sanity metrics, statistical fidelity, temporal fidelity, and downstream utility are included as categories. In addition, domain-specific metrics that apply only to a single data set are included. They are motivated and introduced in Section 4.3.

Metrics Each metric computes a quantifiable measure. They have different bounds, which makes them not directly comparable to each other. These metrics allow for evaluation of a single generative model, but also a comparison between multiple generative models. This enables the assessment of model performance relative to each evaluation dimension. They are also motivated and introduced in more detail in Section 4.3.

Output The framework’s output consists of a JSON file containing the computed metric values in their categories, alongside a set of visualizations. It does not contain a single weighted result value, but the raw metric outputs. These files are then used to analyze the quality of a synthesized data set.

Configuration The framework can be configured to allow for fine-grained control over which aspects are evaluated. Single categories or metrics can be excluded from the evaluation if they are not of concern. Custom metrics can also be passed to TempoBench which will evaluate the metrics alongside the inbuilt metrics directly.

Processing Pipeline A high-level representation of the data flow is given in Figure 4.1 showing the whole processing pipeline when using TempoBench. The first step is to choose a data set of the existing ground truth data sets in TempoBench. From this a generative model is trained. After that, a synthetic data set is sampled from the trained model. With this synthetic data set, TempoBench is invoked with the desired configuration of what categories, metrics, and visualizations are to be computed alongside own custom metrics. Following the computation and processing of the metrics, both the results and visualizations are stored.

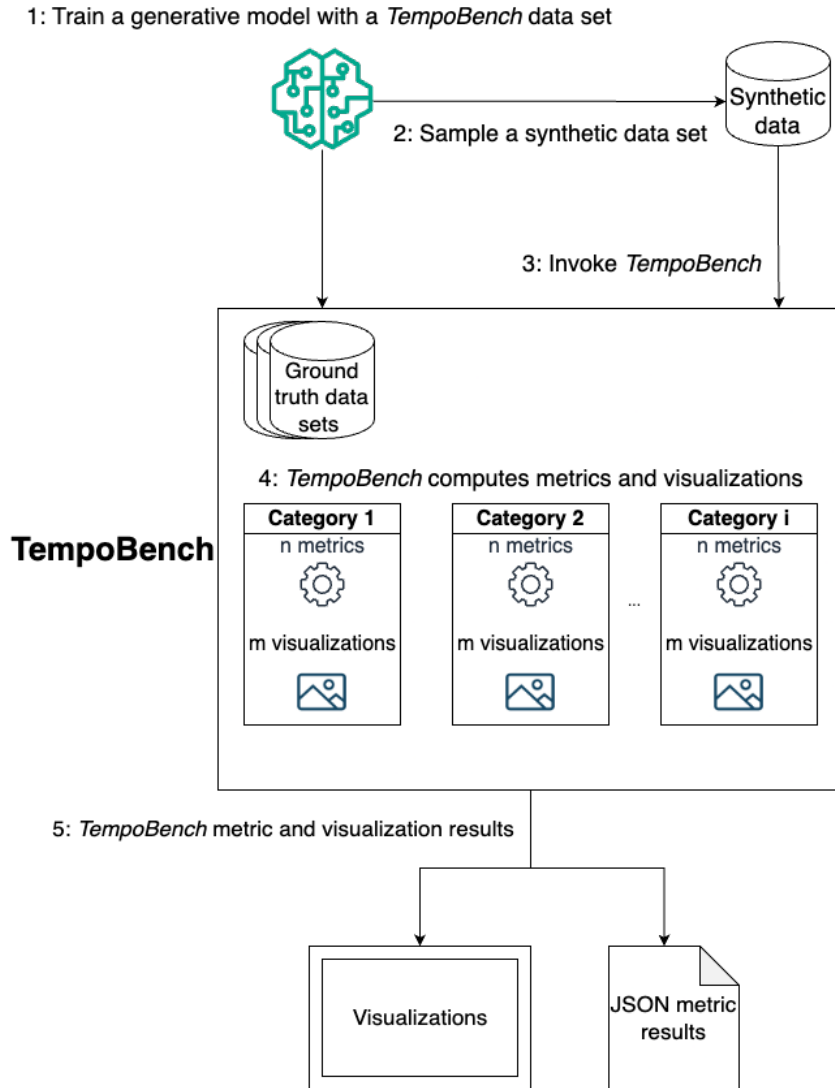


Figure 4.1.: Using a real data set, a generator creates a synthetic data set and invokes *TempoBench* to compute the metrics and visualizations.

In the following, a novel data set that is directly incorporated into *TempoBench* is introduced and discussed. After the presentation of the data set, the distinct categories and their respective metrics are established.

4.2. Transfermarkt soccer data set

As mentioned previously, the first data set to be integrated in *TempoBench* is a soccer data set. The data was joined from multiple tables in the Transfermarkt database into a query result, which is a single table. Duplication was therefore accepted. The goal was to get

4. *TempoBench: A benchmarking framework for synthetic temporal tabular data*

as many different features and especially categorical values. Furthermore, the objective was to have functional dependencies and correlations between columns. This data set will be introduced in this section.

4.2.1. Overview

The data set originates from the Transfermarkt GmbH & Co KG⁵, a German media company that maintains a large soccer database. The data set used in this thesis is an excerpt from this database from a fixed time period. It contains 62.125 tuples representing 4.252 distinct entities. Each entity is a professional soccer player that has played in either the Bundesliga or the Premier League in the time period between 01.07.2010 and 30.06.2020.

For each entity, there exists an ordered sequence of entries along a temporal dimension. These entries are subject to tuple versioning, which means that each has a defined validity period. Two entries of the same entity cannot have the same or overlapping validity dates. The sampling of the entries was performed independently per entity based on the events that are explained in Subsection 4.2.2.

Overall, the data set can be characterized as multivariate, multi-entity, irregularly sampled temporal data with mixed-type data. The 33 columns include temporal (i.e. dates), categorical as well as numerical attributes. The distribution of feature types is the following:

- Numerical: 11
- Temporal: 4
- Categorical: 18

Furthermore, there are 9 static attributes (such as name or position), while the remaining 24 attributes can change over time.

As the data set does not exist in this form in the database, preprocessing had to be performed. As these preprocessing steps require access to the Transfermarkt database, the raw data set will not be published alongside the processed dataset. The preprocessing includes, for instance, setting the validity dates of each tuple or calculating the performance data at the time of the event such as the amount of goals up to this date. This information requires direct access to the Transfermarkt database. The preprocessing will be explained in more detail in the subsequent Subsection 4.2.3.

4.2.2. Sampling

The sampling of the data was done based on discrete events, most of which apply only to a subset of players per event. An exception is a regular, fixed sampling performed on July 1st of each year for all players active in the Bundesliga or Premier League at that time. The data set includes only players who were part of the Bundesliga or Premier League between July 1 2010 and June 30, 2020.

The following events summarized in Table 4.1 were used to sample a new entry for a player:

⁵<https://www.transfermarkt.com/>

Table 4.1.: Reasons for entry creation and their descriptions

Reason	Description
Transfer	A new entry is created whenever a player transfers to or from a club in the Bundesliga or Premier League. If a player subsequently transfers between clubs outside these two leagues, no new entry is generated.
Injury	A new entry is created whenever a player sustains an injury while playing in the Bundesliga or Premier League.
Injury end	A new entry is created whenever a player's injury ends while playing in the Bundesliga or the Premier League.
Market value update	A new entry is created whenever the Transfermarkt market value of a player in the Bundesliga or Premier League is updated. It can increase, decrease, or be confirmed at the current level.
New coach	A new entry is created for all players of a club in the Bundesliga or Premier League whenever their coach changes.
Regular interval	A new entry is created for every player active in the Bundesliga or Premier League on July 1 of each year from 2010 to 2019, representing the start of each season.

This sampling resulted in raw data that required preprocessing. As will be explained in the next subsection, this involved merging entries of the same entity that share the same date and incorporating additional data corresponding to the time of the event.

4.2.3. Preprocessing

During preprocessing, entities were chronologically ordered based on the temporal dimension that marks the start of the validity of the entries. The derived features, such as the age at the time of the event, were calculated and the entries sampled on the same date (for example, an injury and a market value update occurring on the same day) were merged into a single entry. A selection of preprocessing steps is presented in Table 4.2, while the complete list can be found in Appendix A.1.

4. *TempoBench: A benchmarking framework for synthetic temporal tabular data*

Table 4.2.: Preprocessing steps and their descriptions

Preprocessing Step	Description
Entry validity	Start and end dates indicating the validity period of each entry were added based on the sampling timestamps. This makes the data set tuple-versioned.
Merge same-day entries	Entries sampled on the same date for the same entity were merged into a single record. For example, if a player sustained an injury and transferred to the Bundesliga on the same day, both events are represented within the same entry. This prevents illegal states within entities where overlapping validity dates occur.
Injury categorization	Due to the presence of over 300 distinct injury types, injuries were grouped into broader categories. The corresponding injury category was added as an additional column, adding a functional dependency to the data set. The functional dependency is $\{\text{injury}\} \rightarrow \{\text{injury_category}\}$

Since the data is maintained by human administrators at Transfermarkt, it may contain errors or inconsistencies. All known faulty data at the time of writing were removed. For example, there were players with injuries where the end of the injury preceded the start of the injury. These entities were excluded from the data set.

4.2.4. Explanation of the features

The Transfermarkt data set includes a variety of data types, such as numerical values, categorical features, and dates. It also comprises a mix of dynamic variables that change over time and static variables that remain constant per entity. The following Table 4.3 presents an excerpt of the variables used in the data set after preprocessing has been completed. The complete list along with explanations can be found in Appendix A.2.

Table 4.3.: Features and their meaning

Name	Description
player_id	Unique identifier of the player in the Transfermarkt database.
last_name	The player's last name, as listed in the Transfermarkt database.

Continued on next page

Table 4.3 – continued from previous page

Name	Description
validity_start	The date on which the event was sampled and the entry's validity period begins, formatted as YYYY-MM-DD.
validity_end	The date on which the entry's validity period ends, formatted as YYYY-MM-DD.
injury_category	The name of the injury category the player is currently sustaining, if applicable. Otherwise, this field is set to null.
league_id	The Transfermarkt identifier of the league in which the player's club competes.
season_id	Identifier of the season, ranging from 2010 to 2019, corresponding to the respective year the season started.
position	The primary playing position of the player (e.g., goalkeeper, central midfielder, ...) as defined in the Transfermarkt database. As a possible change of primary position is not tracked in the Transfermarkt database, it is considered to be a static feature in this work.
club	The name of the club the player plays for. This field is set to null if the player transfers to a club outside the Bundesliga or Premier League.
market_value	The player's Transfermarkt market value at the start of the entry's validity period.
league_goals	Total number of goals the player had scored in the Bundesliga and Premier League up to the start of the entry's validity period.

An excerpt of a single players sequence can be found in Table 6.7. The subsequent subsection will give a brief descriptive analysis of the data set.

4.2.5. Descriptive analysis

The final data set consists of 62.125 rows, representing 4.252 unique players, and comprising 33 columns. In this section, key characteristics of the dataset are examined to provide an initial understanding of its structure and content.

Due to the event-based sampling strategy, the different events have occurred at varying frequencies. As shown in Figure 4.2, the most frequently sampled event is the *market value update* with over 20,000 entries. This is followed by *transfers*, which account for approximately 12,000 entries. The *regular interval* sampling on July 1 of each year contains

4. TempoBench: A benchmarking framework for synthetic temporal tabular data

slightly over 10,000 rows, while both *injury* and *injury end* events contribute around 9,000 events each. The slightly lower count of *injury end* than *injury* entries is due to two factors: If a player recovers from an injury after transferring to a club outside the Bundesliga or Premier League, no entry is recorded. Then, if the injury ends after the cutoff date of June 30, 2020, the injury end is not contained in the data set. The least frequent type of event is *new coach*, which appears in approximately 6,000 rows.

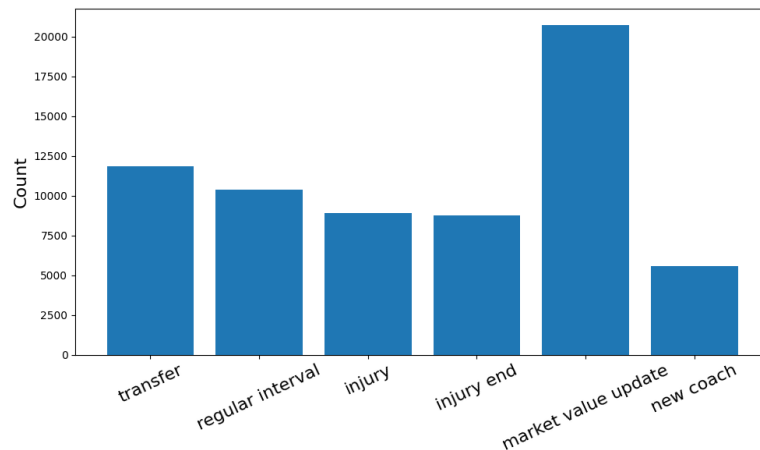


Figure 4.2.: Distribution of reason for sampled events, showing market value update is the most frequent reason. If two events occur on the same date for the same player, both reasons are counted.

Figure 4.3 shows the distribution of the league within the data set. The Premier League has a slightly larger share than the Bundesliga, with over 30,000 entries compared to approximately 25,000 for the Bundesliga. Additionally, around 5,000 entries do not have an associated league, so it is outside the Bundesliga or Premier League. This discrepancy is likely due to the structural difference between the leagues, since the Premier League features 20 participating clubs, whereas the Bundesliga comprises only 18.

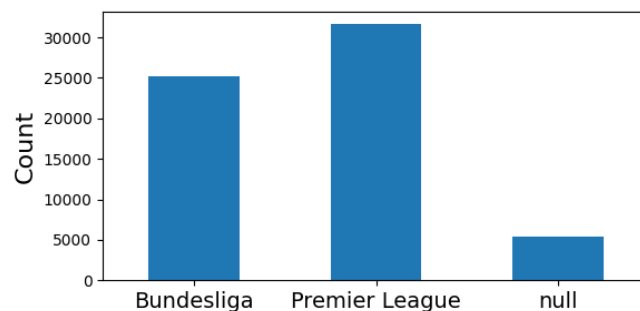


Figure 4.3.: Distribution of leagues showing the Premier League contains more samples than the Bundesliga in the data set.

The sequence lengths in the data set vary greatly, as illustrated in Figure 4.4. While nearly half of the players possess relatively short sequences with fewer than 10 entries, some players record more than 60 sampled events. This variation can be explained by the fact that players may retire or transfer to clubs outside the Bundesliga or Premier League early in the observation period. The longest sequence in the data set belongs to Marko Reus, with a total of 141 entries. This is due to his presence in the Bundesliga throughout the entire observed period and a history of frequent injuries.

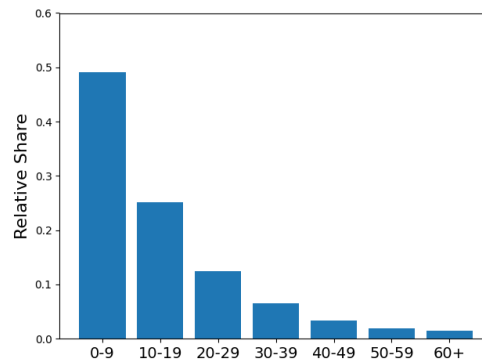


Figure 4.4.: Distribution of sequence length showing most sequences contain less than 20 entries.

The data set reveals several temporal trends, one of which is the change of player market values over time, as illustrated in Figure 4.5. The Premier League consistently shows higher average market values compared to the Bundesliga, although both leagues follow a similar overall trend. Initially, market values decline between 2010 and 2012, but from that point onward they show a steady increase, continuing until the start of the 2019 season.

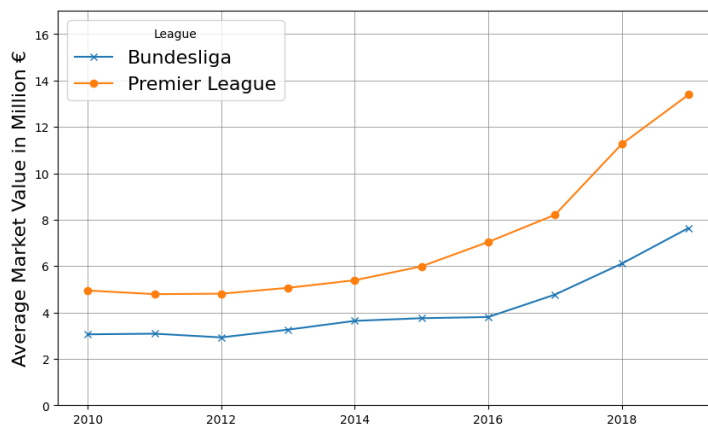


Figure 4.5.: Average market value development in the Bundesliga and Premier League per season. A general upward trend can be seen over the observed time period in both leagues.

4. *TempoBench: A benchmarking framework for synthetic temporal tabular data*

Next, the proposed evaluation categories and metrics for TempoBench are presented and discussed.

4.3. Evaluation Metrics

This section outlines the evaluation approach employed in the benchmarking framework, introducing both the categorization of evaluation aspects and the specific metrics assigned to each category. The structure begins with a conceptual overview of the evaluation categories, followed by a detailed description of the individual metrics used within each category.

4.3.1. Categories

The selection of metric categories within the *TempoBench* framework is largely guided by the existing literature, as discussed in Section 3.1, where evaluation metrics commonly used in related work are reviewed.

Sanity: A foundational category is *sanity*, inspired by the Synthcity project [122], which includes basic checks to ensure that the synthetic data contains the same data types and does not just copy the original data.

Domain-specific: To take into account the unique characteristics of each data set, *domain-specific* metrics are introduced in TempoBench. These metrics address dataset-specific rules and constraints that are not shared between data sets.

Statistical fidelity: Another central aspect is *statistical fidelity* that includes metrics assessing whether core statistical characteristics of the original data set are preserved in the synthetic data. These metrics serve as fundamental indicators of the overall quality data of the data synthesis, such as the distribution of the columns or inter-column relationships.

Temporal fidelity: To capture the temporal dynamics inherent in the data set, a category *temporal fidelity* is included. This category contains metrics that evaluate whether temporal characteristics and patterns are preserved in the synthetic data. For example, it assesses whether updates across columns occur jointly over time, as captured by metrics like the update dependency, or if sequences of categories are the same in the synthetic data as in the ground-truth data.

Downstream utility: The *downstream utility* category reflects the understanding that synthetic data is often used for subsequent analysis and tasks. It is thus not an end goal in itself, but rather an intermediate step for further usage. Therefore, this category evaluates how well synthetic data performs in subsequent tasks such as regression or classification tasks, using the same models as would be applied to the real data.

Visualizations: Finally, visualizations are incorporated into the framework to allow for a qualitative understanding of the synthetic data's fidelity. These visualizations support exploratory comparisons across multiple dimensions such as statistical or temporal fidelity and are therefore not confined to a single aforementioned category. Consequently, visualizations are treated as a separate component of the framework. Each visualization is still part of a category.

It is important to note that other commonly used benchmark categories, such as privacy, are

not included in this framework. The primary focus is on evaluating how well the generative models can capture the real data characteristics, and privacy-related metrics fall outside of this scope, even though considered important. While implemented metrics such as Incoming Nearest Neighbor Distance can be considered to also cover a privacy aspect, it was not the focus. In summary, the components of the framework are the following:

- Sanity
- Domain-specific
- Statistical fidelity
- Temporal fidelity
- Downstream utility
- Visualizations

Choosing the metrics As discussed in Chapter 3, a wide range of evaluation metrics for assessing synthetic data quality has been proposed in the literature. To determine which metrics to include in the benchmarking framework, each candidate metric was evaluated against the following criteria:

- **Alignment with evaluation objectives:** The metric must be relevant to at least one of the defined evaluation categories.
- **Interpretability:** The results computed by the metric should be easy to interpret and support drawing meaningful conclusions.
- **Computational feasibility:** The metric must be computationally efficient when applied to the Transfermarkt data set and a synthetic data set of similar size.
- **Reproducibility:** The metric should be deterministic or exhibit controlled randomness to ensure consistent results across multiple runs.

In the following, each metric is presented with its name and category, followed by a bracketed number indicating the corresponding evaluation challenge (1–3) it addresses, as defined in Figure 1.1.

4.3.2. Sanity

Sanity metrics serve as basic integrity checks that the synthesized data does not contain rather obvious issues. They are domain-agnostic and are applicable to any data set. They are also featured in existing evaluation tools, such as Synthcity by Qian et al. [122].

4. *TempoBench: A benchmarking framework for synthetic temporal tabular data*

SAN1 Duplicate rows (1): The duplicate rows metric scans the synthesized data set and compares it to the original data set to find the proportion of rows of the synthesized data set that are exact copies of the original data. This metric is adopted by Meehan et al. [99] who introduced the data copying test. It is computed the following way:

$$\text{DuplicationRate} = \frac{|D_{\text{syn}} \cap D_{\text{orig}}|}{|D_{\text{syn}}|} \quad (4.1)$$

The duplication rate is the intersection between the synthetic data set D_{syn} and real data set D_{orig} , divided by the total number of rows in the synthesized data set. A high duplication rate may indicate overfitting or lack of generalization in the generative model.

SAN2 Datatype mismatch (1): This metric compares the data type of the columns of the original data set to the data type of columns of the synthesized data set. It is a metric adopted from the Synthcity framework [122]. It is calculated the following way:

$$\text{DataTypeMismatchRate} = \frac{|\{c \in C \mid \tau_{\text{syn}}(c) \neq \tau_{\text{orig}}(c)\}|}{|C|} \quad (4.2)$$

Where C is the set of features and τ returns the Python data type of the feature in the respective data set. It will calculate the proportion of columns that have a different datatype than the one in the original data set. For instance, if the column `height` contains a mix of strings and numbers in the synthesized data set instead of only floats, this discrepancy will be found by this metric. A high data-type mismatch indicates basic problems in the generative models failing to use the data types from the real data.

SAN3 Tuple validity overlap (2): For historical databases, within the given interval from the data set, each entity may have at most one valid record in any point in time. This means the `validity_start` must always be greater than `validity_end` of the predecessor in the entity. To quantify violations, all entries are examined except the first in each entity's ordered sequence (since the initial record has no predecessor) and count those whose interval overlaps with its immediate predecessor.

The violation rate is then calculated as the proportion of these overlapping entries among all non-first records. A higher rate indicates that the generator has failed to capture proper temporal succession within individual entity timelines, and thus reflects poorer model performance.

SAN4 Validity end before start (1): For historical databases, each entries `validity_start` and `validity_end` define an interval in which the entry is valid. It requires `validity_start` $\leq$ `validity_end` for all entries, which means that the entry is valid from the `validity_start` until the `validity_end`. This metric measures the proportion of rows in the synthesized data set for which `validity_end` $<$ `validity_start`. A high violation rate indicates that the generator has not learned this fundamental temporal ordering within the individual entries. Consequently, larger values of this metric correspond to poorer model performance.

4.3.3. Domain-specific

Domain-specific metrics include metrics that incorporate domain expert knowledge and constraints stemming from the semantics of the data set. These constraints reflect rules that must hold true within the context of the data set and are therefore not directly applicable to other data sets. Therefore each data set that is introduced in TempoBench creates their own domain-specific metrics. These metrics aim to detect violation of domain logic and structural consistency that cannot be captured by purely statistical measures. The following apply to the introduced Transfermarkt data set.

D1 Manual inspection (1-3): While manual inspection of sequences is not feasible on a large scale, inspecting a small subset of sequences or even excerpts of sequences is enough to already find deviations from the ground truth sequences. Therefore the manual inspection is also part of the TempoBench framework, that allows for insights that might not directly be covered by the other metrics. It can also yield canonical examples where the generative models failed to follow a domain constraint. This is not necessarily a quantifiable metric and also applies to all data sets, but requires domain knowledge. It does not require explicit implementation when evaluating, but just extracting sequences from the data sets.

D2 Monotonic increase (2): A set of columns C_{mon} that, include for instance `league_played_matches` or `league_minutes_played`, are expected to stay the same or increase in the temporally ordered sequence for the same player. This metric computes the proportion of rows and entities that violate this monotonicity constraint at least once. It is computed by comparing the values of entry e with the ordered sequence index t in C_{mon} for the corresponding values of the preceding entry in the sequence with $t - 1$ if there exists a predecessor. For each c in C_{mon} a comparison is then done where $e_{tc} \geq e_{t-1c}$ has to hold. Otherwise a violation is recorded.

The result is the proportion of rows that include a violation and the proportion of entities that include a violation, which show different detail. The proportion of entities violated gives a rough overview, while the proportion of rows signals if the constraints are violated throughout the entities lifecycle or just for single entries. Furthermore, the proportion of violations per inspected column is reported.

A high rate of violations may indicates that the generator failed to capture basic temporal dependencies and domain rules. It was not able to set into relation those numerical values within the temporal ordering. The higher the proportion of violated rows, the worse does the model perform.

D3 Static data remains unchanged (3): There exist a set of columns that are considered to be static and therefore do not vary over time for an entity. Examples include `date_of_birth` and `foot`, which remain unchanged throughout the player's trajectory because they cannot change as modeled in the Transfermarkt data set. This metric checks whether such attributes remain the same across all time steps for each entity. It computes the proportion of entities that exhibit at least one inconsistency in any of the considered features. A high rate

4. *TempoBench: A benchmarking framework for synthetic temporal tabular data*

of violations indicates that the generator failed to learn these temporally invariant features. The higher the proportion of violated entities, the worse does the model perform.

D4 Timestamp minimum and maximum (1): The ground-truth data set is defined only over the interval from 01.07.2010 to 30.06.2020. No entries outside these dates are considered. We apply the same constraint to the synthesized data, requiring that both `validity_start` and `validity_end` fall within this time window. This metric is then calculated as the proportion of rows whose timestamps fall outside the allowed range. A high violation rate indicates that the generator has not properly learned the data sets temporal scope, producing data points outside the scope.

D5 Maximum validity time (1): The data set’s overall temporal scope spans exactly ten years, from 01.07.2010 to 30.06.2020, so no single entry can be valid for more than this period. We define the validity duration of an entry i as

$$\Delta_i = \text{validity_end}_i - \text{validity_start}_i$$

This metric calculates the proportion of rows in the synthesized data set that violate the constraint and have a validity period exceeding 10 years. A high rate of violation indicates that the generator has failed to respect the upper-bound temporal constraint, resulting in poorer model performance.

D6 Functional dependencies (1): The ground truth data exhibits a variety of functional dependencies (FDs), from simple one-to-one mappings to more complex relationships, including a temporal dimension. For example a simple time-invariant mapping is

$$\{\text{coach_id}\} \rightarrow \{\text{coach}\}$$

and, incorporating a more complex relationship including a temporal dimension,

$$\{\text{season_id}, \text{club}\} \rightarrow \{\text{international_competition}\}$$

To evaluate whether these dependencies are reproduced in the synthetic data set, we first extract the set of FDs $\mathcal{F}_{\text{orig}}$ from the ground-truth data and the set $\mathcal{F}_{\text{syn}}$ from the synthetic data. To quantitatively evaluate the approximation of the synthesized data to the ground truth data with respect to the FDs, precision, recall and F1 score are calculated [51]:

$$\text{Precision} = \frac{|\mathcal{F}_{\text{orig}} \cap \mathcal{F}_{\text{syn}}|}{|\mathcal{F}_{\text{syn}}|}, \quad \text{Recall} = \frac{|\mathcal{F}_{\text{orig}} \cap \mathcal{F}_{\text{syn}}|}{|\mathcal{F}_{\text{orig}}|}, \quad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Low precision indicates dependencies in the synthetic data that are not present in the real data set, while low recall reveals missing relationships that were present in the original. F1 combines the two scores into a single metric considering both. A low F_1 -score therefore indicates the generative model’s failure to enforce domain-specific functional dependencies.

D7 Reason for new tuples (1): As described in Section 4.2.2, each entry with index j contains one or more sampling reason where the following have an impact on the entries valid values:

$$R = \{\text{injury, injury end, transfer, regular interval, new coach}\}$$

For each reason $r \in R$ there are domain-specific constraints C_r that apply to the entry in the temporally ordered sequence with index j :

$$\begin{aligned} C_{\text{injury}} &: (\text{injury} \neq \text{NULL}) \wedge (\text{injury_category} \neq \text{NULL}), \\ C_{\text{injury end}} &: (\text{injury}_j = \text{NULL}) \wedge (\text{injury_category}_j = \text{NULL}) \wedge \\ &\quad (\text{injury}_{j-1} \neq \text{NULL}) \wedge (\text{injury_category}_{j-1} \neq \text{NULL}) \\ C_{\text{transfer}} &: \text{club_id}_j \neq \text{club_id}_{j-1}, \\ C_{\text{regular interval}} &: \text{validity_start}_j \in \{01 \text{ July of any year}\}, \\ C_{\text{new coach}} &: \text{coach_id}_j \neq \text{coach_id}_{j-1} \end{aligned}$$

A violation is then present if the constraint of the reason for the entry does not apply to the entry. The proportion of rows that violate at least one of the constraints are then calculated. A higher rate of violation indicates the generator failed to adhere to the domain rules given in the Transfermarkt data set.

D8 Consistency across players for new coach (2): When a club transfers to a new coach at time t_c , every player active at that club at t_c should have an entry whose `validity_start` = t_c and contain matching `coach_id` and `coach` values. To calculate the violation rate, this is computed for all unique coach transfers in the data set. A higher rate indicates that the generator failed to enforce consistent, simultaneous coach changes across all affected players. If the constraint is violated, it would mean that a club has a different coach depending on what player is inspected.

D9 Derived column consistency (1): The column `age` is derived from two columns `validity_start` and `date_of_birth` for each entry in the data set. This metric validates whether the age is correct at the time of the `validity_start`. A high proportion of violation indicates the generative model lacks an understanding of the mathematical relationship between these features.

D10 Maximum increases (2): According to the Transfermarkt data set, a player's maximum playtime in a Bundesliga or Premier League match is 90 minutes. International matches such as in the Champions League or Europa League have a maximum playtime of 120 minutes. This metric computes the violations in too large increases in the performance data taking into consideration the previous entries values for the performance data and the current performance data.

The proportion of violations for rows and entities is then calculated. A high rate of violation indicates the generator failed to learn the relationship between the performance data columns

4. *TempoBench: A benchmarking framework for synthetic temporal tabular data*

and their possible increase per match. The higher the proportion of violations, the worse is the performance of the generative model.

D11 Invalid values (1): Invalid values include individual constraints per column. While the sanity metrics check for compatible data types, the actual value ranges are not validated. The following list is an excerpt for the ranges values can have:

- `goals_scored`: $[0, \infty)$
- `season_id`: $[2010, 2019]$
- `international_competition`: $\{'CL', 'EL', ''\}$

The proportion of rows and entities that violate these constraints is computed. A high rate of violation means that the generator did not learn the valid boundaries of the data and may be generating invalid or previously unseen category values.

D12 Logical ordering (2, 3): Next to the monotonic increase in performance data, there also exists a logical ordering for the reason the sampling was performed. For the injury feature, there exists a logical ordering that dictates when injuries and their corresponding end events can logically appear within the sequence of a player. Specifically, two constraints must hold:

- **Injury after injury end:** An injury event can only occur if there was no previous injury in the sequence or if the previous injury has already ended.
- **Injury end after injury:** An injury end can only occur if there was a previous injury that has not yet ended.

This metric computes the proportion of entities and rows that violate this constraint. A high rate of violation indicates that the generator did not learn this logical ordering of events. The proportion of violations per rule is calculated. It is then averaged across all rules, while still reporting the violation per rule to allow fine-grained insights. These domain violations indicate the generators' inability to approximate the domain in a meaningful way. The higher the proportion of violations, the worse does the model perform.

4.3.4. Statistical fidelity

Statistical fidelity measures the fulfillment of the synthesized data to preserve the statistical properties and distributions of the original dataset [117]. This includes basic statistics, univariate distributions, and multivariate relationships such as correlation.

The evaluation can be applied to the full data set or also be extended to include domain-specific subsets, such as individual seasons or player positions in the Transfermarkt data. Additionally, they can be evaluated for a specific point in time, such as a snapshot of the database. This allows for more granular insights into where the generator works well or fails to capture the original data sets details. Ensuring the statistical fidelity to the original data set makes it useful for downstream tasks or analysis [117].

S1 Elementary statistics (1): Elementary statistics provide a high-level overview that compares the basic properties of the original and synthetic data set. These statistics are computed for both numerical and categorical features. They are adopted from the TabSynDex metric proposed by Chundawat et al. [26].

For numerical values, the metric computes the deviation between real and synthesized data sets with respect to the mean, median and the standard deviation alongside their respective actual values. Additionally, the minimum and maximum values for each column are reported to allow for easy comparison of the value range and to detect discrepancies in the coverage of the feature space. The relative deviation to the ground truth values is also indicated for comparing across features easily. The average of these deviations is then calculated.

For categorical variables, the mode is compared directly and the normalized Shannon entropy is computed to assess the difference in category distribution between real and synthetic data. The Shannon entropy is calculated the following way [69]:

$$H(X) = - \sum_{x \in \mathcal{X}} P(x) \log P(x) \quad (4.3)$$

where X is a discrete random variable. The proportion of categorical features sharing the same mode in both the original and synthetic data is determined, followed by the calculation of the entropy deviation from the ground truth.

These elementary statistics are computed for the whole data set and then also for subsets grouped by season for the Transfermarkt data set specifically. This allows for a deeper insight into the data and see if the changes over the domain's temporal dimension are also reflected in the synthesized data set.

Big differences in these elementary statistics can already indicate possible aspects the generator failed to cover. For instance, if the ranges for the numerical values differ greatly, this shows that the generator did not learn to synthesize this attribute properly.

S2 Univariate distribution divergence (1): To measure how well the synthesized data approximates the marginal distributions of the real data set, this metric compares the univariate distributions of each applicable feature independently. For example, it is used by Li et al. [90] when proposing and evaluating the TTS-GAN. The Jensen-Shannon (JS) divergence is used as a bounded measure of similarity between distributions. It yields a value in the range $[0, 1]$ where 0 means the distributions are identical and 1 are completely different distributions. Another advantage is that it can deal with cases where different values within a category are present in the data set. It is calculated the following way:

$$\text{JSD}(P \parallel Q) = \frac{1}{2} D_{\text{KL}}(P \parallel M) + \frac{1}{2} D_{\text{KL}}(Q \parallel M) \quad \text{with} \quad M = \frac{1}{2}(P + Q) \quad (4.4)$$

P and Q are the respective ground-truth and synthetic distributions of a feature and D_{KL} is the Kullbach-Leibler divergence [23]. This metric is applied to each individual applicable feature, whether numerical or categorical, without considering temporal dependencies and averaged afterwards. A high divergence value indicates that the generative model failed to capture the true distribution of the features. In particular, consistently high divergence across many features may suggest issues such as mode collapse in GANs.

4. *TempoBench: A benchmarking framework for synthetic temporal tabular data*

S3 Bivariate distribution divergence (1): To measure how well the interplay between variables is represented by the generator and stays true to the original bivariate data distributions, this divergence is measured.

- For numerical-numerical distributions, the Jensen-Shannon divergence can be used as in the univariate case. This again yields values in the range $[0, 1]$ and is therefore easily interpretable and standardized.
- For numerical-categorical distributions, the Jensen Shannon divergence can be compared per category between the original and synthesized data set and averaged. Again, it yields values in the range $[0, 1]$ and is easily interpretable.

If the difference between the bivariate distributions is large, this indicates a failure of the generator to correctly generate the relationships between variables and how they occur together.

S4 Column correlation (1): Inter-column correlations provide insight as to whether the generator has learned to capture dependencies between features. This metric is adapted by the correlations computed in TabSynDex [26]. This metric evaluates how well the correlations between features in the synthesized data set approximate the ones in the original data set. Different correlation measures are used depending on the data type of the variable pairs:

- **Numerical-Numerical:** The Pearson correlation coefficient is used, as it allows for easy interpretation, is standardized in $[-1, 1]$ and widely used in other works [26], [162]. It is calculated as follows:

$$r_{X,Y} = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^n (X_i - \bar{X})^2} \sqrt{\sum_{i=1}^n (Y_i - \bar{Y})^2}} \quad (4.5)$$

- **Numerical-Categorical:** Eta-Squared [27] is used for the association between numerical and categorical features. It is used as it's easily interpretable and normalized. It has the range $[0, 1]$ and is calculated the following way:

$$\eta^2 = \frac{\sum_{k=1}^K n_k (\bar{Y}_k - \bar{Y})^2}{\sum_{i=1}^n (Y_i - \bar{Y})^2} \quad (4.6)$$

where K is the total number of categories and n is the total number of samples across all categories. n_k denotes the number of observations in category k . $\bar{Y}_k$ is the mean of category k and $\bar{Y}$ is the overall mean of the numerical variable.

- **Categorical-Categorical:** Cramér's V [28] is used to measure the degree of association between two categorical features. It is used in other works [162] and is easily interpretable as it is in the range $[0, 1]$. It is calculated the following way:

$$V = \sqrt{\frac{\chi^2}{n(\min(K_r, K_c) - 1)}} \quad (4.7)$$

where χ^2 is the chi-squared statistic computed from the contingency table of the two categorical features and K_r and K_c are the number of categories in each respective feature.

To quantitatively compare the correlation structure of the synthesized data with the real data directly, the metric calculates the L1 distance between the correlation matrices. Since the Pearson correlation ranges from $[-1, 1]$ and the other two measures are bounded in $[0, 1]$, the Pearson values are linearly transformed to $[0, 1]$ using the mapping $x \mapsto (x + 1)/2$ to ensure scale consistency. This means that each feature can only have a maximum distance of 1, instead of the numerical values having a maximum value of 2.

These correlations are computed globally for the whole data set, as well as per player position (goalkeeper, striker, ...) specifically for the Transfermarkt data set. This positional breakdown allows detecting whether the model captures domain-specific relationships. For instance, the correlation between matches played and goals scored is expected to differ between a goalkeeper and a striker.

In addition to the overall matrix distance, per column pair deviation between real and synthetic correlation scores are given to allow fine-grained evaluation of the synthesized data.

S5 Precision score (1): The precision score, as proposed by Sajjadi et al. [126], evaluates whether samples from the synthetic data fall within the support of the real data distribution. This metric is used to assess how well individual synthetic rows resemble realistic samples, making it suitable for detecting anomalous rows in the synthesized data.

More formally, precision quantifies the relative share of synthetic data points that are near clusters of real data samples. First, the samples are embedded in a feature space $\mathbb{R}^k$. The ball operator B that defines the radius of the closest K nearest neighbors in the real data set needs to be defined. A synthetic sample is then considered precise if it falls within the radius of a ground-truth data sample. The precision score is calculated as follows [139]:

$$p_i(D_{orig}, D_{syn}) := \frac{1}{|D_{syn}|} \sum_{\hat{x} \in D_{syn}} \mathbf{1}\{\exists x \in D_{orig} : \hat{x} \in B(x, d\text{NN}_K(x, D_{orig}))\} \quad (4.8)$$

where $B(x, d\text{NN}_K(x, D_{orig}))$ defines the ball around each real sample to its K nearest neighbors.

The precision score is then the proportion of samples that lie within the radius of a real sample. The score is bounded in $[0, 1]$, where 0 means that all samples are far from the real samples, while 1 means all samples fall within the radius of the real samples.

This metric functions similarly to an anomaly detector by catching synthetic rows that deviate from the patterns present in the original data. High precision indicates that the generative model produces realistic and high-quality samples.

S6 Train-on-Real, Test-on-Synthetic (TRTS) (1): Esteban et al. [43] propose this Train-on-Real, Test-on-Synthetic metric which is adopted in this framework. The real data is split into train and test sets. A model is then trained for a task on real train data, such as a classification or prediction task.

4. *TempoBench: A benchmarking framework for synthetic temporal tabular data*

Afterwards, it is evaluated on the held-out real test set and also on the synthetic data set. If the performance of the model for the synthetic data is similar to the performance on the real data, this means the generative model is able to generate realistic samples, as it is assumed the features have been learned properly. If performance is much worse than on the real test set, it can be assumed that essential characteristics of the data have not been properly learned.

S7 Discriminative score (1): The discriminative score is adopted from Yoon et al. [164]. The real and synthetic data are merged into a single labeled data set, indicating if the row is real or synthetic. It is also split into a train and a test set.

A binary classifier is then trained on this train data set to discriminate between individual rows whether they belong to the real or synthetic data set. The classification error of the trained model on the held-out test set is then used to evaluate how well individual rows are approximated by the generative model. The higher the classification error, the more indistinguishable the synthetic from the real data.

This score can be used to evaluate how well individual rows are sampled by the generative model and can also be seen as an anomaly detection. A low classification error rate of the binary classifier in discriminating between real and synthetic data indicates the generators inability to create realistic rows.

Further There exist further metrics that compute aspects of statistical fidelity. These include but are not limited to comparing clusters of real and synthesized data [152] or calculating the recall similar to the precision score [83]. If the data set should contain of real-synthetic pairs, other metrics are applicable as well. Examples for these metrics include calculating the RMSE between sequences [139] and Percent Root Mean Square Difference [59].

4.3.5. Temporal fidelity

Temporal metrics evaluate how well the synthesized data preserves temporal characteristics inherent to temporal data [43]. These can be short or long-term. They include within-entity dependencies such as update dependencies between columns. Other aspects include the length of the sequences generated and the mean time each entry is valid in the synthesized data in comparison to the original data. But they also include comparisons of sequences found in the synthesized and original data.

T1 Incoming nearest neighbor distance (1, 2): Incoming nearest neighbor distance (INND) was proposed by Arnout et al. [10] and measures the minimum distance of a synthesized time series to an original sequence with a distance function. Depending on the type of sequences present, different distance functions can be used. It is calculated the following way [139]:

$$\text{INND} = \frac{1}{N} \sum_{i=1}^N \min_j d(\hat{x}_i, x_j), \quad (4.9)$$

where x_j is from a subset of the real data set D_{orig} , $\hat{x}_i$ are sequences from a subset of D_{syn} and $d(\cdot, \cdot)$ is a distance function.

Because the Transfermarkt data set is irregularly sampled and has non-aligned timestamps, we use Dynamic Time Warping (DTW) as $d(\cdot, \cdot)$, following Stenger et al. [139]. DTW finds an optimal warping path between two sequences, allowing for a non-aligned temporal axis [29]. Similarly to Choi et al. [24], a custom Dynamic Time Warping distance measure is used to allow not only comparison of numerical values but also categorical values. For numerical values, the values are normalized, and Euclidean distance is used as in the standard Dynamic Time Warping. For categorical values, the Hamming distance [54] is used and a weight of 0.3 is given if the categories are different.

After computing the INND for each sequence $\hat{x}_i$, it is averaged across all sequences. A single scalar value is returned, where the smaller the distance, the better [139].

The higher the INND, the further the synthesized sequences deviate from real sequences. Depending on the goal of the synthesis, this can be interpreted in different ways. However, in general, the lower the value, the better. For computational efficiency, a subset of the sequences can be used here, and therefore not all sequences are considered in this metric.

T2 Mean sequence length (1): This metric compares the mean sequence length per entity in the synthesized data set to that of the original data set. Sequence length refers to the number of temporally ordered entries associated with a single entity.

By reporting only the mean values, this metric provides a quick indication of whether the generative model respects the temporal structure of the data. Significant deviations in the mean sequence length may suggest that the model failed to preserve the temporal relationships between rows.

T3 Mean validity (1): This metric compares the mean validity of each entry in the synthesized data set with the original data set. This only applies to data sets that not only have a sequence index, but also include validity features.

For the Transfermarkt data set, validity refers to the absolute time between the `validity_start` and `validity_end` of each entry.

Only the mean values and the deviations from the ground-truth data are reported, allowing for a quick indication of whether the generative model respects the temporal structure of the data. If the synthesized mean validity differs greatly, this can indicate that too few or too many entries per entity are in the data set.

T4 Intra-class distance (1, 2): The intra-class distance (ICD) measures how similar sequences within the same data set are. It is an adopted metric from Leznik et al. [89] and is calculated as follows:

$$\text{ICD} = \frac{\sum_{x_j \in D_{syn}} \sum_{x_k \in D_{syn}} d(x_j, x_k)}{|D_{syn}|^2} \quad (4.10)$$

Sequences in the data set are compared with each other using a distance function similar to the INND metric. Again, the distance function used for the Transfermarkt data set is

4. *TempoBench: A benchmarking framework for synthetic temporal tabular data*

the DTW mentioned in the INND metric originally used by Choi et al. [24]. This is a complementary metric because the generator is supposed to create a diverse range of entities, not just a single canonical entity trajectory. While not performed by Leznik et al. [89], the ICD of the synthetic data can also be compared to the ICD of the real data set, allowing us to compare the variability of the synthetic data with the ground-truth data.

A low ICD might indicate the failure of the generator to synthesize the whole range of values and, for example, be an indicator of mode collapse in GANs. For computational efficiency, a subset of the sequences are used here, and, therefore, not all sequences are considered in this metric. If the goal is to synthesize samples similar to ground-truth data, an ICD similar to the ground-truth data is also a mark of quality.

T5 Update-Dependency (2): Update dependency mining treats each row update as a transaction of feature names and identifies which columns tend to change together using association rule mining [1]. Given a minimum support and confidence threshold, mining a data set yields the association rules R_{orig} for the original data set and R_{syn} for the synthetic data set. For example, in the Transfermarkt data set, it will yield an association rule $\{league_goals\} \rightarrow \{league_minutes_played\}$, which means that if the goals scored in the league change, the played minutes will also change with a certain confidence. We then evaluate how well the synthetic data preserves these update dependencies with

$$\text{Precision} = \frac{|R_{orig} \cap R_{syn}|}{|R_{syn}|}, \quad \text{Recall} = \frac{|R_{orig} \cap R_{syn}|}{|R_{orig}|}, \quad F_1 = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

A high F_1 score indicates that the update dependencies between the columns are adhered to correctly in the synthetic data. A low precision means that there exist update dependencies that are not present in the real data. A low recall indicates that the synthesized data set did not include many rules that are present in the real data.

T6 Transition matrix distance (2, 3): For each non-static categorical feature, we build a transition matrix that records how often each category changes (or remains the same) from one time step to the next (or any chosen lag) in both the original and synthesized data sets. Each matrix entry is normalized by the total number of transitions so that each row sums up to one. We then compute the $L1$ distance between the two matrices for each feature to quantify how much the synthetic transitions deviate from the real ones.

We report both the raw and normalized distances for each feature, as well as an average across all features. A higher distance indicates that the generator failed to reproduce the real temporal transitions of that categorical feature. This metric can be computed for arbitrary lag values.

T7 Update frequency (2): This metric assesses whether the synthetic data approximates the same frequency of updates as the real data by comparing the distributions of time intervals between consecutive updates for each entity. For each row, we compute the elapsed time (e.g. days) between its timestamp and the previous update of that entity. This yields two distributions, one from the original data set and one from the synthetic data set.

To quantify their difference, we use the Jensen-Shannon divergence previously introduced in the statistical metrics. For the Transfermarkt data set, the days between the `validity_start` of consecutive entries of an entity is used.

A higher divergence indicates that the generator did not accurately learn the typical spacing between updates, while a low divergence shows that synthetic updates occur with a timing similar to that of the real data.

T8 Discriminative score for sequences (1-3): This metric is adapted by the discriminative score of Yoon et al. [164] to evaluate entire sequences rather than individual rows. The entire time series of each entity is treated as one labeled example, and all sequences are merged into a single data set with binary labels indicating whether the sequence is real or synthetic. This data set is then split into training and holdout test sets at the sequence level.

A binary classifier is trained on the training set to distinguish real from synthetic sequences. The trained model's classification error on the test set is then evaluated. A high error (close to 50 %) indicates that synthetic sequences are hard to distinguish from real ones, while a low error shows that the generator fails to capture the time series.

This metric measures how convincingly the model reproduces the full sequence of entities and can also act as a structured anomaly detector for implausible synthetic sequences.

T9 Inter-row differences (2): This is a adopted metric from Suh et al. [140] who compare the differences of the numerical values for each time step in the original and synthetic sequences. This metric can only be applied to time series that are sampled regularly in equally spaced intervals, as irregularly sampled time series have varying spaces between entries. For each non-static feature, the observed differences can be mapped to a distribution for the real and synthetic data. Then the distributions can be compared with the Jensen-Shannon divergence. An average across all numerical distribution differences is then computed. Higher discrepancies indicate that the generative model did not learn the temporal dependencies between numerical values within the sequences.

Further metrics Further metrics can also include measuring the autocorrelation as done by Wiese et al. [155]. Adding Outgoing Nearest Neighbors (ONND) can complement the INND metric, indicating mode collapse if there is a low INND but a high ONND [10]. Instead of calculating the nearest distance of a synthetic sequence to a real sequence, ONND calculates the opposite by computing the average distance of a real sequence to the nearest synthetic sequence. When the aim of the synthesis is to have original-synthetic pairs, e.g. extending an existing sequence, a range of other metrics can be used, including directional symmetry [14]. It can also include further data mining techniques to shed light on other aspects than the plain subsequences that occur in both data sets. Frequent episode mining can extract patterns that occur within a specific time frame [114]. Patterns like "leg injury" → "other leg injury within 6 months" can be found with this. This not only measures the sequences occurring in this order, but also if the temporal interval is similar. The measurement of trend is also possible as proposed by Leznik et al. [89].

4.3.6. Downstream utility

Utility is a quality dimension that measures the suitability of synthetic data in training a model for a downstream task compared to the original data [139]. Downstream applications can include prediction and classification [139].

U1 Train-on-Synthetic-Test-On-Real (1-3): Train-on-Synthetic-Test-on-Real (TSTR) is a popular metric when evaluating the utility of a synthesized data set to train a model for a downstream task [43], [140]. A model is trained on the real data and the synthetic data and afterwards the results are compared on a held-out test set of the real data. If the model trained on the synthesized data performs equally well, it “demonstrates the ability of the synthetic data to be used for real applications” [43].

In this metric, one regression and one classification is proposed using CatBoost for the Transfermarkt data set [121]. For the regression of predicting an entries `market_value`, the root mean squared error (RMSE) and the R^2 values are compared. The more similar the model trained on synthetic data is with respect to RMSE and R^2 is to the original RMSE and R^2 , the better. In general, the lower the RMSE and the higher the R^2 , the better is the performance of the synthesizer. The RMSE is defined as follows [51]:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (4.11)$$

y_i is the real value and $\hat{y}_i$ is the predicted value. So the square root of the average error squared is calculated. The R^2 value is calculated as [30]:

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} \quad (4.12)$$

where $SS_{\text{tot}} = \sum_{i=1}^n (y_i - \bar{y})^2$ is the sum of error if the model always predicts the average y value and $SS_{\text{res}} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$ is the sum of residuals from the model predictions. The R^2 is the proportion of variance in the output variable explained by the input variables [76].

For the classification, the trend of the `market_value` should be predicted given that an entry has received a market value update. It can go up and down or be confirmed and stay the same. To compare the model trained on synthetic data to the model trained on original data two metrics can be used: The accuracy of the classification and the agreement rate as proposed by Bindschaedler et al. [15]. The agreement rate measures to what extent the classifiers trained on the synthetic and real data sets have learned the same patterns in the data, therefore a high agreement rate is considered to be a good result [15]. If the models trained on synthetic data perform worse, this indicates that the generator did not learn essential relationships in the data, thus making it not at all or only to a limited extent usable in a downstream task.

U2 Predictive score (2): Predictive score is a metric adopted by Yoon et al. [164]. A model is trained on the synthesized data set to predict the next sequence step values. The evaluation is then performed on the original data set and Yoon et al. use the Mean Absolute Error (MAE) to quantify the performance of the trained model [164].

For the Transfermarkt data set the Mean Absolute Error is not sufficient and therefore needs to be extended to include the categorical attributes. This can be achieved by using the Hamming distance [54] for the categorical attributes.

The predictive score indicates whether the generative model captured the conditional distributions over time. A low score means a low deviation from the original data, while a high value indicates very different predictions.

U3 Algorithm Comparison (1-3): Algorithm comparison was initially proposed by Lin et al. [92] and can be adopted within this framework. For this, a number of models of different algorithms are trained for a downstream task. The performance of these models is then compared between the models trained on the real data set and synthetic data set on a held-out test set. The ranking of algorithms can then be compared using the Spearman rank correlation coefficient [133] where the higher it is, the better [139]. This measures the relative effectiveness of the algorithms and can indicate whether the synthetic data sets are useful for choosing adequate models for a downstream tasks and generalize well to real data sets.

U4 Generalized Sequential Pattern Mining (2, 3): Generalized Sequential Pattern Mining (GSP) is a pattern mining technique to elicit often occurring sequences in temporal data [134]. It defines a sequence S as an ordered list of itemsets $S = \langle (i_{1,1}, i_{1,2}, \dots), (i_{2,1}, \dots), \dots \rangle$. A minimum support threshold that is a fraction is defined, and GSP then finds all subsequences in the sequence set that occur more often than the defined threshold. Then it will output all subsequences that occur more often than the threshold, which can be sets of arbitrary size.

GSP is applied to categorical features to find sequences that often occur in the original and synthesized data set. Single features are considered in isolation, so sequences that observe multiple features are not considered in this metric. Therefore, the itemsets are always single items, such as just the club in isolation. Only subsequences with more than one element are considered, as one element subsequences are of no concern to us.

The patterns found in the ground-truth and synthesized data sets P_{orig} and P_{syn} are then compared. We compare these two sets using

$$\text{Precision} = \frac{|P_{orig} \cap P_{syn}|}{|P_{syn}|}, \quad \text{Recall} = \frac{|P_{orig} \cap P_{syn}|}{|P_{orig}|}, \quad F_1 = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

A high F_1 score indicates that the synthesized data preserves the sequences occurring in the real data. Low precision means that there exist incorrect patterns and low recall means that the synthetic data does not include many real patterns.

4.3.7. Visualizations

While direct visual evaluation of synthesized tabular data is inherently limited [139], visualizations can still reveal important structural and temporal characteristics. By plotting aggregated statistics, distributions, or time-dependent trends, domain-specific expectations and anomalies become easier to spot. They include validation of, for instance, the presence of seasonality or periodic patterns across the observed time window. This can help identify whether the generative model captures key real-world behaviors encoded in the data. The following visualizations show a comparison between the original data and an example synthetic data set.

V1 Column Distribution Plot (1): To qualitatively evaluate marginal distributions, real and synthetic distributions should be plotted. This allows for fast inspection of what parts of a distribution the generative model failed to synthesize and gives an indication of how well the distributions are approximated. Figure 4.6 shows an example distribution comparison between the market value of players. This visualization, for instance, directly shows that the synthetic data does not approximate the original data very well.

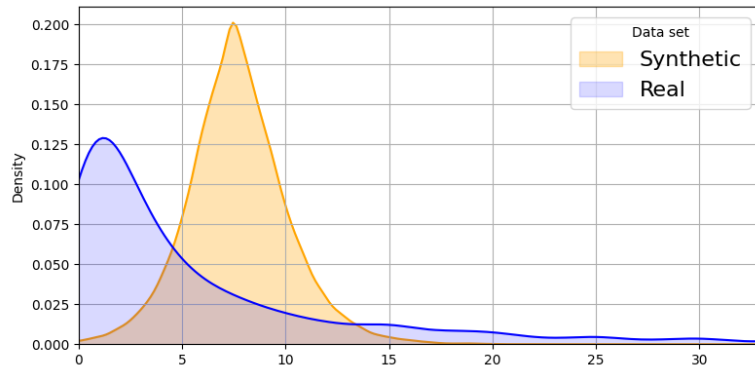


Figure 4.6.: Market value marginal distribution showing the synthetic data (orange) fails to approximate the real data (a).

V2 Column correlation (1): The column correlation can also be plotted as shown in Figure 4.7. The used correlation matrices are from the previously introduced statistical metric column correlation (S4). The synthetic data is expected to contain correlations similar to those of the ground-truth data. By visualizing, we can directly see the differences in the correlation structure and where we would expect correlations to occur. Features that represent temporal values, are ids or are the first name, last name, or pseudonym of the player are omitted, as they do not yield useful correlations. Therefore, 23 features are left which are compared in this plot.

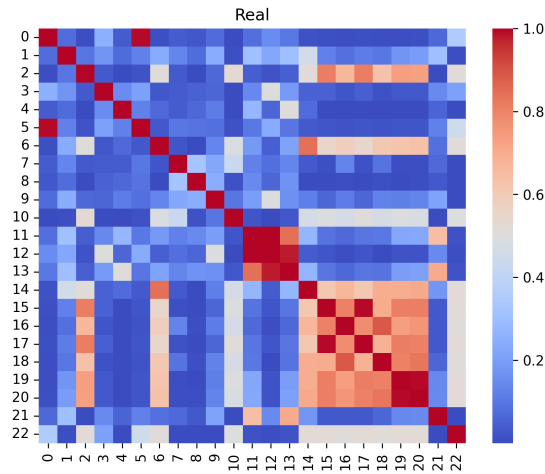


Figure 4.7.: The ground-truth column correlations, which the synthetic data has to approximate.

V3 Sequence length distribution (1): The sequence length distribution is plotted in Figure 4.8 with synthetic sample data for comparison. As in this example, there are no longer sequences in the synthetic data, which reveals that the generator did not learn the relationship between rows for entities. This can also be done for more fine-grained insights such as only looking at one bin such as sequence lengths 1 to 9.

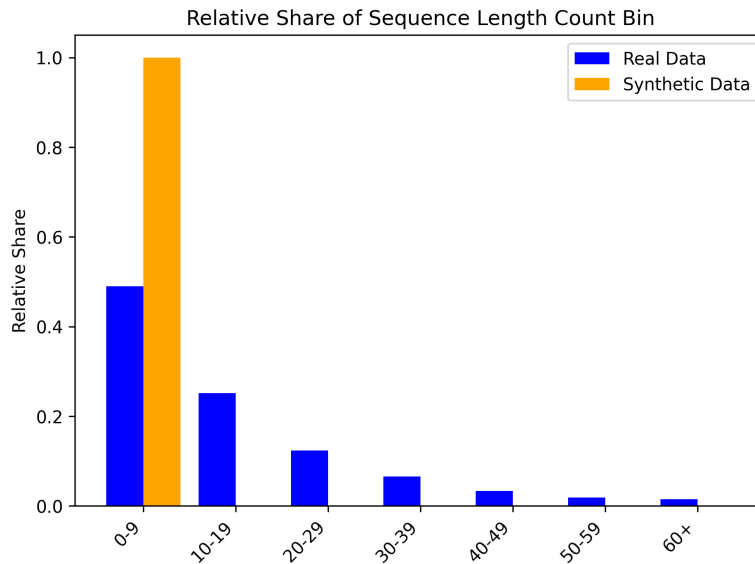


Figure 4.8.: The sequence length distribution, where the synthetic data (orange) fails to reflect the real data (blue) distribution.

4. TempoBench: A benchmarking framework for synthetic temporal tabular data

V4 Confusion matrix for classification downstream task (1-3): For the classification task mentioned in metric U1 a confusion matrix can be created to allow quick evaluation of the classes for which the synthesizer struggled to create realistic samples. In the exemplary confusion matrix shown in Figure 4.9 it can be seen that while the classification for up is very accurate, the model struggled to classify down or same.

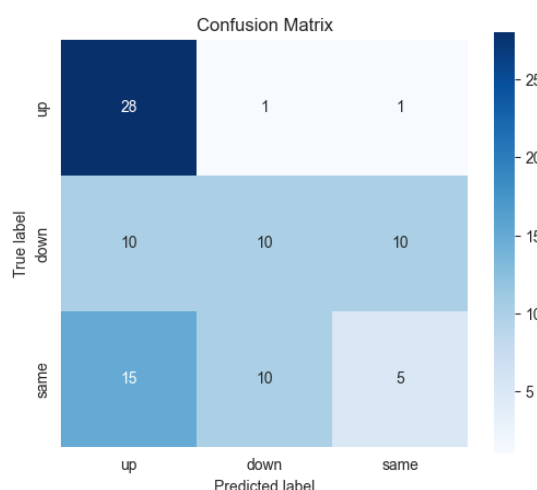


Figure 4.9.: Confusion matrix showing the up category can be categorized well, while down and same are not as separable for the classifier.

There also exist visualizations that are dependent on the data set used and therefore domain-specific. Therefore, the following visualizations only apply to the Transfermarkt data set. However, they can be easily translated to be applied to other data sets as well.

V5 Events (samples) per month (1): The events occurring per month during the total observed time window are plotted in Figure 4.10 with synthetic sample data. We expect the synthetic data to match the most events occurring around June to August and December to February. The synthetic data in this example does not match the real data for most months, which is most obvious for the peak in month 07 in the real data and the higher values in the synthetic data for the months 03 to 05.

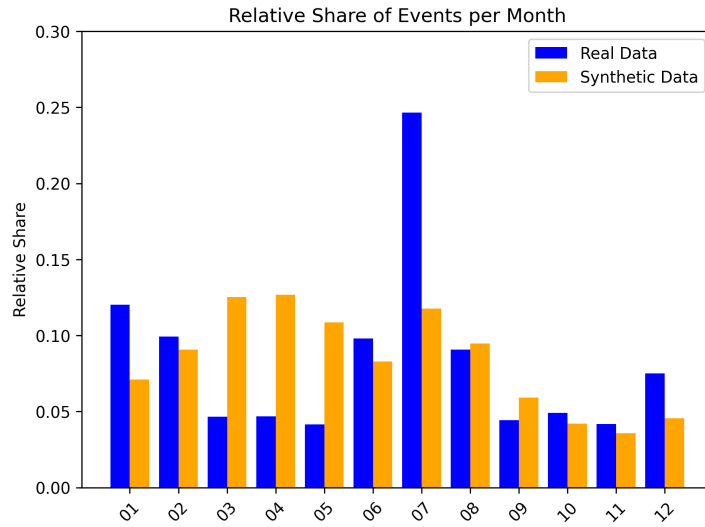


Figure 4.10.: The events in the real data (blue) have a peak in months 07 and 01, while the synthetic data (orange) fails to approximate this.

V6 Injuries per month (1): The injuries that occur per month during the total observed time window are plotted in Figure 4.11 with synthetic sample data. The synthetic data is expected to match the distribution of injuries occurring, which is mostly uniform except for May to July where the fewest injuries occur as the fewest matches are played.

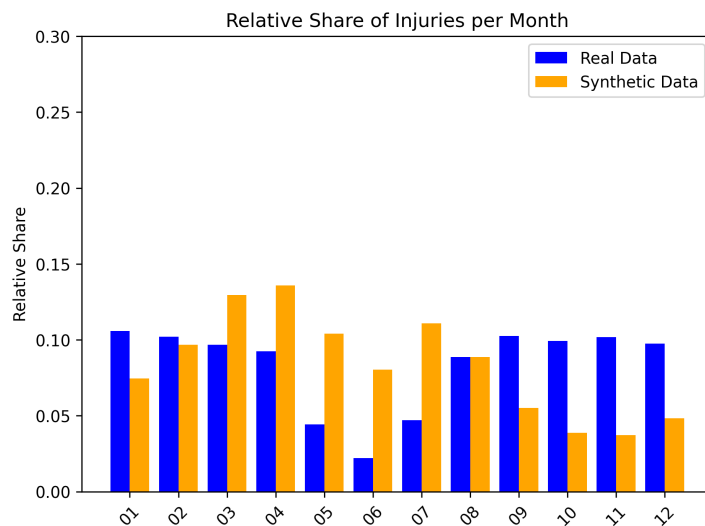


Figure 4.11.: The injuries per month in the real data (blue) are almost uniformly distributed except for months 05 to 07. The synthetic data (orange) fails to reproduce this.

4. TempoBench: A benchmarking framework for synthetic temporal tabular data

V7 Market value per season (1-3): The market values for the Bundesliga and Premier League are sampled on 01.07 of each season and plotted in Figure 4.12 with synthetic data sample. The synthetic data is expected to match the trend of the market value over time and also differentiate between the leagues where the Premier League is expected to have higher market values.

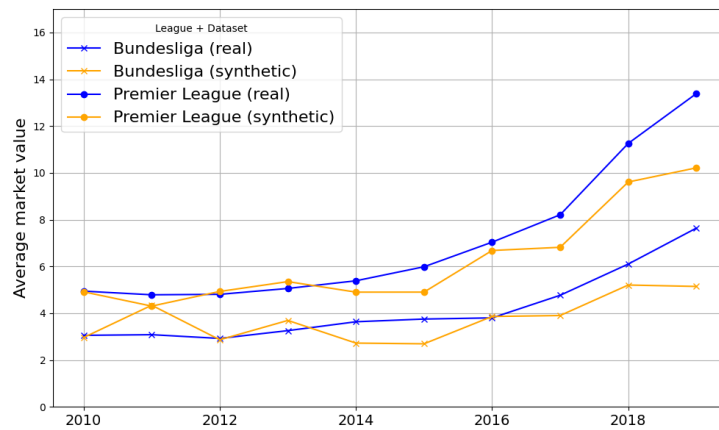


Figure 4.12.: The market values per season in the real data (blue) increase over time where the Premier League consistently contains higher values. The synthetic data (orange) reflects this general upward trend.

V8 Transfer fee per season (1-3): The average transfer fee per season plotted in Figure 4.13. The synthetic data is expected to match the transfer fees and the trend over the seasons. As can be seen, the synthetic data was not able to distinguish between the leagues and shows similar absolute values for both.

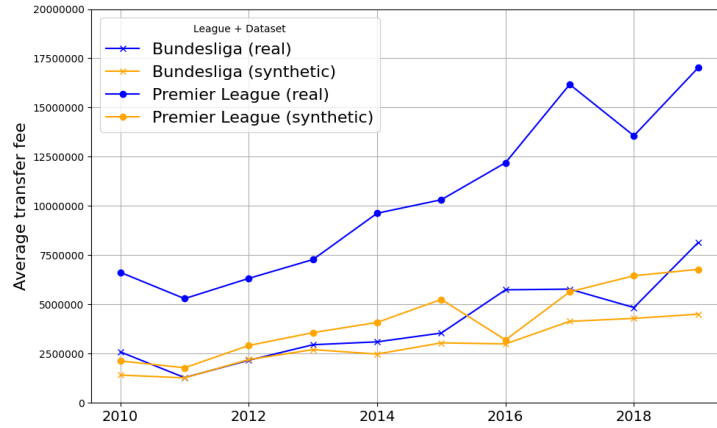


Figure 4.13.: The transfer fee per season increases in the real data (blue) over time, which is reflected in the synthetic data (orange). But the synthetic data underestimates the absolute values.

V9 Transfers per month (1): The transfers per month during the time window is plotted in Figure 4.14. The synthetic data is expected to match the distribution of transfers over the month, respecting an inherent domain rule that the transfer windows are open in winter and summer. Other transfers for free agents or to other leagues can still happen outside these windows.

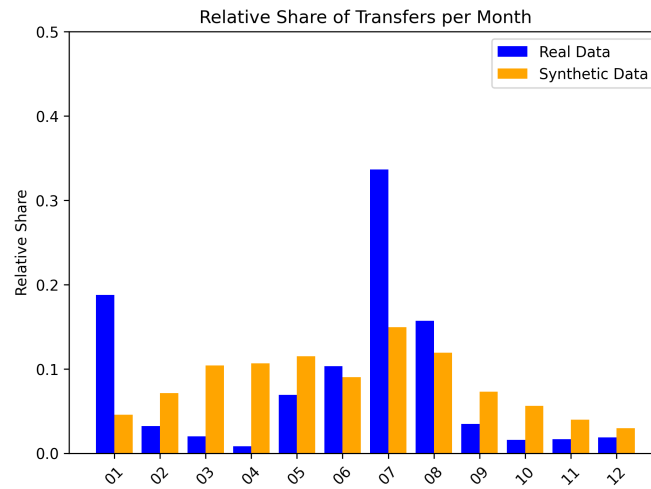


Figure 4.14.: Transfers per month showing the real data (blue) contains two peaks in months 01 and 07. The synthetic data (orange) fails to approximate the distribution.

Further Other visualizations that can be used include the Principal Component Analysis and t-SNE [90], [102], [135]. Furthermore, autocorrelation curves can be used as shown by Zheng et al. [173].

5. Prototypical implementation

This chapter presents the prototype implementation of the TempoBench framework. It begins by outlining the requirements for the prototype, followed by a description of the overall architecture. Subsequently, the implemented evaluation metrics are discussed and the chapter concludes with implementation details.

5.1. Requirements for the prototype

The benchmarking framework prototype must satisfy a variety of requirements, which can be categorized into *functional* and *non-functional* requirements. The overall goal is to provide a powerful and easy-to-use TempoBench prototype, including all categories and various visualizations. The functional requirements can be seen in Table 5.1, while the non-functional requirements are listed in Table 5.2. Basic functional requirements are computing metrics from all categories or storing the results of the TempoBench evaluation.

Table 5.1.: Functional requirements

Requirement ID	Description
F Req. 1	Compute sanity metrics.
F Req. 2	Compute domain-specific metrics.
F Req. 3	Compute statistical fidelity metrics.
F Req. 4	Compute temporal fidelity metrics.
F Req. 5	Compute downstream utility metrics.
F Req. 6	Create visualizations comparing real and synthetic data.
F Req. 7	Support extensibility through user-defined metrics and visualizations.
F Req. 8	Export benchmarking metric results in JSON.
F Req. 9	Export benchmarking visualizations results as PNGs.
F Req. 10	Allow configuration of to be computed metrics and visualizations.

5. Prototypical implementation

For the non-functional requirements, the benchmarking framework should be executable cross platform and follow software engineering best practices, while also documenting the code.

Table 5.2.: Non-functional requirements

Requirement ID	Description
NF Req. 1	The framework must have a modular design to enable replacement or extension of existing components.
NF Req. 2	Ease of use with API that has a single parameterized function.
NF Req. 3	Documentation in form of README.
NF Req. 4	Must be executable cross-platform (Windows, Linux, MacOS).

5.2. Architecture

The TempoBench prototype is designed in a modular way, allowing easy extension of new metrics or data sets. It follows best-practices in software engineering through well-defined interfaces and separation of concerns. The overall architecture is illustrated in Figure 5.1.

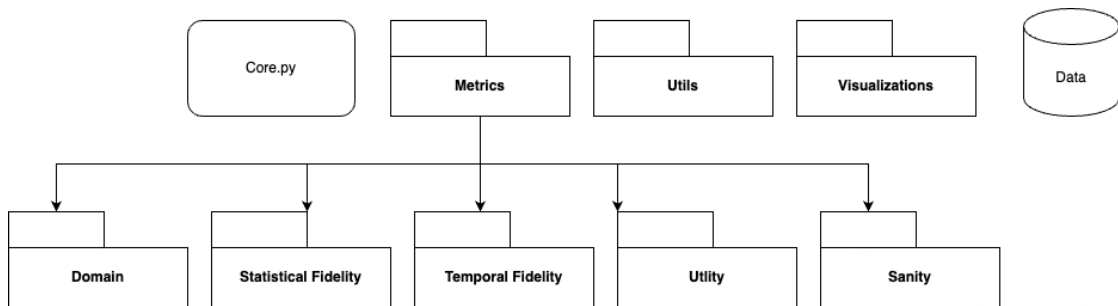


Figure 5.1.: TempoBench prototype architecture

The prototype is composed of the following high-level modules:

Core.py `core.py` serves as the primary API for clients using the TempoBench prototype. It handles input validation and orchestrates the computation of both metrics and visualizations. As the central component of the benchmark workflow, it manages the full process from user input to output generation. Additionally, `core.py` is responsible for storing the results. The quantifiable metrics are stored in JSON format, while the visualizations are stored in PNG format for inspection.

Metrics The `metrics` module is organized into five subpackages, each representing a core category of the benchmark, such as statistical fidelity or temporal fidelity. It defines common interfaces for metrics, which also enables clients to implement and integrate custom metrics easily. These custom metrics are computed alongside the built-in ones, which also implement this interface.

Utils The `utils` package provides general-purpose utility functions for the TempoBench prototype. This includes loading real data sets into Pandas DataFrames and retrieving meta-data, such as whether columns are numerical or categorical. It also offers shared used distance and normalization functions used across different metrics. Additionally, `utils` provides functionality for data set splitting, preprocessing steps before training a generative model, and postprocessing of the resulting synthetic data set. In the prototype, this is implemented for the integrated Transfermarkt data set.

Visualizations The `visualizations` package contains the visualizations used to qualitatively compare the real data with the synthetic data. It includes visual outputs such as temporal development of transfer fees or the distribution of individual columns, enabling insights into the similarities and differences between data sets.

Data The data directory contains the Transfermarkt data set used in the prototype. It also contains a predefined train / test split and their respective preprocessed versions.

As illustrated in Figure 5.2, a client first generates synthetic data by training on the original training data set using an external generator. Second, the client may optionally define custom metrics and visualizations that are not included in TempoBench. The third step is the invocation of the prototype, which triggers the core module to orchestrate the execution of all selected metrics and visualizations. Upon completion, it stores the evaluation results in a JSON file and generates visualizations as PNG to the specified output locations.

5. Prototypical implementation

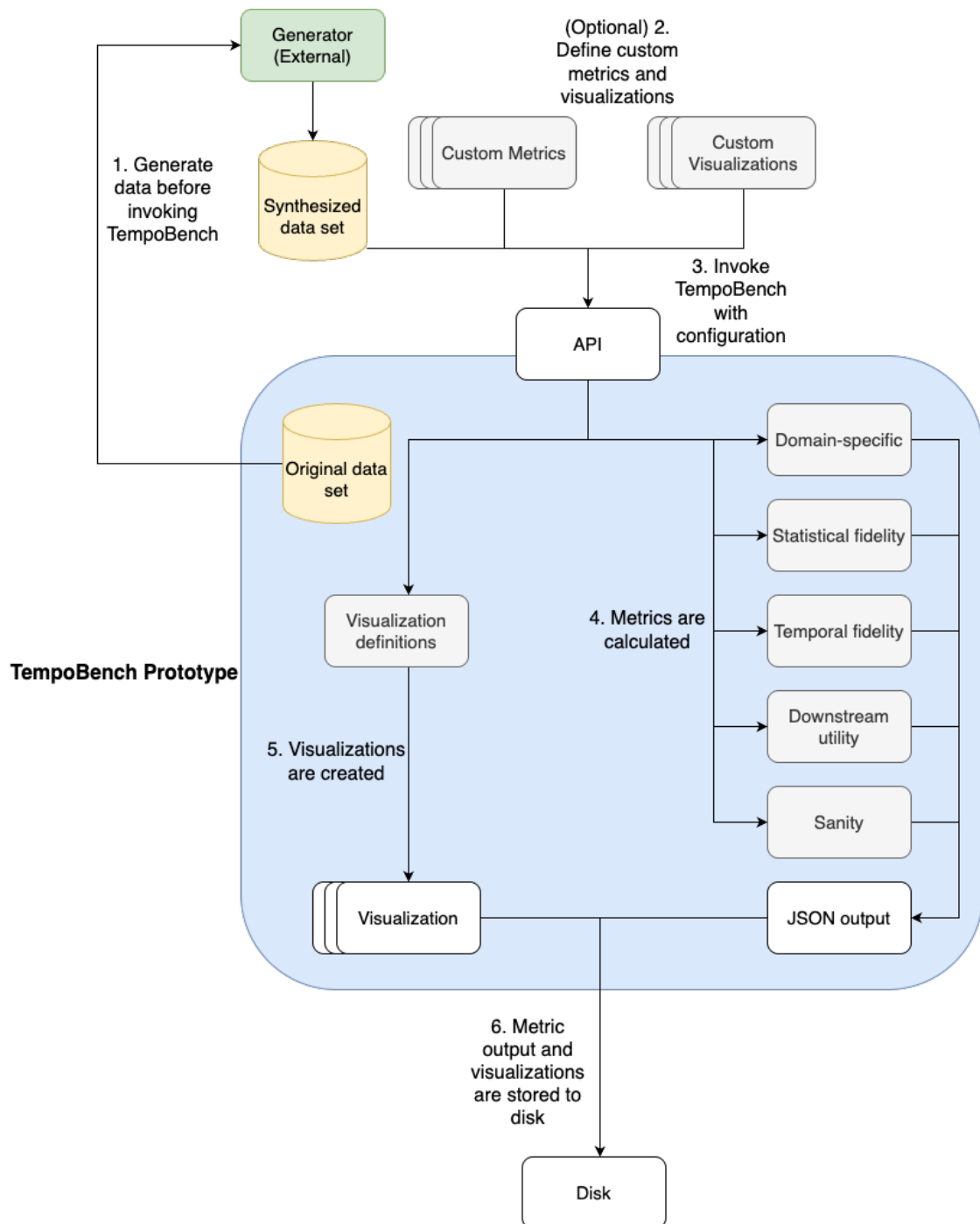


Figure 5.2.: A generator trained on the original data synthesizes a data set. Custom metrics and visualizations can be defined with which TempoBench is invoked. TempoBench then orchestrates the calculation of metrics and creation of visualizations, which are stored to disk.

5.3. Implemented evaluation metrics

The client invokes TempoBench via the API exposed by the `core.py` module. The API accepts several parameters for configuring the benchmark run. The list of parameters that can be passed are presented in Table 5.3.

Table 5.3.: TempoBench prototype input parameters

Name	Required	Description
X_syn	Yes	The synthesized data set to be evaluated.
Generator name	Yes	The name of the generator with which the data set was synthesized.
Metrics	No	A list of predefined metric names and/or custom metric implementations to be computed.
Metric categories	No	A list of metric categories to restrict computation to specific types of metrics (as introduced in the architecture).
Visualizations	No	A list of predefined and/or custom visualizations to be generated.
Visualization categories	No	A list of visualization categories to limit the types of visualizations generated.
Visualization path	No	File system path where the generated visualizations should be stored.
Output path	No	File system path where the benchmark results should be stored.

5.3. Implemented evaluation metrics

Due to time constraints, not all proposed metrics have been implemented in the TempoBench prototype. This means that the evaluation metrics to be implemented have to be prioritized. In general, the primary goal was to achieve coverage in all categories. This means that there are metrics implemented for each category, together with visualizations. The following will go over each category and explain why these metrics were chosen. After this, Table 5.4 gives an overview of the metrics implemented.

Sanity For the sanity category, all four proposed metrics were implemented. They were simple to implement and give a good first impression of the synthetic data, which makes them perfect for a prototypical implementation.

Domain When choosing the domain metrics implemented, the goal was to also include metrics that verify the unique temporal constraints of the Transfermarkt data set. As evaluating

5. Prototypical implementation

those temporal features are a central piece in the framework, for example, the monotonic increase of features has been implemented as this is a very basic constraint that allows for evaluation if the synthesized data mimics the ground truth evolution of performance data. Another basic requirement is that the static data remains consistent during the lifetime of an entity, which is also evaluated.

In addition to that, various other metrics have been implemented that specifically evaluate the temporal constraints inherent in the Transfermarkt data set. These include, for instance, the metric that checks the maximum time a tuple can be valid. Another metric ensures that the synthesized data does not create samples outside the time interval of the real data.

This allows the prototype to identify generative models that do not accurately synthesize validity timestamps.

Statistical fidelity The primary objective of the prototype was to provide an overview of the statistical characteristics of the synthesized data.

To this end, the elementary statistics metric was selected because it offers an intuitive and easily interpretable first impression of the synthetic data compared to the ground-truth data. It also enables a detailed analysis of where the generative model approximates the original data and where it fails to do so. This metric serves as a good tool for understanding the degree of deviation from the ground-truth data and gives an all-round view.

The Jensen-Shannon divergence was implemented to get a sense of the marginal distributions and how well those are approximated by the generative models. Complementary to this, visualizations of the marginal distributions were also implemented, as they can be easily compared to the ground truth, which allows for qualitative comparison of synthesis quality.

The correlation metric was implemented to evaluate how well the interactions between the features are preserved in the synthetic data. The corresponding correlation matrices were also visualized to allow quick visual inspection and to draw a simple comparison between the models.

Together, these metrics allow for a solid foundation for evaluating the statistical fidelity of the synthesized data set. While the other proposed metrics capture more nuanced aspects of data quality, the selected metrics offer a robust and interpretable starting point for statistical evaluation.

Temporal fidelity For temporal fidelity, the goal was to incorporate metrics that evaluate how the sequences in the synthetic data differ from or resemble those found in the ground-truth data. An emphasis was placed on incorporating metrics that evaluate the dynamics of categorical features over time.

The intra-class distance was selected as a metric to evaluate the variability within the generated sequences and to see whether this variability aligns with the real data set. With this metric, we can easily see if there are different kinds of sequences or if mode collapse has occurred in the generative model. Furthermore, it highlights whether the diversity of synthetic sequences matches the ground-truth sequences. 200 sequences were chosen as subset size.

The incoming nearest neighbor distance was chosen to assess whether the synthesized samples are resembling the original data set's sequences. This is considered an important metric

5.3. Implemented evaluation metrics

to evaluate whether the trajectories of the real and synthetic sequences can be aligned structurally and semantically. For the subset size in the synthetic data set, 50 sequences were chosen.

The transition matrix distance allows a perspective by quantifying how well the generative model captures the transition between categorical values. It enables insights into evaluating the fidelity with which likelihood category values change from one to the other. The update dependency metric captures inter-feature dependencies in temporal changes using the a priori algorithm and treating all columns that changed from one row to the next as the set of items in a transaction. It can assess whether a change in one feature consistently leads to a change in another feature and how this is retained in the synthetic data set. This is an essential property that the generative model can learn as it also incorporates constraints within the data set where one feature might not change without another feature. In the Transfermarkt data set, a player cannot score a goal without playing a match, so a change in goals scored must also have a change in played matches in the same row. Capturing these dependencies is critical to generate realistic data.

Additionally, simple summary metrics such as the average validity period of the entries and the mean length are added as basic overview metrics. For the mean length of sequences, the distribution is also visualized to easily see if the generative models can match the ground-truth distribution in this regard. This helps to determine whether generative models produce sequences of realistic duration.

Taken together, the temporal metrics allow for a comprehensive overview of temporal aspects of the synthetic data with special emphasis put on the temporal dynamics of categorical features.

Downstream utility The downstream utility metrics give an indication of how useful the data is for a subsequent task such as training machine learning models.

In the current prototype, two variants of the Train-on-Synthetic-Test-on-Real metric were implemented. The first is a regression task that aims to predict the market value of a player's entry. A regression model is trained on the real train data and synthetic data, respectively, and evaluated on the held-out real test set.

The second task is a classification problem, in which the objective is to predict whether the market value of a player increases, decreases, or remains the same. Temporal features such as the delta in performance data (e.g. games played or goals scored) or days since the last update in market value have been used, but they do not yet yield reliable results. Consequently, the classification task has been excluded from the final evaluation.

The pattern mining performed with the GSP algorithms finds subsequences in categorical features that occur in the original data and in the synthetic data and compares to what degree they contain the same often occurring subsequences. This has been implemented for key features such as league, club, and injury_category. This metric evaluates whether the synthetic data preserves the temporal patterns found in the real data for these categorical features.

In summary, the implemented TSTR regression task and GSP algorithm provides a measure of how useful the synthetic data is for a downstream task.

5. Prototypical implementation

Visualizations For the visualizations not already mentioned within the categories, all have been implemented except for the confusion matrix, as the classification is not evaluated in the results.

The implemented metrics and visualizations, including those discussed above, are summarized in Table 5.4 for clarity and completeness.

Table 5.4.: Overview of implemented evaluation metrics and visualizations

Category	Metric
Sanity	(SAN1) Duplicate rows (SAN2) Datatype mismatch (SAN3) Tuple validity overlap (SAN4) Validity end before start
Domain	(D1) Manual inspection (D2) Monotonic increase (D3) Static data remains unchanged (D4) Timestamp min. and max. (D5) Maximum validity time
Statistical	(S1) Elementary statistics (S2) Univariate distribution divergence (with visualization V1) (S4) Column correlation (with visualization V2)
Temporal	(T1) Incoming nearest neighbor distance (T2) Mean sequence length (with visualization V3) (T3) Mean validity (T4) Intra-class distance (T5) Update dependency (T6) Transition matrix distance
Downstream utility	(U1) TSTR-Regression (U4) GSP
Visualization	(V5) Events (samples) per Month

Continued on next page

Table 5.4 – continued from previous page

Category	Metric
Visualization	(V6) Injuries per month
	(V7) Market Value per season
	(V8) Transfer fee per season
	(V9) Transfers per season

5.4. Implementation details

For creating the TempoBench prototype, Python was used as the main programming language. It entails a huge ecosystem in machine learning, is widely used in the community, and is regarded the best programming language for machine learning related tasks [105]. As this is a prototypical implementation, performance and computational efficiency were not top priorities.

The following libraries were the main third-party dependencies used to implement the framework prototype:

- Pandas: Data container, manipulation and analysis [154]. It is used across all metrics and visualizations as data container.
- Numpy: Numerical computing with support for large, multidimensional arrays and matrices [58]. It is for example used for computing statistics (S4).
- Sklearn: Machine learning library with tools for classification, regression, clustering, and preprocessing [118]. Is used to calculate the RMSE or R^2 score in TSTR (U1).
- Catboost: Gradient boosting library optimized for categorical features and fast performance [121]. It is used for the downstream utility regression and classification task (U1).
- Scipy: Scientific computing with modules for optimization, integration, and statistics [150]. It is for example used when calculating the Jensen-Shannon divergence (S2).
- Mlxtend: Provides an implementation of the Apriori algorithm for frequent itemset mining [123]. It is used when calculating the update dependency metric (T5).
- GSPPY: Implements the Generalized Sequential Pattern algorithm for sequence mining [120].
- fastdtw⁶: Efficient approximation of Dynamic Time Warping algorithm. It is used in comparing sequences such as in T1 or T4.

⁶<https://github.com/slaypni/fastdtw>

5. Prototypical implementation

- `matplotlib`: Library for creating visualizations in Python [64]. It is used across all visualizations.

The interfaces in the scope of TempoBench that a client can later implement with their own custom metrics include the `metric` and the `visualization` interface. A metric needs to implement the interface with the methods `compute` which takes as a parameter the original and synthesized data and returns a dictionary with the result of the metric, which can therefore be dynamically chosen for each metric. It also has to implement the methods `name` and `category`, which only return the name and category of the metric for grouping in the resulting JSON. The full definition can be found in Appendix A.3.

Visualizations on the other hand follow a slightly different interface. The central method `create` takes the original and synthetic Pandas DataFrame as input, along with a path that specifies where the resulting image should be stored. Furthermore, each visualization implements the methods `name` and `category`, which are used to organize and store the output at the specified location. The full definition can be found in Appendix A.3

A guide on how to configure and run TempoBench is also included in the README file in the GitHub repository⁷. The prototype fulfills all above mentioned requirements.

⁷<https://github.com/bepo1337/tempobench/blob/main/README.md>

6. Evaluation results and discussion

To show the usefulness and ability of TempoBench, the prototypical evaluation framework is applied to assess the performance of various generative models. This chapter outlines the experimental setup, presents the models used, and analyzes the performance of the models. The results are discussed in the context of the implemented evaluation metrics and visualizations, providing insights into the strengths and weaknesses of each model.

6.1. Experimental setup

This section describes the experimental setup used to evaluate the generative models. It provides an overview of the models and describes their training process. The Transfermarkt data set, introduced earlier, serves as the ground truth for both training the models and evaluating the quality of the synthetic data.

6.1.1. Transfermarkt data set train/test split

Similar to Xu et al. [159], the Transfermarkt data was split into train and test data. The split was not performed using random indices, but was partitioned by the unique `player_id` to ensure that sequences remain intact and are not mixed between partitions. This is a result of the multi-entity and temporal data.

Specifically, the training set contains 80% of the set of `player_ids`, while the test set covers the remaining 20%. The generative models were then trained exclusively on the training set, as the training of the models on the test set was deemed to have too few samples to accurately capture the whole diversity of sequences. The exact number of entities and rows can be seen in Table 6.1.

Table 6.1.: Dimensionality of train/test data split

Split	#entities	#rows
Training	3401	49693
Test	851	12432

6.1.2. Generative models evaluated

For evaluating models based on the proposed TempoBench prototype and the Transfermarkt data set, three different generative models were selected. Two of them do not specifically

6. Evaluation results and discussion

target temporal data, while the last one does. They were chosen due to usage in other works [26],[162],[163] and because they cover different approaches to synthesizing data. The three models are the following:

- **Tabular Variational Autoencoder (TVAE)** proposed by Xu et al. [159] which is not explicitly handling temporal data.
- **TabSyn** proposed by Zhang et al. [165] is an integration of a diffusion model within a variational autoencoder architecture, which has shown promising results across multiple datasets and also does not handle temporal data explicitly.
- **Probabilistic Autoregressive Model (PAR)** by the Synthetic Data Vault is used which can explicitly handle mixed-type multivariate temporal data [166], [129].

6.1.3. Pre- / and postprocessing

Before passing the data to the generative models, the following steps were performed to preprocess the data:

- Split all dates from the YYYY-MM-DD format into three feature columns with `feature_name_year`, `feature_name_month`, `feature_name_day` to allow models to treat the dates as a combination of categorical features.
- The IDs were converted into string values for `player_id`, `coach_id`, `club_id` to allow models to treat them as categories.
- Split the `reason` column into six binary features `reason_new_coach`, `reason_injury`, `reason_injury_end`, `reason_transfer`, `reason_regular_interval`, `reason_market_value_update` to allow models to better differentiate between the reasons and their impact on the row.
- Null values in categorical columns such as `injury` or `club` are replaced with "N/A" in order to have no null values for training models that cannot deal with missing values.

After the generation has been completed, the postprocessing reversed these steps and parsed the features back into the format of the Transfermarkt data set. The pre- and postprocessing scripts can be found in the GitHub repository⁸ and can be used to postprocess data sets from other generative models as well. They also deal with parsing data between formats, e.g. float variables to integers where the real data set contains integer values.

6.1.4. Training the generative models

Due to time constraints, all models were trained only once. The following describes the training setup for each generative model. After training has been completed, a synthetic data set is sampled with the size of the Transfermarkt data set using the respective fitted model.

⁸<https://github.com/bepo1337/tempobench>

TVAE For generating a synthetic data set with TVAE, Synthcity [148] was used in version v0.2.11. The source code was slightly altered to allow the internal split of the overall training data into train and validation data to also be done by `player_id` instead of random indices⁹. In addition to the default parameters, the following parameter was adjusted:

- `n_min_iter`: 1000, which sets the minimum training iterations.

The training was carried out on Google Colab within a CPU environment.

TabSyn For training and generating a synthetic data set with TabSyn, the implementation by Amazon-Science is used [4]. The source code was also slightly altered to allow the use of JSON files as input¹⁰. As TabSyn requires a target variable defined for the built-in evaluation of ML efficacy, a dummy variable `target` is created for all rows and initialized to "0" and after the generation of samples is discarded. The inbuilt evaluation of TabSyn is not used and therefore this does not have an effect.

During training, TabSyn applies one-hot-encoding to categorical variables and constructs a lookup table for the embedding [165]. As the Transfermarkt data set contains categorical variables that have very high cardinality such as `injury`, not all possible injury categories are present in the training data set from the training/validation split. So there are injuries that are present in the validation dataset, but not in the training set. This leads to an error during training. So, the following further preprocessing has been done:

- For each `player_id` in the validation set, a new combination of `player_id`, `first_name`, `last_name`, `citizenship` and `pseudonym` have been chosen. The values were randomly sampled from the training set if they did not exist in the validation set. So if the `first_name` exists in the training set, but the `last_name` does not, only the `last_name` will be changed for this player. The `first_name` will remain unchanged.
- If the `injury` of a row is not in the training set, it will be replaced by an `injury` from the same `injury_category` from the training set.

The default parameters of the latest version of the official Amazon Science GitHub repository were used to train the TabSyn model and generate samples [4]. The training was carried out on Google Colab with a T4 GPU.

SDV Probabilistic Autoregressive Model For training the PAR model, the implementation of SDV was used in version 0.7.0 [129]. As PAR supports explicit temporal data only for the sequence index, the same preprocessing as for the other two models was applied with the `validity_start` additionally serving as sequence index. The default parameters were used in training except for the number of epochs, which were suggested to be 1024 in the repository. Due to computational cost and time efficiency, this was cut down to 750 epochs¹¹. The training was carried out on Google Colab and an A100 GPU was needed

⁹<https://github.com/bepo1337/synthcity>

¹⁰<https://github.com/bepo1337/tabsyn>

¹¹<https://github.com/bepo1337/DeepEcho>

6. Evaluation results and discussion

due to additional required RAM in comparison to the other two methods. The need for a high-RAM A100 GPU made training much more resource intensive.

6.1.5. Synthesized datasets

After training the models, the synthetic data sets were sampled from these fitted models. For TabSyn and TVAE the exact number of samples were able to be generated, which match the training data set size. For PAR on the other hand, only the number of sequences to be generated could be passed to the sampling function. Therefore, more sequences were generated and then random sequences were picked to match the size of the training data set by sampling brute-force subsets of the sampled data. Table 6.2 gives a brief overview of the synthesized data sets and their dimensionality.

Table 6.2.: Dimensionality of synthetic data

Model	#entities	#rows
Transfermarkt (training data)	3401	49693
TVAE	3173	49693
TabSyn	2639	49693
PAR	1632	49632

6.2. TempoBench results

In this section, the benchmark results from the TempoBench prototype is discussed for all the evaluated models. All categories and metrics will be compared one after another, and in the end a conclusion will be drawn for each model, summarizing where they have their strengths and weaknesses. The complete results for each model can be sought in the prototype repository <https://github.com/bepo1337/tempobench>.

For each generation model, the benchmark was run a single time with Python 3.10. The Transfermarkt data training set is used for the comparison of the metrics with the synthesized data sets. A complete run with all implemented metrics and visualizations took around 30 minutes to complete on a MacBook Air with an M3 chip.

6.2.1. General

By simply looking at the dimensionality of the synthesized data sets introduced in Table 6.2 we can already see vast differences. While they all contain the same amount of rows as the training data set with the slight exception of PAR, their amount of generated entities differs. While the real training set contains 3401 unique players, TVAE generated a dataset of 3173

players, TabSyn of 2639 and PAR generated the fewest entities with only 1632 which is less than half of the original training data set.

6.2.2. Sanity

The prototype implements both sanity metrics, in which none of the models appear to show any shortcomings, as is presented in the following.

SAN1 Duplicate rows For row-copying, all models showcased that they did not leak single rows and therefore did not copy data from the original data set.

Table 6.3.: Comparison of duplicate rows

Model	Duplicate Rows (%)
TVAE	0%
TabSyn	0%
PAR	0%

SAN2 Datatype mismatch The same results show in the datatype mismatch metric where all synthesized data sets matched the column type of the ground-truth data set.

Table 6.4.: Comparison of datatype mismatch

Model	Datatype Mismatch (%)
TVAE	0%
TabSyn	0%
PAR	0%

SAN3 Tuple validity overlap For the overlap of the validity timestamps, TabSyn performed the best with 38.2% of the rows violating this constraint. PAR stands at a violation rate of 53.3%, while TVAE has the most at 71.8% as can be seen in Table 6.5. This shows that none of the models learned the temporal relationship within each sequence in which the tuples occur after each other. This indicates that they randomly generate dates instead of setting them in relation to each other.

6. Evaluation results and discussion

Table 6.5.: Comparison of tuple validity overlap

Model	Rows Violated (%)
TVAE	71.8%
TabSyn	38.2%
PAR	53.3%

SAN4 Validity end before start The validity end before start metric shows that the models did not learn that the data set contains valid times for the tuples as can be seen in Table 6.6. TVAE violated this constraint the least with only 23.5% of rows. In contrast, TabSyn and PAR perform significantly worse, with violation rates of 42.5% and 45.5%, respectively. A rate of almost half of the samples shows that the generators rather randomly guessed the dates instead of setting them in a logical relation.

Table 6.6.: Comparison of validity end before start

Model	Rows Violated (%)
TVAE	23.5%
TabSyn	42.5%
PAR	45.5%

For the sanity metrics, all models perform equally well on row duplication and datatype mismatch, but show rather weak performance for the metrics involving the validity of tuples.

6.2.3. Domain

The domain-specific metrics already showed various differences between the models. The results of the prototype domain-specific metrics are presented in the following.

D1 Manual inspection Due to the time effort required to analyze the sequences, only one sequence per model has been manually inspected. The following only show excerpts of the sequence with selected aspects. The full sequences with all 33 features can be found in the respective data set in the GitHub repository of this thesis. The matches and goals in the tables presenting the sequence excerpts refer to the `matches_league` and `goals_league`. First of all, a real sequence that is from the player with ID 7858 will be inspected. Table 6.7 shows this excerpt from the sequence. This sequence is sorted temporally, with the dates omitted for space-related reasons. Here we can see that the position of the player stays the same over time and is therefore a static attribute in the Transfermarkt data set. It is also

worth noting that the club ID remains consistent for a club, thereby creating a functional dependency. We can also observe that the performance data in the form of matches and goals is increasing monotonically, while the played matches can increase without the goals increasing. This is not true vice versa, as can be seen in the last row. While the player is injured, none of the performance data can increase, which can also be seen in the table.

Table 6.7.: Real sequence excerpt

#	position	club_id	club	matches	goals	injury
1	Right Winger	31	FC Liverpool	155	26	None
2	Right Winger	631	FC Chelsea	155	26	None
3	Right Winger	631	FC Chelsea	155	26	Groin problems
4	Right Winger	631	FC Chelsea	155	26	None
5	Right Winger	631	FC Chelsea	155	26	None
6	Right Winger	631	FC Chelsea	158	27	Achilles tendon rupture
7	Right Winger	631	FC Chelsea	158	27	Achilles tendon rupture
8	Right Winger	631	FC Chelsea	158	27	None
9	Right Winger	631	FC Chelsea	162	27	None

Next, sequences from the generative models will be inspected for the same columns. Table 6.8 shows an excerpt for TVAE with player ID 14177. We can see the exact opposite from the real sequence excerpt. The sequence is all over the place with respect to all features. First, the position is no longer a static feature, changing for each entry. Then the club and club ID also change for every column, and we can note that club ID 931 is used twice in the row #2 and #9. So, the functional dependency is no longer valid. The performance data appears to be quite disorganized. The features no longer increase monotonically and seem to be generated at random. Furthermore, the player increases his played games and goals between row #1 and #2, while being injured. Therefore, this sequence generated by TVAE can be considered very different from a real sequence.

Table 6.8.: TVAE sequence excerpt

#	position	club_id	club	matches	goals	injury
1	Defensive Midfield	4	VfL Wolfsburg	80	0	Muscular problems
2	Centre-Forward	931	Manchester United	272	16	Muscular problems
3	Right-Back	533	West Bromwich	44	1	Groin problems

Continued on next page

6. Evaluation results and discussion

Table 6.8 – continued from previous page

#	position	club_id	club	matches	goals	injury
4	Central Midfield	985	FC Blackpool	1	0	None
5	Goalkeeper	-1	FC Nürnberg	1	0	Muscular problems
6	Attacking Midfield	16	SC Freiburg	1	0	Muscular problems
7	Goalkeeper	24	TSG Hoffenheim	0	0	None
8	Left midfield	1181	Gladbach	84	11	Back contusion
9	Centre-Forward	931	FC Ingolstadt	0	0	Arm fracture

A similar pattern can be observed for TabSyn. The excerpt of a TabSyn sequence can be seen in Table 6.9. Again, the position changes for almost every row, violating the static constraint of this feature.

Again, the functional dependency between the club ID and the club is not respected and the changes observed in each entry do not reflect the patterns seen in the real data. Similarly to TVAE, the number of matches played and goals scored fluctuate arbitrarily, not adhering to the monotonicity constraint. The performance metrics vary when the player is injured, which also deviates from the real data set. An additional inconsistency can be observed in the final two rows, where an invalid transition between two injuries occurs. In the Transfermarkt data set, an injury first has to end before a new one can begin.

Table 6.9.: TabSyn sequence excerpt

#	position	club_id	club	matches	goals	injury
1	Centre-Forward	1132	Tottenh. Hotspur	79	23	None
2	Left Winger	985	FC Nürnberg	94	66	Hairline fracture
3	Goalkeeper	29	FC Everton	0	0	None
4	Left-Back	933	VfL Wolfsburg	21	0	Infection
5	Att. Midfield	4795	TSG Hoffenheim	59	0	None
6	Left-Back	1132	Bayer Leverkusen	12	0	Torn muscle fiber
7	Left midfield	31	FC Liverpool	184	11	None
8	Def. Midfield	65	Fortuna Düsseldorf	80	3	Bruise
9	Def. Midfield	24	FC Southampton	91	5	Corona virus

PAR shows differences from the previous two models, as illustrated in Table 6.10 for the player with ID 745. In particular, the static feature `position` remains unchanged throughout the sequence of the player. For the other features, similar violations can be observed. The

functional dependency between club and club ID is violated, as can be seen for club ID 512 which has two different clubs. Again, the performance data does not show a monotonic increase. Furthermore, the goals scored increase when the number of matches does not. Additionally, performance data changes throughout a players injury. It should also be noted that the range of matches and goals remain relatively stable, without abrupt fluctuations observed in the other models.

Table 6.10.: PAR sequence excerpt

#	position	club_id	club	matches	goals	injury
1	Centre-Forward	762	FC Bayern München	27	0	Coccyx contusion
2	Centre-Forward	512	FC Everton	14	0	None
3	Centre-Forward	33	FC Fulham	54	3	None
4	Centre-Forward	4	AFC Sunderland	52	0	None
5	Centre-Forward	65	FC Schalke 04	19	0	Coccyx contusion
6	Centre-Forward	512	Fortuna Düsseldorf	15	2	None
7	Centre-Forward	631	Gladbach	21	0	None
8	Centre-Forward	512	Bolton Wanderers	73	0	Coccyx contusion
9	Centre-Forward	289	AFC Sunderland	35	1	Coccyx contusion

D2 Monotonic Increase The results presented in Table 6.11 indicate that none of the models correctly learned the monotonic increase characteristics of these columns. Although all models violate this constraint with more than 85% of entities, PAR demonstrates the lowest violation rate at 85.6%. The proportion of violated rows, on the other hand, is the highest for PAR which lies at 96.6% compared to 74.0% for TabSyn and 83.6% for TVAE. The proportion of rows that violate the constraint might be a more interesting metric in this case, as entities can contain multiple violations. Regardless, the models all failed to preserve this characteristic in the generated data.

Table 6.11.: Comparison of monotonic increase violations for entities and rows

Model	Entities Violated (%)	Rows Violated (%)
TVAE	93.0%	83.6%
TabSyn	93.4%	74.0%
PAR	85.6%	96.6%

The models also contain similar patterns when inspecting which columns were most frequently violated, as can be seen in Table 6.12. While age, league_goals and interna-

6. Evaluation results and discussion

tional_played_matches vary between 33% and 47% of violation for all models, international_goals is only violated in 14.7% of rows for TVAE and 17.2% for TabSyn. This can be attributed to the distribution of this feature having more rows with a value 0 and therefore not changing between rows. PAR shows more than double the rate of violation for international_goals which can be interpreted as PAR not having learned this marginal distribution properly.

Table 6.12.: Comparison of excerpt for monotonic increase violation details

Model	age	league_goals	international_played_matches	international_goals
TVAE	44.3%	33.2%	34.7%	14.7%
TabSyn	44.9%	35.7%	36.4%	17.2%
PAR	46.8%	43.2%	46.2%	35.6%

D3 Static data remains unchanged For the static data remains unchanged metric, we can see great differences between the models. As Table 6.13 shows, TVAE and TabSyn did not learn that there exist features that remain static throughout the lifetime of an entity. PAR, on the other hand, perfectly handled this type of data and did not violate this at all. This is due to PAR explicitly incorporating static features during training.

Table 6.13.: Comparison of static data remains unchanged

Model	Entities Violated (%)
TVAE	93.9%
TabSyn	95.8%
PAR	0%

D4 Timestamp minimum and maximum For the given fixed time interval in the Transfermarkt training data that only contains data from 01.07.2010 to 30.06.2020, the models also generated data outside of this interval. As shown in Table 6.14, only 4.8% of tuples generated by TVAE and 9.4% by TabSyn fall outside the time interval, whereas PAR exhibits the highest deviation with 12.4%. A general trend starts to present itself, where PAR deviates from the other two models in terms of divergence from the distribution and valid feature values.

Table 6.14.: Comparison of timestamp minimum and maximum

Model	Rows Violated (%)
TVAE	4.8%
TabSyn	9.4%
PAR	12.4%

D5 Maximum validity time All models performed well with respect to the maximum validity time constraint. As shown in Table 6.15, TabSyn and PAR each violated this constraint in only 0.2% of rows. TVAE achieved the best performance with less than 0.01% of rows not adhering to this constraint. This is likely due to the way the models handle the dates, which are made up of multiple categorical values during training. They have only very limited combinations of day, month, and year to create tuples with validity of more than 10 years.

Table 6.15.: Comparison of maximum validity time

Model	Rows Violated (%)
TVAE	<0.01%
TabSyn	0.2%
PAR	0.2%

In summary, none of the models demonstrated strong performance across the domain-specific metrics. No model consistently outperformed the others, each exhibiting weaknesses in capturing the domain-specific temporal constraints. One exception is the 'static data remains unchanged' metric, where PAR performed very well with 0% of violations, which is due to this type of static features explicitly being incorporated in the training of PAR.

Aside from this, the overall results indicate that the models struggled to preserve key constraints inherent in the Transfermarkt data set.

6.2.4. Statistical fidelity

The statistical fidelity metrics showed various differences between the models, which are presented below. The test set of the Transfermarkt data set is also listed as reference values for the generative models.

S1 Elementary Statistics We first examine the categorical features. The results in Table 6.16 show the comparison of elementary categorical statistics across the whole data set. It can be seen that TabSyn was the best model for approximating categorical attributes. It

6. Evaluation results and discussion

correctly identified the most frequent category (mode) in 58.3% of cases which matches the test set and had an average deviation of entropy of only 4.6%. PAR identified 41.7% of the modes and an average entropy deviation of 8.6%. TVAE was better than PAR in matching the mode with exactly 50.0% but was significantly worse at approximating the entropy which stands at an average of 23.8% deviation across all categorical columns.

Table 6.16.: Comparison of elementary categorical statistics

Model	Mode matches (%)	Avg. entropy deviation (%)
Test	58.3%	2.4%
TVAE	50.0%	23.8%
TabSyn	58.3%	4.6%
PAR	41.7%	8.6%

The results in Table 6.17 present a comparison of elementary numerical statistics across the entire data set. TVAE does not outperform the other models in any of these statistics. While TabSyn is best on average in approximating the median, standard deviation, and maximum, PAR was best at approximating the mean and minimum. Overall, TabSyn consistently outperforms TVAE in all elementary statistics, whereas PAR exceeds TVAE only in terms of mean and minimum deviations.

Table 6.17.: Comparison of elementary numerical statistics deviations with respect to the training data

Model	Mean (%)	Median (%)	STD (%)	Min (%)	Max (%)
Test	6.3%	2.4%	7.4%	0.2%	21.8%
TVAE	32.4%	42.1%	25.7%	0.7%	20.2%
TabSyn	25.0%	17.8%	23.8%	0.6%	4.0%
PAR	18.3%	100.5%	56.0%	0.1%	42.5%

The metric also contains elementary statistics per season. TVAE did not generate tuples with a season different from 2018 and therefore the models are not compared in the per-season detail.

S2 Jensen-Shannon divergence For the marginal distributions, the results in Table 6.18 show that TabSyn is able to approximate marginal distributions the best. It has an average Jensen-Shannon divergence of 0.20, while TVAE and PAR show a very similar performance

with 0.35 and 0.32 respectively. When considering the three features with the highest divergence for each model, there is only very little overlap. Only TabSyn and TVAE share the `injury` feature as one with the highest divergence. The set of highest divergence features per model is otherwise disjunct. The opposite is shown for the features with the lowest divergence. All three models contained `league` as the lowest or second lowest divergence feature. Furthermore, TabSyn and PAR both include `foot`. Furthermore, PAR and TVAE share `international_competition` as a feature with the least divergence. All of these features share a low cardinality, appearing simple to approximate for the generative models.

Table 6.18.: Comparison of average Jensen-Shannon divergence

Model	Avg. Jensen-Shannon divergence
Test	0.14
TVAE	0.35
TabSyn	0.20
PAR	0.32

For visual inspection of the distributions, four representative columns were selected, which include two numerical and two categorical features.

The first feature is `league_played_matches` which is illustrated in Figure 6.1. As shown in (b), TabSyn closely approximates the original distribution, demonstrating strong performance. TVAE (a) on the other hand struggled to capture the shape of the distribution properly and contains multiple smaller peaks. PAR (b) performs the worst, with the distribution center shifting from the ground-truth center. Whereas the distributions approximated by TabSyn and TVAE are left-skewed, PAR's output is more similar to a Gaussian distribution. Next, `market_value` is analyzed, as shown in Figure 6.2. Again, TabSyn (b) approximates the distribution very well, while TVAE (a) also approximates the general distribution well apart from adding another peak. PAR (c) exhibits a similar pattern to that observed with `league_played_matches`, producing a shifted distribution that is not left-skewed and instead resembles a Gaussian distribution.

6. Evaluation results and discussion

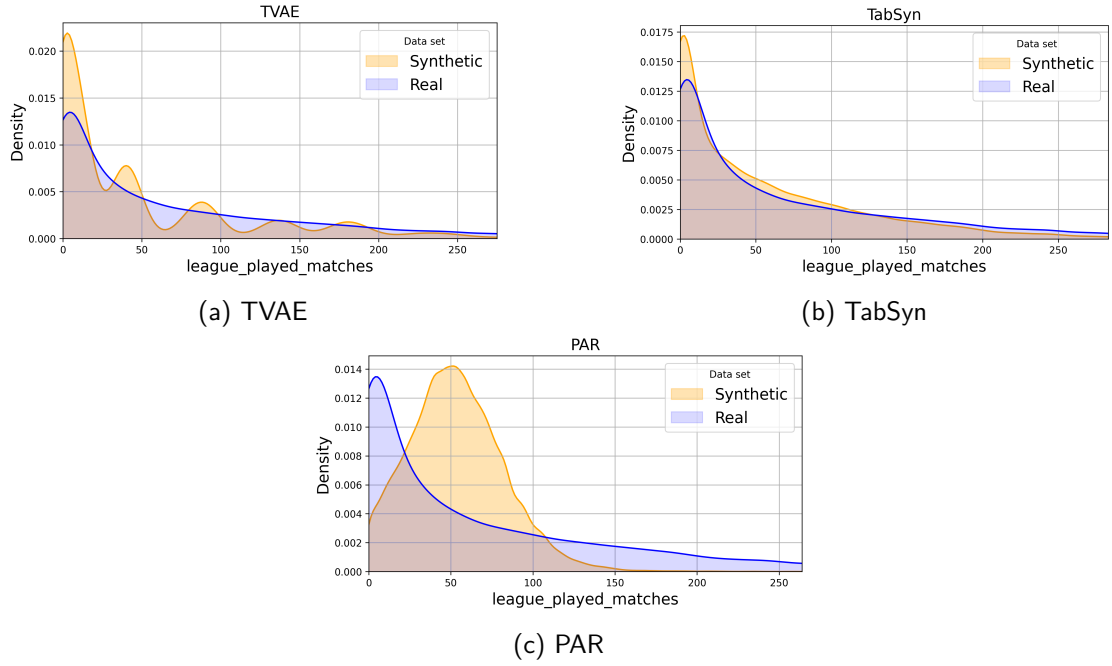


Figure 6.1.: Result of the distribution comparison to the ground truth of `league_played_matches` for TVAE (a), TabSyn (b) and PAR (c). TVAE approximates the general trend of the data, while TabSyn is the best at reflecting the ground-truth distribution. PAR exhibits a different density center and does not cover the real data properly.

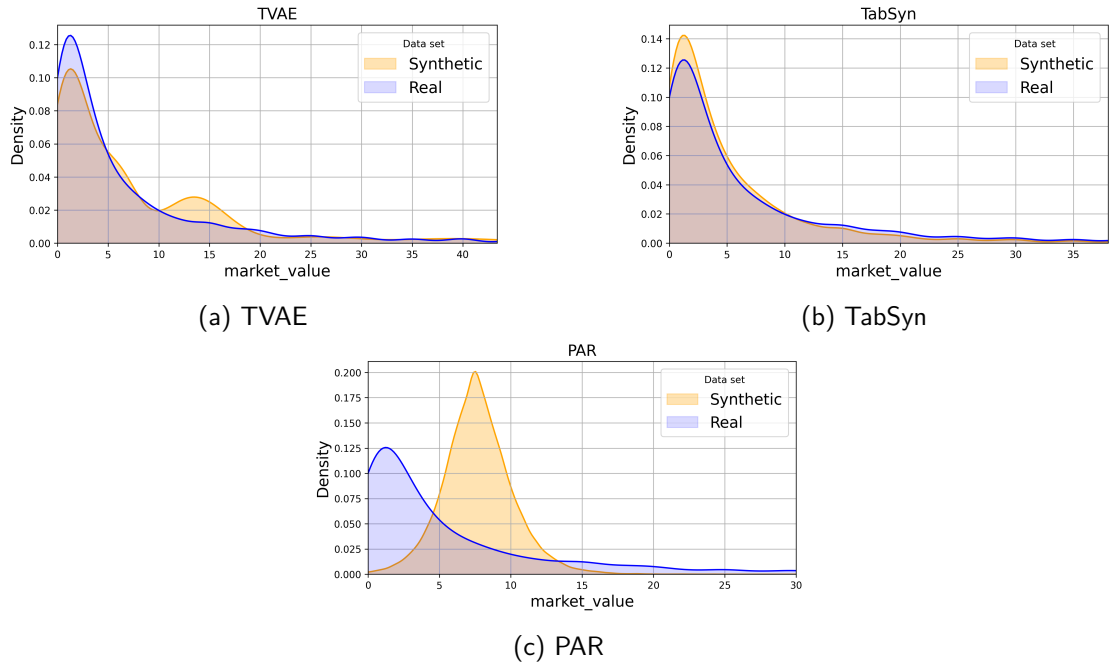


Figure 6.2.: Result of the distribution comparison to the ground truth of `market_value` for TVAE (a), TabSyn (b) and PAR (c). TVAE demonstrates a general understanding of the data structure, whereas TabSyn closely replicates the original distribution. PAR exhibits a significantly different central tendency and fails to capture the left-skewness present in the original data.

6.2. TempoBench results

Then, the categorical feature `market_value_category` is compared in Figure 6.3. Among the models, TVAE (a) performs the best in approximating the distribution of this category. While not matching the distribution exactly, it comes closest to the ground truth compared to TabSyn (b) and PAR (c). In contrast, TabSyn underrepresents the LOW category and exhibits too high proportion for the MEDIUM category. PAR deviates from the original distribution for all categories, showing the poorest performance.

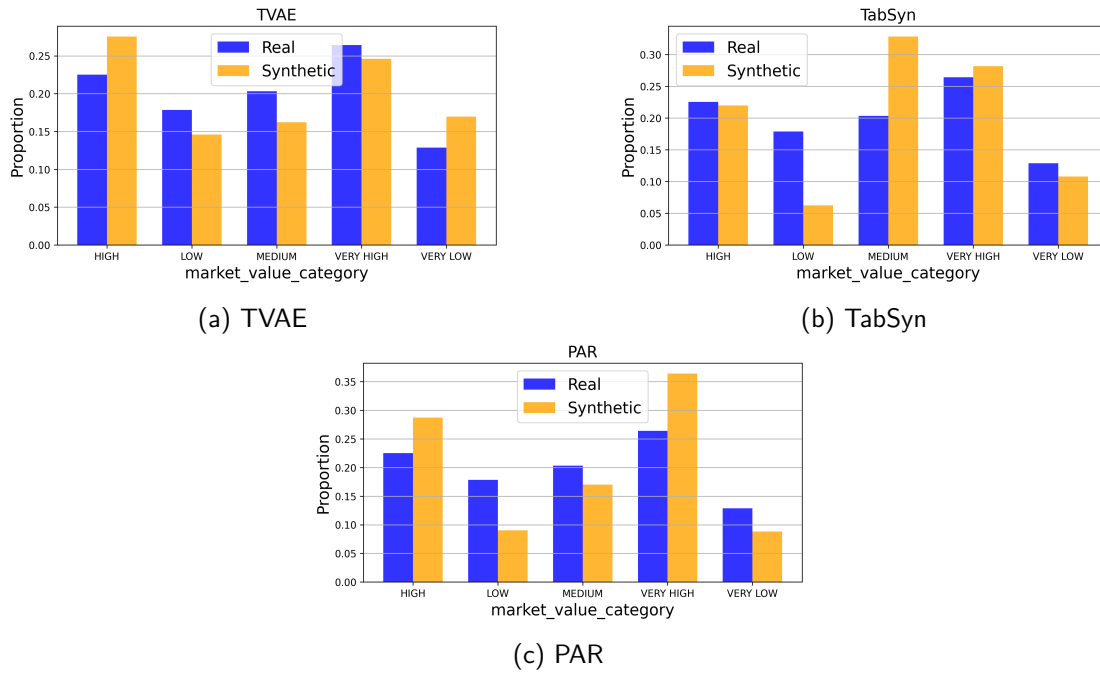


Figure 6.3.: Result of the comparison to the ground truth of `market_value_category` for TVAE (a), TabSyn (b) and PAR (c).

The categorical feature `league` is analyzed next, as shown in Figure 6.4. All models perform comparably in approximating the real distribution, with TVAE (a) showing the best performance. Both TabSyn (b) and PAR (c) contain minor deviations, although neither of the two models deviates substantially from the ground-truth distribution.

6. Evaluation results and discussion

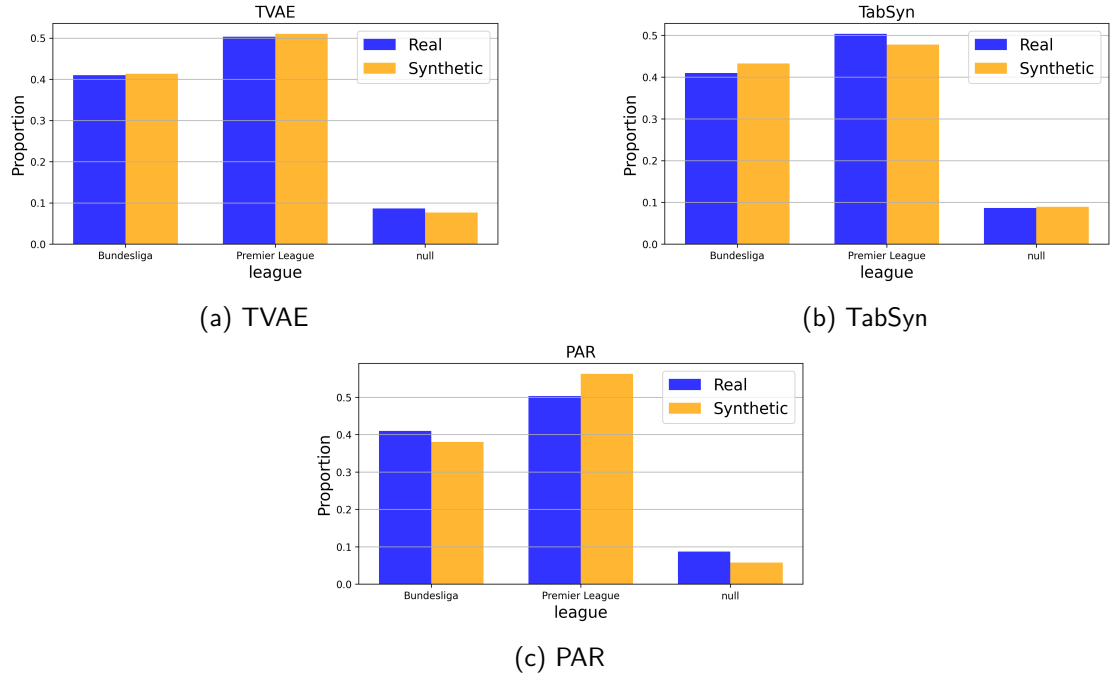


Figure 6.4.: Result of the comparison to the ground truth of `league` for TVAE (a), TabSyn (b) and PAR (c).

The remaining visualized distributions can be found in the GitHub repository ¹².

S4 Correlation The correlation results in Table 6.19 show that TabSyn is able to approximate the correlation structure of the original data the best. It achieves a normalized L1 distance of 0.06, slightly outperforming TVAE, which achieves 0.07. It also outperforms PAR which has a distance of 0.10. The maximum possible distance from the ground truth correlation matrix is 453.11.

Table 6.19.: Comparison of correlation structure

Model	Correlation matrix distance	Norm. correlation matrix distance
Test	14.4	0.03
TVAE	33.5	0.07
TabSyn	27.2	0.06
PAR	44.9	0.10

¹²https://github.com/bepo1337/tempobench/tree/main/results_for_thesis/visualizations

6.2. TempoBench results

The correlation is also visualized as correlation matrices in Figure 6.5. In the ground-truth matrix, strong correlations can be found especially in indices 14 to 20, corresponding to the player performance data such as minutes played, goals scored, and also market value and are preserved by all models. None of the models capture the correlation between indices 0 and 5 which are the features `injury_category` and `injury` which is present in the original data. However, the high correlation in indices 11 to 13, which are `club`, `league` and `coach`, is best preserved by TabSyn (b). PAR (c) does not pick up on this correlation at all. The visual inspection yields few other mentionable deviations.

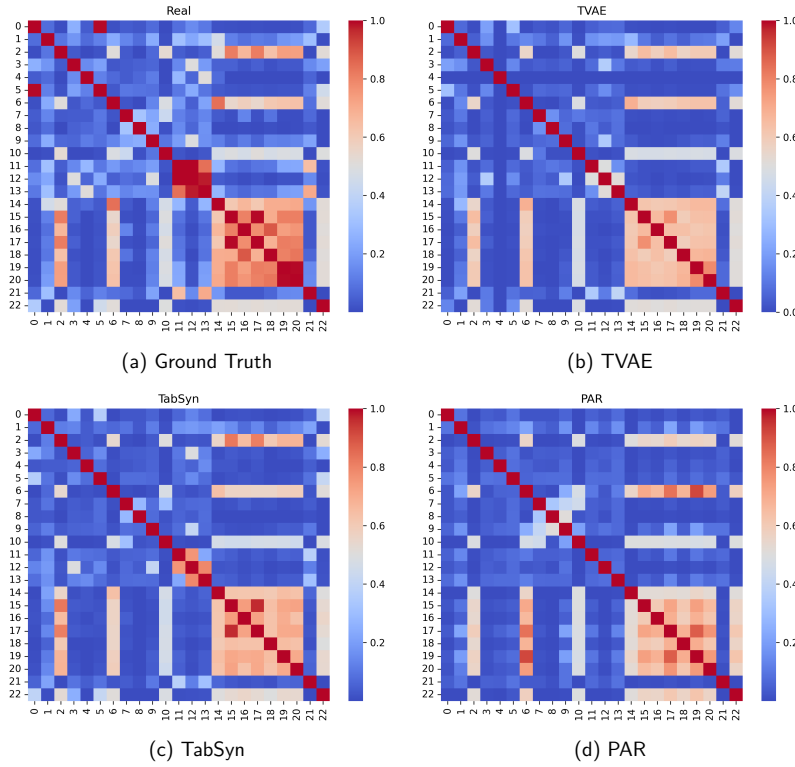


Figure 6.5.: Comparison of the correlation matrices to the ground-truth data (a). TVAE (a), TabSyn (b) and PAR (c) show very similar performance, where TabSyn appears to pick up on more correlations.

Furthermore, the average correlation matrix distance computed per player position is also reported in Table 6.20. While all generative models perform worse on the average correlation matrix per position, TabSyn still comes out on top. With an average normalized L1 distance of 0.11, it trumps TVAE with 0.13 and PAR with 0.14. Therefore, all models perform worse on the per position correlation. This is expected as this is a three-feature correlation instead of the previously discussed two-feature correlation because we now take into account the position.

6. Evaluation results and discussion

Table 6.20.: Comparison of correlation structure averaged over positions

Model	Correlation matrix distance	Norm. correlation matrix distance
Test	43.7	0.10
TVAE	57.6	0.13
TabSyn	45.5	0.11
PAR	60.2	0.14

In summary, it can be seen that TabSyn performed the best of all models for statistical fidelity. It showcases strong performance when approximating the ground-truth data. Especially for the elementary statistics of categorical attributes, it stood out as the best model. The distribution and correlation were also best characterized by TabSyn. The visual inspection also makes it clear that PAR performs significantly worse than the other models for numerical values. The categorical values were best approximated by TVAE for the evaluated visualizations. This also shows for the Jensen-Shannon divergence, where TabSyn performed best on average.

6.2.5. Temporal fidelity

For temporal fidelity, the prototype also finds vast differences between the data quality of the generative models, as will be discussed below. The Transfermarkt test set is also listed as reference value for the generative models.

T1 Incoming nearest neighbor distance The incoming nearest neighbor distance comparison can be seen in Table 6.21. This shows that TVAE produces the most similar sequences with respect to the ground-truth sequences with the custom Dynamic Time Warping distance. TabSyn has a slightly larger distance, whereas PAR produces the sequences that deviate the most from the ground truth. This can be interpreted as PAR stays the least true to the temporal dynamics present in the ground-truth data set.

Table 6.21.: Comparison of incoming nearest neighbor distance

Model	Avg. incoming nearest neighbor distance
Test	16.9
TVAE	36.5
TabSyn	48.1
PAR	71.0

T2 Mean sequence length The mean sequence length synthesized by the models compared to the ground truth can be seen in Table 6.22. Notably, PAR, which is the model that is explicitly able to model sequences, has the largest deviation in mean sequence length. While the sequences in the ground truth on average contain 14.6 entries, PAR on average contained 30.4 entries. TVAE performed the best with a deviation of only 7%, while TabSyn generated sequences with an average length of 18.8 which is a deviation of 29%. This again shows that PAR does not consistently approximate the temporal dimension.

Table 6.22.: Comparison of average sequence length

Model	Avg. sequence length	Deviation (%)
Ground truth (training)	14.6	-
Ground truth (test)	14.6	0%
TVAE	15.7	7%
TabSyn	18.8	29%
PAR	30.4	108%

The visualization of the sequence length distribution also reveals differences. The distribution of the sequence lengths from 1 to 10 and more can be seen in Figure 6.6. While all models approximate the distribution from lengths 1 to 10 closely to the ground truth, PAR (c) generates noticeably more sequences with a single entry than the ground truth. Only little differences can be seen and the models do not show anomalies.

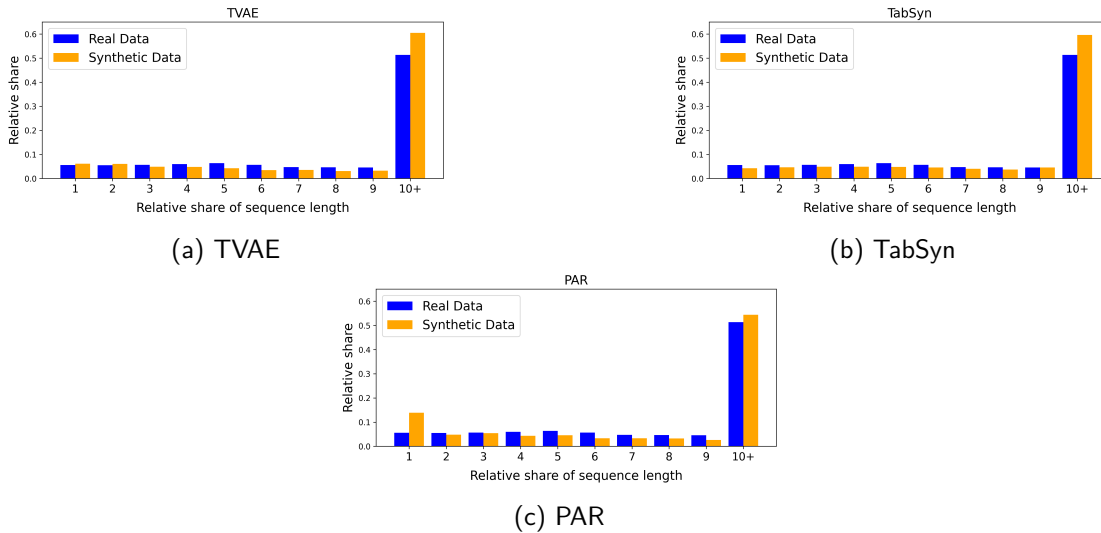


Figure 6.6.: Result of the comparison to the ground truth of sequence length to 10 for TVAE (a), TabSyn (b) and PAR (c).

6. Evaluation results and discussion

When examining the full distribution of the sequence lengths, as shown in Figure 6.7, all models follow the general left-skewed trend of the ground-truth data. Most sequences contain fewer than 20 entries for all models. None of the models perfectly replicate the sequence length distribution, but PAR (c) once again contains the most obvious deviation. Its proportion of sequences containing more than 60 entries is much higher than in the ground-truth data or the other models data sets. While the share in ground-truth data of sequences with more than 60 entries is less than 5%, it is approximately 20% in the PAR data set.

TVAE (a) on the other hand, almost does not contain any sequences longer than 49 entries. TVAE and TabSyn treat the `player_id` as just another feature and do not treat it as an entity identifier, therefore it is not surprising to see they match the statistical distribution of this variable.

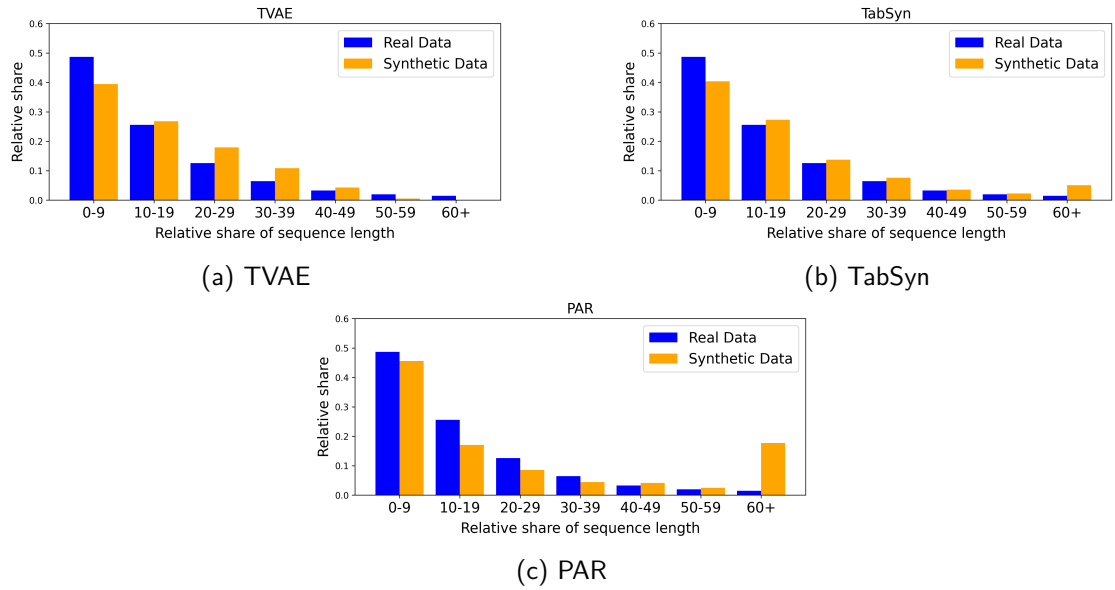


Figure 6.7.: Result of the comparison to the ground truth of sequence length for TVAE (a), TabSyn (b) and PAR (c).

T3 Mean validity The comparison of mean validity between the ground truth and the synthesized data sets that can be seen in Table 6.23, which exposes all models to perform very poorly. TVAE achieves a deviation of 96% in mean validity. While the ground-truth data has a mean validity of 168 days, TVAE has a mean validity of 329 days. TabSyn and PAR perform even worse, having a mean validity of 546 and 1119 days, respectively. This again shows that the models did not understand the relationship between the validity features in the data set.

Table 6.23.: Comparison of mean validity

Model	Mean validity in days	Deviation (%)
Ground truth (training)	168	-
Ground truth (test)	169	1%
TVAE	329	96%
TabSyn	546	225%
PAR	1119	568%

T4 Intra-class distance Significant differences in intra-class distance between the models are observed, as shown in Table 6.24. PAR displays the largest deviation, with an intra-class distance of 154.0 compared to the ground truth of only 60.7, which is a deviation of 156%. In contrast, TVAE performs the best with a deviation of only 5% from the ground truth. TabSyn deviates from the original data with 37%, showing weaker performance than TVAE. This means TVAE has the least variability in the sequences, while TabSyn deviates a bit from the real data and PAR generates vastly different sequences. Therefore, PAR generates the most diverse set of sequences, while TVAE resembles the ground-truth intra-class distance the most.

Table 6.24.: Comparison of intra-class distance

Model	Intra-class distance	Deviation from ground truth(%)
Ground truth (training)	60.7	-
Ground truth (test)	57.5	5%
TVAE	57.6	5%
TabSyn	83.0	37%
PAR	154.0	156%

T5 Update dependency For the update dependency metric, no results can be reported as all synthesized data sets had too large item sets in the association rules resulting in the algorithm running for a very long time. While for the ground-truth data, the computational time is only 3 seconds, 15 minutes was not enough time to find item sets for the synthetic data sets on an M3 MacBook Air.

Consequently, they cannot be analyzed due to limitations in computational resources and time. In general, an excessive number of item sets can be interpreted negatively for all

6. Evaluation results and discussion

models as too many features changed simultaneously.

T6 Transition matrix distance The 1-lag transition matrix distances for categorical features, as presented in Table 6.25, report very little variation across models. With an average distance ranging from 0.73 to 0.75, no significant differences can be detected.

The greatest difference in performance between the models is observed in the matrix distance for `reason`, where TVAE achieves the lowest distance with 0.43, and PAR the highest with 0.54. A general, it can be noted that higher category cardinality is associated with greater transition matrix distances. The features `international_competition` and `league` both have less than 5 possible values, and the generative models perform very well on those with respect to the other features. On the other hand, high-cardinality features such as `coach` and `club` yield poorer performance and have higher transition matrix distances. This suggests greater difficulty for models in capturing complex transitions associated with high-cardinality categorical features.

Table 6.25.: Comparison of normalized 1-lag transition matrix distance

Metric	Test	TVAE	TabSyn	PAR
Average distance	0.09	0.73	0.75	0.73
<code>injury_category</code>	0.00	0.91	0.92	0.89
<code>market_value_category</code>	0.01	0.74	0.73	0.66
<code>reason</code>	0.26	0.43	0.51	0.54
<code>injury</code>	0.27	0.78	0.82	0.72
<code>club</code>	0.03	0.92	0.96	0.98
<code>league</code>	0.00	0.49	0.47	0.47
<code>coach</code>	0.15	0.92	0.98	0.95
<code>international_competition</code>	0.00	0.64	0.63	0.63

Further temporal visualizations Other visualizations include the number of events per month (V5) describing the periodicity over the observed time interval is shown in Figure 6.8. The ground-truth data contains two peaks, one in month 01 and one in month 07 where most events are occurring. Then, between those peaks the events first decrease, and when approaching the next peak, increase again.

PAR (c) performs best by approximating the share of each month very similar to the original data. While TabSyn (b) approximates the general trend, it has a pronounced peak in month 12 and a notably low value for month 01, showing a big discrepancy to the ground-truth data for these months. Furthermore, the peak for month 07 is not as prominent.

TVAE (a) also approximates the overall trend of the distribution. However, it deviates significantly in terms of relative share. Additionally, it exhibits peaks in months 05 and 11, which are not present in the ground-truth data.

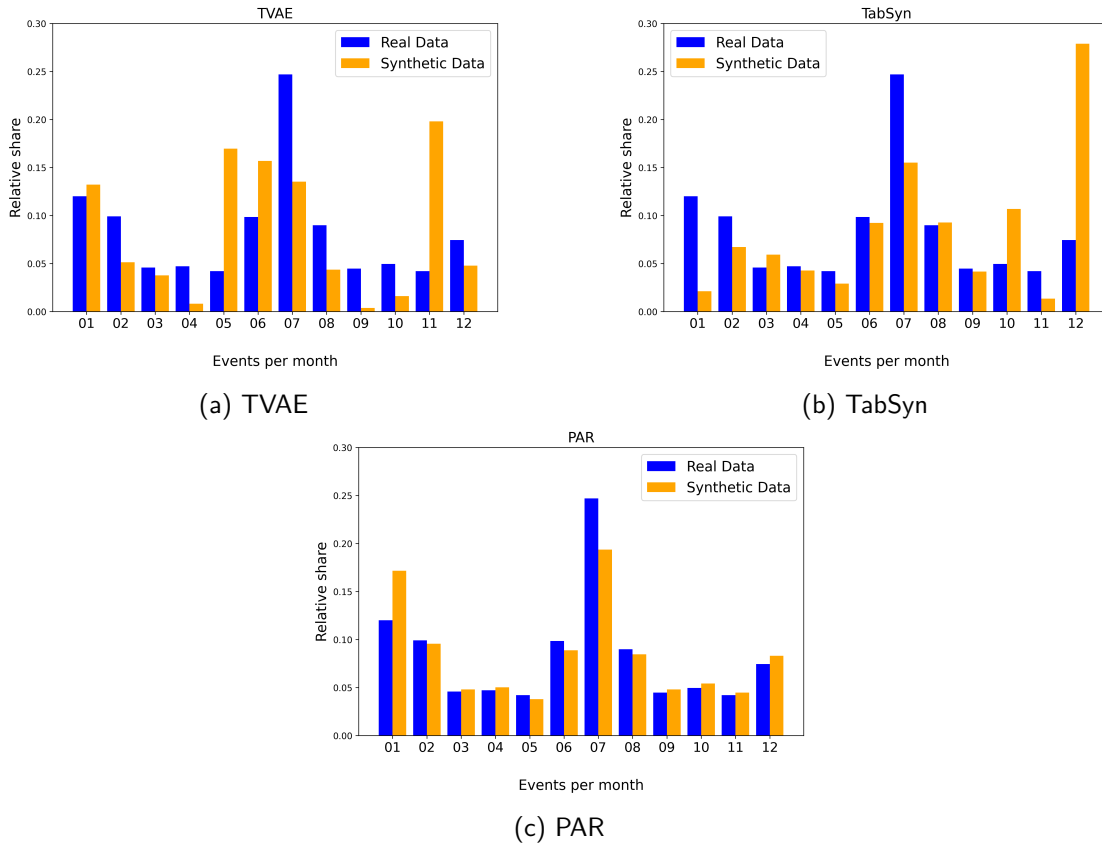


Figure 6.8.: Result of the distribution comparison to the ground truth of events per month for TVAE (a), TabSyn (b) and PAR (c).

Next, Figure 6.9 displays a more specific visualization (V6) by only looking at the injuries that occur each month. While the ground-truth data is approximately uniformly distributed with a dip between months 05 and 07, none of the models accurately replicate this pattern.

6. Evaluation results and discussion

TVAE (a) produces a different distribution, with months 05 to 07 containing above average relative shares. Additionally, it exhibits two peaks in months 01 and 11 which are not present in the ground-truth data. For TabSyn (b), a similar pattern as for the previously discussed overall sampled events is found. It features a large peak in month 12 and in general fails to reflect the ground-truth data distribution accurately. PAR (c) also diverges from the ground truth, showing peaks for months 01 and 07. A similar pattern as for the overall events can be seen here where between months 01 and 07 the relative share decreases and then increases again as it approaches the peaks in month 01 and 07.

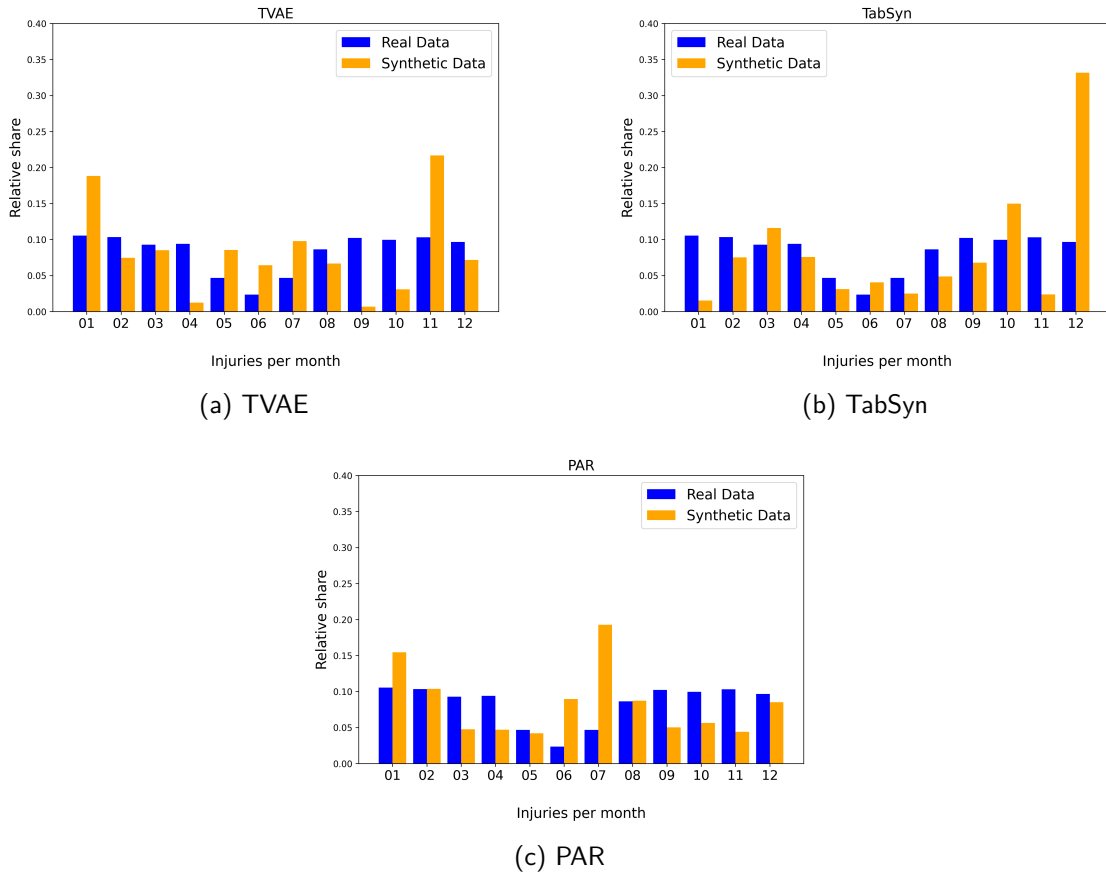


Figure 6.9.: Result of the distribution comparison to the ground truth of injuries per month for TVAE (a), TabSyn (b) and PAR (c).

Figure 6.10 illustrates the market value per season (V7) for both the Bundesliga and the Premier League. In the ground-truth data, the Premier League consistently demonstrates higher market values across all seasons. Both the Premier League's and Bundesliga's market values remain relatively stable until 2012, after which they begin to increase steadily, with the rate of growth accelerating.

TVAE (a) failed to generate samples with a market value outside of 2018, rendering it incomparable to ground truth in this context and demonstrating a complete failure in this

6.2. TempoBench results

aspect.

TabSyn (b) appears to capture the general upward trend. It accurately resembles the pattern from the ground-truth data. In contrast, PAR (c) fails entirely to reflect any trends and does not differentiate between Bundesliga and Premier League. In summary, TVAE and PAR completely fail to reproduce the observed temporal trend. Despite PAR incorporating the temporal information during training, PAR is unable to reproduce the progression of the market values for the players. The relatively successful approximation of TabSyn might be related to the model learning the interaction between the season, league, and the market values.

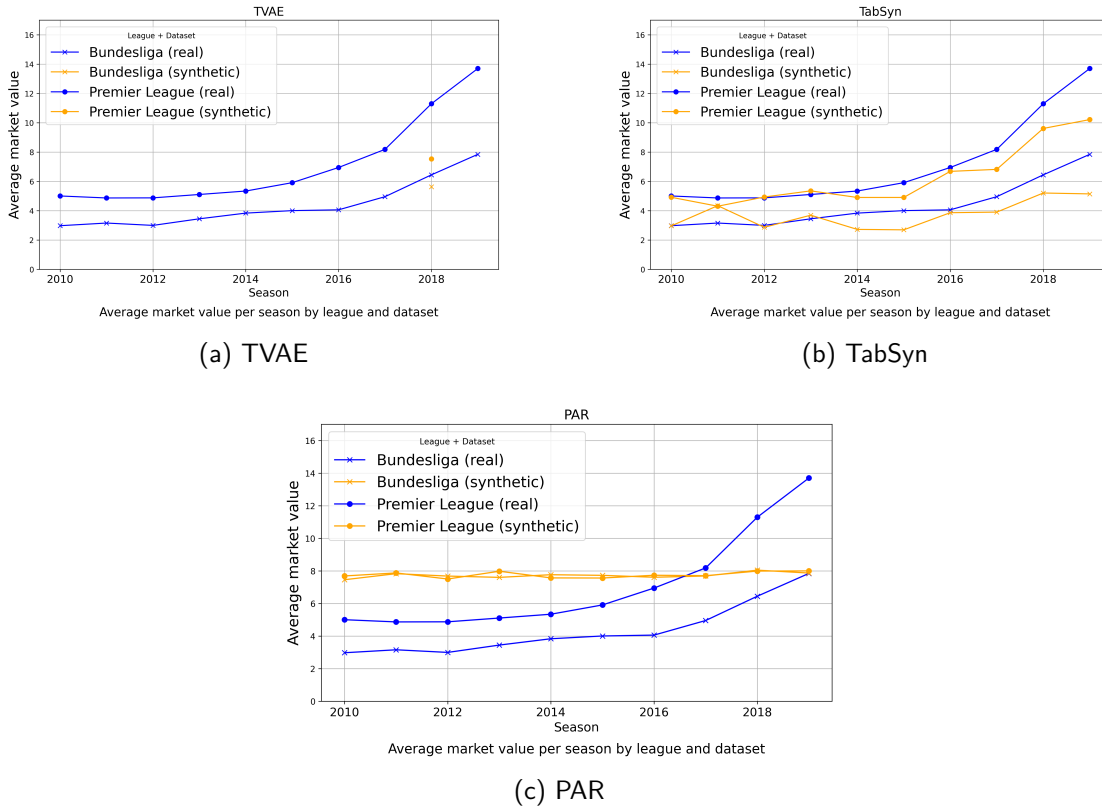


Figure 6.10.: Result of the market value per season and per league progression for TVAE (a), TabSyn (b) and PAR (c) compared to the ground truth.

Figure 6.11 presents the transfer fee per season that is closely related to the visualization of the market value per season (V8). In the ground-truth data, there are similarities to the previous market value visualization. While the Premier League consistently reports higher transfer fees than the Bundesliga, both leagues display a general upward trend over time. TVAE (a) is once again not comparable, as it only generated samples for the season 2018. TabSyn (b) has an erratic and inconsistent pattern. Most values exceed the visualization's scale, surpassing 20 million. Notably, from 2016 onward, the Bundesliga shows higher values than the Premier League. The trend for the Premier League continuously declines from at

6. Evaluation results and discussion

least 2014, which is also not a reflection of the ground-truth data.

PAR (c), in contrast to its performance in the market value per season visualization, provides a plausible approximation. The Premier League transfer fees are consistently higher than the Bundesliga transfer fees. While the absolute values do not match, the general trend of increasing transfer fees between 2010 and 2019 is preserved.

Among the models, PAR most accurately approximates the ground-truth transfer fee pattern over time. TVAE and TabSyn, on the other hand, can be considered ineffective to capture these trends. PAR's relatively strong performance can likely be attributed to taking into account the temporal dimension during training. The poor performance of TabSyn is unexpected, given it showed good performance in approximating the marginal distributions.

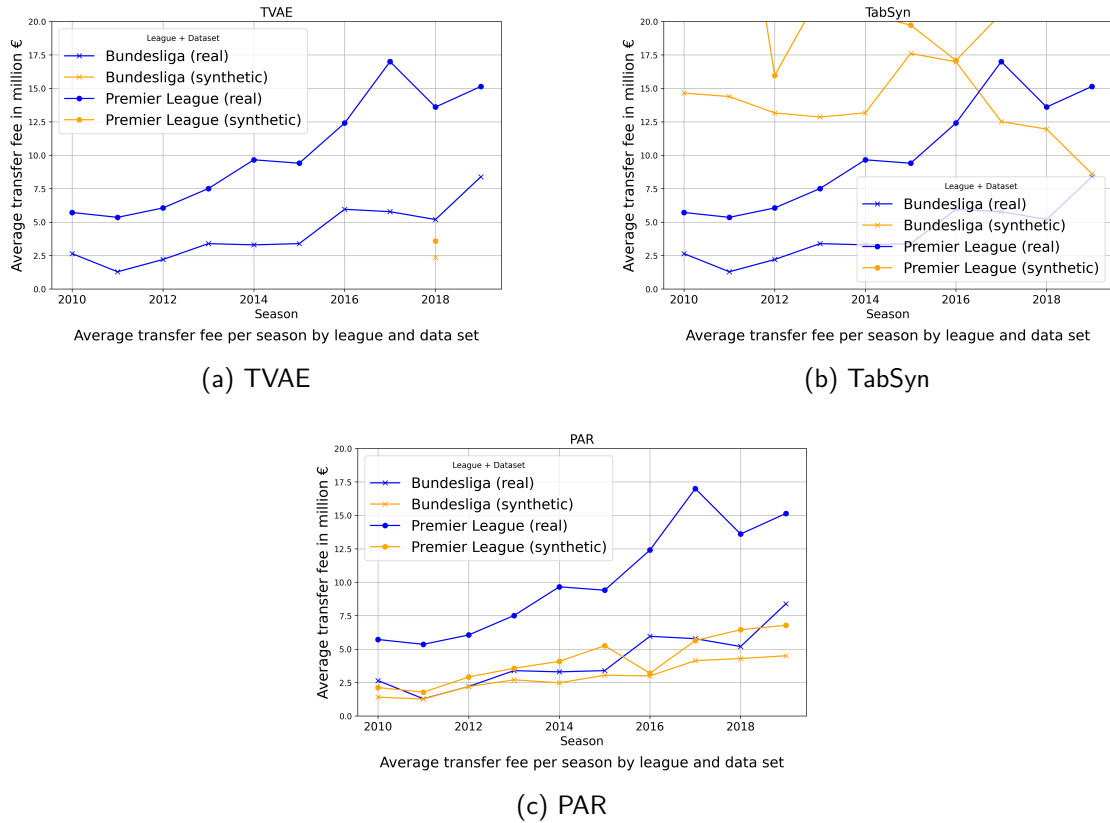


Figure 6.11.: Result of the transfer fee per season and per league progression for TVAE (a), TabSyn (b) and PAR (c) compared to the ground truth.

The final temporal visualization depicts the transfers per month (V9), as illustrated in Figure 6.12. The ground-truth data reveals a clear seasonal pattern when players are transferring. There are two distinct peaks in months 01 and 07. The general volume of transfers from month 05 to month 08 is significantly higher than the rest. This reflects the timing of the transfer windows of the leagues under consideration.

All models demonstrate a general ability to capture this seasonality. While TVAE (a) exhibits

6.2. TempoBench results

similar trends, it does not exactly match the relative proportions. TabSyn (b) matches the trend in the ground-truth data to a lesser extent and introduces spurious peaks, particularly for months 10 and 12. It also underestimates the transfers in month 01. PAR (c) performs well in replicating the periodic structure in the transfers per month. Although it contains a slightly elevated baseline during low-activity months, it accurately captures the peaks in months 01 and 07, as well as the increased volume in months 06 and 08.

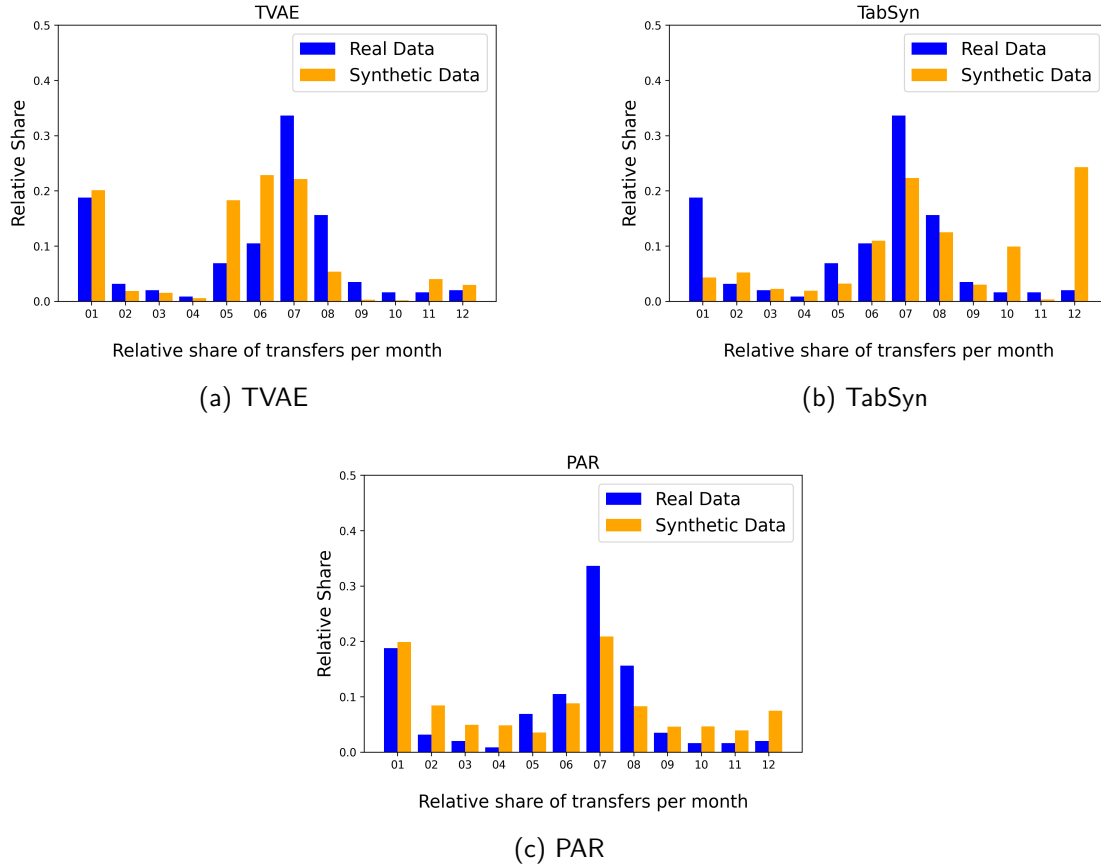


Figure 6.12.: Result of the transfers per month distribution for TVAE (a), TabSyn (b) and PAR (c) compared to the ground truth.

In summary, none of the models accurately capture the temporal characteristics of the ground-truth data. PAR shows substantial deviations in sequence length, generating a disproportionately large number of very long sequences. None of the models were able to replicate the transitions between categorical features. The visualizations highlight the distinct strengths and limitations of the models to capture temporal dynamics.

6.2.6. Downstream utility

For downstream utility, only the regression is evaluated, as the classification task did not yet produce stable results on the ground-truth data set and is therefore implemented but not evaluated in this scope of comparing the generative models.

U1 Train-on-Synthetic-Test-on-Real Regression For the regression task of predicting market value, Table 6.26 summarizes the model performance. The regression model trained on the real Transfermarkt data achieves an RMSE of 5.8 and a R^2 value of 0.79, indicating a good ability to predict the market value.

Among the synthetic data generators, TabSyn performs best, yielding an RMSE of 7.5 and R^2 of 0.65. TVAE follows with an RMSE of 8.6 and R^2 of 0.52. In contrast, PAR demonstrates the weakest performance, with an RMSE of 12.4 and R^2 of just 0.02. These results suggest that the data produced by PAR is ineffective for this downstream regression task, highlighting a significant limitation of this model.

Table 6.26.: Comparison of TSTR-Regression

Model	RMSE	R^2
Ground truth	5.8	0.79
TVAE	8.6	0.52
TabSyn	7.4	0.65
PAR	12.4	0.02

D4 Generalized Sequential Pattern Mining The results for the GSP mining reveal differences in the performance of the models as can be seen in Figure 6.13.

When mining subsequences for `injury_category`, TVAE (a) performs best with a notable F1 score of 0.74. TabSyn (b) achieves an F1 score of 0.6 while the mining does not yield any patterns present in the original data for PAR (c) and therefore an F1 score of 0.

For `league`, all models find patterns, while TabSyn performs the best with an F1 score of 0.53. TVAE achieves an F1 score of 0.44 and PAR again performs the worst with 0.29.

A similar trend is found in `club` where TabSyn achieves the best F1 score of 0.46. However, TVAE finds the least patterns here with an F1 score of 0.08. But then again, PAR has the lowest F1 score with only 0.06 while having a precision of 1.

TVAE and TabSyn finding so many patterns, even though they do not explicitly have a mechanism for temporal or sequential data, can most likely be explained by the frequent changing of columns in every row, resulting in many possible subsequences to be found. PAR's failure to identify any subsequences for `injury_category` is unexpected and requires further investigation for possible reasons.

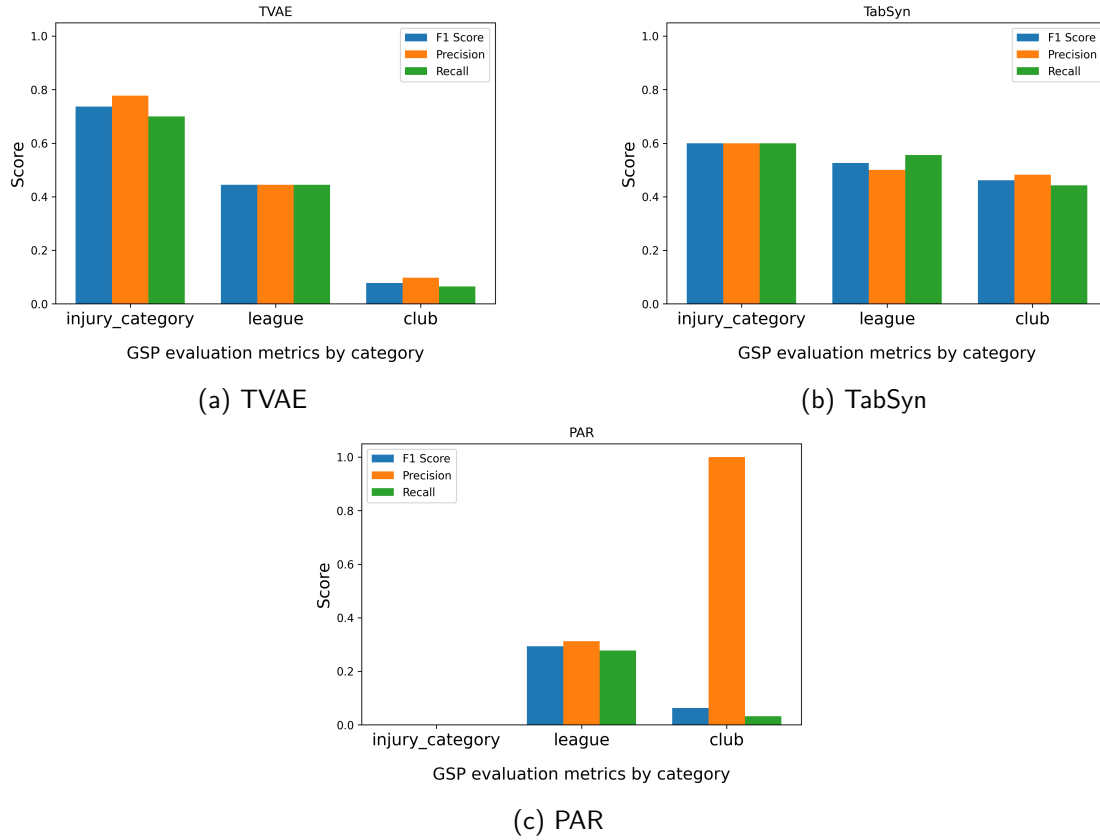


Figure 6.13.: Generalized Sequential Pattern mining results for TVAE (a), TabSyn (b), and PAR (c) showing PAR performs poorly while TVAE and TabSyn find a lot of pattern that exist in the real data.

In summary TabSyn outperforms the other two models in TSTR, while the performance of PAR indicates the data is unusable for subsequent regression downstream tasks. PAR performed poorly in pattern mining and TSTR, where the other two models achieved F1 scores of 0.6 and higher.

6.3. Discussion

When evaluated using the TempoBench prototype, none of the generative models match the ground-truth data. Each model exhibits distinct strengths and weaknesses across different aspects of this evaluation. This section provides an analysis for each model and summarizes key findings of the preceding metric analysis. In addition, an overview and a direct comparison between the models will be given in Table 6.27 after discussing each model in isolation.

TVAE TVAE does not present any irregularities in the sanity metrics with regarding row duplication and datatype mismatch where it performs without defects. When considering

6. Evaluation results and discussion

the metrics regarding the tuple validity, the model performed poorly. For domain-specific metrics, TVAE shows a high rate of violation among all metrics with the exception of the checks for maximum validity and the valid timestamps. This suggests that TVAE mostly learned the validity date range specified in the training data. The visual inspection of a sequence excerpt reveals that the sequence is unrealistic, with feature values changing in an arbitrary and inconsistent way.

In terms of statistical fidelity, TVAE demonstrates similar performance. It ranks lowest on almost all of the measured metrics when compared to the other two models, with particularly pronounced deviations in entropy for categorical features. This is confirmed in the high Jensen-Shannon divergence. The visual inspection of the correlation matrix offers insight that TVAE has managed to grasp the basic interactions between the columns, albeit without replicating their exact degree of correlation.

Regarding temporal fidelity, TVAE generates sequences that are most similar to the ground truth with respect to nearest neighbor distance, intra-class distance or sequence length distribution, and mean length. However, TVAE fails entirely in certain visual evaluations due to only generating samples with a constant `season_id`, which comprises temporal diversity. In the other visualizations, the model demonstrated limited ability to approximate the periodicity of the events. The downstream utility metrics further suggests that the data generated by TVAE is not completely unsuitable for subsequent tasks, but it lacks strong resemblance with the real data. For the pattern mining it also performed reasonably well, finding a lot of the patterns present in the ground-truth data.

In summary, TVAE's strength is approximating the sequence length and variability within the sequences. In contrast, almost all other evaluation aspects can be described as weaknesses of the model. This is likely due to many metrics being aimed at temporal properties that are also present in the domain-specific metrics, where TVAE does not explicitly model temporal data and has no inherent concept of multi-entity temporal data.

TabSyn TabSyn does not exhibit any irregularities with respect to the sanity metrics, generating a valid data set and not duplicating rows from the real data set. Similarly to TVAE, this does not extend to the sanity metrics regarding the tuples' validity timestamps.

Again, its performance in domain-specific metrics reveals limitations. For example, the monotonic increase within a player's trajectory is not preserved. Similarly, it violates constraints related to consistency in static data. Inspection of an example sequence shows implausible dynamics, with features that vary unpredictably and lack consistency.

In terms of statistical fidelity, TabSyn performs very well. It most accurately captures the modes of categorical variables and has the smallest deviation in entropy, indicating a close match with the ground truth. Furthermore, it achieves the best scores for the Jensen-Shannon divergence, demonstrating the best ability to approximate the distribution of the ground-truth data. This impression is further manifested when performing the visual inspection of the distributions, where TabSyn closely mirrors the underlying distributions. It also performs best on the correlation that matches the general trend of the ground-truth data.

Regarding temporal fidelity, TabSyn maintains consistency to the distribution of sequence length. However, it deviated a lot from the mean validity and also underperforms in the

transition matrix distance or approximating the transfer fee per season. Although the model captures the trend for market value over time reasonably well, it struggles to reproduce the periodic structure of the event occurrences. In terms of downstream utility, TabSyn outperforms the other models. However, the usability of the data remains questionable, given the achieved R^2 value of 0.65. For the pattern mining it managed to find patterns, while not performing exceptionally well.

In summary, TabSyn's strengths lie in modeling statistical fidelity that is demonstrated in elementary statistics, correlation, and Jensen-Shannon divergence. Limitations are obvious in domain-specific metrics and temporal evaluations. TabSyn lacks explicit mechanisms to incorporate temporal dimensions. Given these architectural limitations, it would have been surprising to see TabSyn performing very well in temporal metrics and visualizations.

PAR PAR is the only model among those evaluated that explicitly incorporates multi-entity sequential data during training. Like the other models, passed the first two sanity checks, synthesizing data adhering to the ground-truth data types and not replicating any rows. Again, the metrics regarding tuple validity revealed a first model weakness.

In the domain-specific metrics, PAR displays weaknesses in almost all evaluated metrics. One notable exception is its adherence to the static feature constraint, where it correctly preserves the fixed values over the lifetime of a sequence. This can be attributed to PAR's explicit modeling this type of data that it refers to as "context data" during training. However, its performance on other domain metrics is poor. The inspection of a sequence excerpt reveals that PAR violates the least constraints and most closely resembles the ground-truth compared to the other two models but is still very poor.

With respect to statistical fidelity, PAR again performs poorly. It demonstrates big deviations in the elementary statistics of numerical values and struggles to approximate feature distributions. This is evident in both the Jensen-Shannon divergence and the visualizations, where PAR-generated features had peaks that differed from the ground-truth data. Furthermore, the correlation it produced was also the weakest out of the evaluated models for the whole data set and also for the per position correlation matrices. This indicates a lack of understanding of the intra-row relationships.

In the area of temporal fidelity, PAR deviates from the ground-truth data. The distribution of sequence lengths shows that it generates sequences that are very long in excess proportion. Additionally, the sequences are the furthest from the ground truth out of the evaluated models. The variability within the data set as measured by the intra-class distance is the highest for PAR. The transition for categorical features is not well approximated by PAR.

Nonetheless, the visualizations reveal that PAR is very good at approximating how many events occur in each month. This is valid for the transfer per month. However, the evolution of market value per season reveals that PAR basically generates samples that do not have any inherent trend over time. They lack meaningful progression over time and fail to reflect patterns such as the consistent higher market values in the Premier League than in the Bundesliga and the upward trend over the observed period.

In terms of downstream utility, PAR is effectively unusable. It achieves an extremely low R^2 of 0.02 in TSTR. It also performs very poorly on the pattern mining task (GSP) where

6. Evaluation results and discussion

it does not find any of the patterns from the ground-truth data for `injury_category`. For the other two categories it finds patterns, but the results were weak. The synthesized data offers no downstream value in this context.

The weaknesses in the temporal apprehension of the synthesized data set can most likely be attributed to PAR only modeling the sequence index and not being able to handle other temporal attributes in the remaining features. This shortcoming likely contributed to its failure to model essential temporal relationships, particularly those related to validity dates.

Table 6.27 shows an overview of the performance of the models and where their strengths and weaknesses lie across the measured categories. The thresholds used for differentiation are provided in Appendix A.4. Metrics like INN are excluded from this table, as their thresholds are highly dependent on the intended goal of the synthesis, whether to closely replicate the training data or generate a more diverse set of samples. It is especially obvious that all models performed very poorly on the temporal metrics. This shows the usefulness and necessity of introducing an evaluation benchmark like TempoBench.

TempoBench framework prototype The proposed TempoBench framework and its accompanying prototype demonstrate the utility of the evaluation approach. It effectively identifies critical weaknesses by quantifying to which extent temporal constraints are satisfied and how well temporal characteristics are captured by the models. At the same time, it provides a holistic overview through additional categories, such as statistical fidelity and downstream utility.

The consistently poor performance for all evaluated models shows the effectiveness of TempoBench in assessing whether generative models can accurately reflect the structures in the real data. Moreover, the weak results on the domain-specific metrics for all models highlights the importance of using these metrics, which check the unique characteristics of a data set. Finally, the variation in model performance between categories but also within categories demonstrates that TempoBench offers a comprehensive and nuanced assessment of the synthetic data quality. This wide spectrum allows for comparison between models and lays a baseline for future evaluation of models.

Table 6.27.: Overview evaluation result and comparison between models

Category	Metric	TVAE	TabSyn	PAR
Sanity	Duplicate rows	++	++	++
	Datatype mismatch	++	++	++
	Tuple validity overlap	--	--	--
	Validity end before start	--	--	--
Domain	Manual inspection	--	--	--
	Monotonic increase	--	--	--
	Static data remains unchanged	--	--	++
	Timestamp min. and max.	+	o	-
	Maximum validity time	+	+	+
Statistical	Elementary statistics	o	+	o
	Jensen-Shannon divergence	o	+	o
	Numerical distribution visualization	+	++	-
	Categorical distribution visualization	++	+	+
	Correlation	+	++	+
Temporal	Mean sequence length	++	o	--
	Mean validity	--	--	--
	Intra-class distance	+	-	--
	Transition matrix distance	--	--	--
	Temporal visualizations	-	o	o
Downstream utility	TSTR-Regression	o	+	--
	GSP	-	o	--

7. Conclusion

This chapter gives a summary of the thesis and answers the research questions. The limitations of this work are also mentioned, and this chapter concludes with an outlook to future work.

7.1. Summary

In this thesis, TempoBench is developed and presented, which is a benchmarking framework for synthetic temporal tabular data. A soccer data set by Transfermarkt is directly incorporated into the framework. TempoBench allows the evaluation of generative models with respect to different categories achieving a holistic view of the synthesized data set.

Conceptual design of an evaluation framework The initial research question was what categories and metrics can be used in a unified evaluation benchmarking framework for temporal tabular data. For this, TempoBench has been proposed with a conceptual design covering a comprehensive list of categories and metrics that was found to evaluate temporal tabular data and aggregated into a single unified framework. In addition, metrics that are not yet used in other evaluation frameworks are proposed, adding to the landscape of evaluation metrics and broadening the possible aspects to evaluate. A prototype has been implemented that contains a representative subset of metrics from the proposed framework. A novel soccer data set from Transfermarkt has been directly incorporated in the TempoBench framework that further diversifies the field of temporal tabular data research. In combination with the metrics, it allows for a comprehensive evaluation of generative models.

Evaluation of existing generative models The second research question was how current generative models perform when synthesizing temporal tabular data. For this, the TempoBench prototype has been used. Three models, TVAE [159], TabSyn [165] and PAR [166] have been trained on the Transfermarkt data set incorporated in the TempoBench framework. While only PAR contains explicit modeling of a temporal dimension, they all perform very poorly on the implemented evaluation metrics. This shows that TempoBench is capable of evaluating models and highlighting their strengths and weaknesses. It also indicates that other models than the ones evaluated are needed to synthesize this type of temporal data, creating a baseline for further research in this area.

The TempoBench code and Transfermarkt data set are also published under <https://github.com/bepo1337/tempobench>.

7. Conclusion

7.2. Limitations

Limitations are also present in the TempoBench framework, its prototypical implementation, and the evaluation of the generative models.

The first limitation is the prototype that only implements a subset of the proposed evaluation framework, allowing only for an evaluation limited to these metrics. The whole concept of TempoBench was not tested in its entirety.

For training generative models, the split of the training and validation data was also slightly different for the models as they split the data in different ways. While TVAE splits the data after converting the training data set to a tensor representation, TabSyn splits the data before training the model, allowing for a controlled split of training and validation set.

For the custom dynamic time warping function used in the intra-class distance and incoming nearest neighbor distance the weight of the categorical differences has been arbitrarily chosen at 0.3. This requires further experimentation and research to find a good weight for the categorical attributes.

Furthermore, for generating the synthetic data sets, only one data set has been evaluated for each model instead of multiple runs, allowing for variance to be in the results. For generative models, most of the default parameters have also been used for training. This might also not have yielded the optimal results for the given model.

The same pre- and postprocessing has also been used for all the models where a tailored strategy for each model might yield better results.

When comparing the models in the overview table, the thresholds that have been chosen are subjective and do not allow for direct comparison across metrics as they are not standardized. They are meant to give a simple overview.

7.3. Future work

There exist many possibilities to further the research in this area that are pointed out in this section.

Implement more metrics An important direction for future work is the implementation of additional metrics to enable a more holistic evaluation of generative models. This would support a more nuanced analysis across different categories and provide deeper insights into model performance.

Introduction of other categories Certain aspects of the data quality are currently not addressed, such as privacy-related metrics. In future work, especially when incorporating data sets where privacy is as critical as performance, it will be essential to include appropriate privacy-focused metrics.

Integrate generative models in the framework To facilitate easier comparison of generative models, they can also be integrated into the TempoBench framework, enabling streamlined generation of new data. Integrating the training procedure into the framework can

lead to a more controlled environment. This would increase the comparability and increase transparency in how the sampled data sets have been trained.

Introduce other data sets Currently, only the Transfermarkt data set is included in the TempoBench framework. TempoBench's generic conceptual approach makes it possible to add other data sets that introduce further possibilities to compare generative models. Different types of temporal data sets can be integrated, to represent not only the current temporal model with validity timestamps, but also with just a sequence index, for example. The temporal model of the existing Transfermarkt data set can also be changed to use sequence indices instead of the validity intervals.

More generic implementation The current prototypical implementation of TempoBench is specifically tailored to the Transfermarkt data set. Therefore, to support additional data sets, the framework prototype must be generalized to enable easier integration of new data sets.

Evaluate additional generative models Additional generative models should be evaluated with the TempoBench framework. The models evaluated within the scope of this thesis performed very poorly. This can be attributed to the fact that they are not explicitly designed to generate the multi-entity temporal data that is present in the Transfermarkt data set. For example, TimeAutoDiff [140] can be evaluated next, as it can handle multi-sequence data sets and showed promising results on other publicly available data sets. Evaluating different synthesis approaches such as Kamino [50] which use denial constraints can also be a possible prospect of future work to model temporal properties with these constraints. They can integrate properties such as monotonic increase for columns in relation to the timestamps as a constraint.

Evaluate extending existing sequences The evaluated synthesis approaches only evaluated creating completely new sequences. We did not look at extending existing sequences and how proficient synthesizers are in this regard. Therefore this is another aspect of further research.

Bibliography

- [1] Rakesh Agrawal, Tomasz Imieliński, and Arun Swami. Mining association rules between sets of items in large databases. *SIGMOD Rec.*, 22(2):207–216, June 1993.
- [2] Haleh Akrami, Sergül Aydınoğlu, Richard M. Leahy, and Anand A. Joshi. Robust variational autoencoder for tabular data with beta divergence. *CoRR*, abs/2006.08204, 2020.
- [3] Moustafa Alzantot, Supriyo Chakraborty, and Mani B. Srivastava. Sensegen: A deep learning architecture for synthetic sensor data generation. *CoRR*, abs/1701.08886, 2017.
- [4] Amazon Science. (iclr 2024 oral) mixed-type tabular data synthesis with score-based diffusion in latent space. <https://github.com/amazon-science/tabsyn>. Accessed: 2025-04-17.
- [5] Constanza L Andaur Navarro, Johanna A A Damen, Toshihiko Takada, Steven W J Nijman, Paula Dhiman, Jie Ma, Gary S Collins, Ram Bajpai, Richard D Riley, Karel G M Moons, and Lotty Hooft. Risk of bias in studies on prediction models developed using supervised machine learning techniques: systematic review. *BMJ*, 375, 2021.
- [6] Yihao Ang, Qiang Huang, Yifan Bao, Anthony K. H. Tung, and Zhiyong Huang. Tsgbench: Time series generation benchmark. *Proc. VLDB Endow.*, 17(3):305–318, 2023.
- [7] Vaibhav Anu, Wenhua Hu, Jeffrey C Carver, Gursimran S Walia, and Gary Bradshaw. Development of a human error taxonomy for software requirements: A systematic literature review. *Information and Software Technology*, 103:112–124, 2018.
- [8] Patricia A. Apellániz, Juan Parras, and Santiago Zazo. An improved tabular data generator with VAE-GMM integration. In *32nd European Signal Processing Conference, EUSIPCO 2024, Lyon, France, August 26-30, 2024*, pages 1886–1890. IEEE, 2024.
- [9] Martín Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein GAN. *CoRR*, abs/1701.07875, 2017.
- [10] Hiba Arnout, Johanna Bronner, and Thomas Runkler. Evaluation of generative adversarial networks for time series data. In *2021 International Joint Conference on Neural Networks (IJCNN)*, pages 1–7, 2021.

- [11] Anmol Arora and Ananya Arora. Machine learning models trained on synthetic datasets of multiple sample sizes for the use of predicting blood pressure from clinical data in a national dataset. *PLoS One*, 18(3):e0283094, March 2023.
- [12] Amit Bahree. *Generative AI in action*. Manning Publications, New York, NY, October 2024.
- [13] André Bauer, Simon Trapp, Michael Stenger, Robert Leppich, Samuel Kounev, Mark Leznik, Kyle Chard, and Ian T. Foster. Comprehensive exploration of synthetic data generation: A survey. *CoRR*, abs/2401.02524, 2024.
- [14] Karan Bhanot, Miao Qi, John S Erickson, Isabelle Guyon, and Kristin P Bennett. The problem of fairness in synthetic healthcare data. *Entropy (Basel)*, 23(9):1165, September 2021.
- [15] Vincent Bindschaedler, Reza Shokri, and Carl A. Gunter. Plausible deniability for privacy-preserving data synthesis. *Proc. VLDB Endow.*, 10(5):481–492, January 2017.
- [16] Vadim Borisov, Tobias Leemann, Kathrin Seßler, Johannes Haug, Martin Pawelczyk, and Gjergji Kasneci. Deep neural networks and tabular data: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 35(6):7499–7519, June 2024.
- [17] Vadim Borisov, Kathrin Seßler, Tobias Leemann, Martin Pawelczyk, and Gjergji Kasneci. Language models are realistic tabular data generators. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.
- [18] Stavroula Bourou, Andreas El Saer, Terpsichori Helen Velivassaki, Artemis C. Voulkidis, and Theodore B. Zahariadis. A review of tabular data synthesis using gans on an IDS dataset. *Inf.*, 12(9):375, 2021.
- [19] Nicolas Boursin, Carl Remlinger, and Joseph Mikael. Deep generators on commodity markets application to deep hedging. *Risks*, 11(1):7, 2023.
- [20] Anna L. Buczak and Erhan Guven. A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Commun. Surv. Tutorials*, 18(2):1153–1176, 2016.
- [21] Patricio Cerda and Gaël Varoquaux. Encoding high-cardinality string categorical variables. *IEEE Trans. Knowl. Data Eng.*, 34(3):1164–1176, 2022.
- [22] Donald D. Chamberlin and Raymond F. Boyce. SEQUEL: A structured english query language. In Gene Altshuler, Randall Rustin, and Bernard D. Plagman, editors, *Proceedings of 1974 ACM-SIGMOD Workshop on Data Description, Access and Control, Ann Arbor, Michigan, USA, May 1-3, 1974, 2 Volumes*, pages 249–264. ACM, 1974.
- [23] Krishnendu Chaudhury. *Math and architectures of deep learning*. Manning Publications, New York, NY, March 2024.

- [24] Kukjin Choi, Jihun Yi, Changhwa Park, and Sungroh Yoon. Deep learning for anomaly detection in time-series data: Review, analysis, and guidelines. *IEEE Access*, 9:120043–120065, 2021.
- [25] Radostin Cholakov and Todor Kolev. Transformers predicting the future. applying attention in next-frame and time series forecasting. *CoRR*, abs/2108.08224, 2021.
- [26] Vikram S Chundawat, Ayush K Tarun, Murari Mandal, Mukund Lahoti, and Pratik Narang. A universal metric for robust evaluation of synthetic tabular data. *IEEE Transactions on Artificial Intelligence*, 5(1):300–309, 2022.
- [27] Jacob Cohen. *Statistical power analysis for the behavioral sciences*. routledge, 2013.
- [28] Harald Cramér. *Mathematical Methods of Statistics (PMS-9)*. Princeton University Press, 1999.
- [29] Hector Cuesta and Sampath Kumar. *Practical data analysis*. Packt Publishing, Birmingham, England, 2 edition, 2016.
- [30] Jules Damji, Brooke Wenig, Tathagata Das, and Denny Lee. *Learning spark*. O'Reilly Media, Sebastopol, CA, 2 edition, August 2020.
- [31] Fida K. Dankar, Mahmoud K. Ibrahim, and Leila Ismail. A multi-dimensional evaluation of synthetic data generators. *IEEE Access*, 10:11147–11158, 2022.
- [32] Sajad Darabi and Yotam Elor. Synthesising multi-modal minority samples for tabular data. *CoRR*, abs/2105.08204, 2021.
- [33] Maria F. Davila R., Sven Groen, Fabian Panse, and Wolfram Wingerath. Navigating tabular data synthesis research understanding user needs and tool capabilities. *SIGMOD Rec.*, 53(4):18–35, 2025.
- [34] J. Demetrovics and V. D. Thi. Some results about functional dependencies. *Acta Cybern.*, 8(3):273–278, May 1988.
- [35] Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. Timevae: A variational auto-encoder for multivariate time series generation. *CoRR*, abs/2111.08095, 2021.
- [36] Prafulla Dhariwal and Alex Nichol. Diffusion models beat gans on image synthesis. *CoRR*, abs/2105.05233, 2021.
- [37] Anne Driemel, Amer Krivosija, and Christian Sohler. Clustering time series under the fréchet distance. In Robert Krauthgamer, editor, *Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2016, Arlington, VA, USA, January 10-12, 2016*, pages 766–785. SIAM, 2016.
- [38] Yuntao Du and Ninghui Li. Systematic assessment of tabular data synthesis algorithms. *arXiv preprint arXiv:2402.06806*, 2024.

- [39] Cynthia Dwork. Differential privacy. In *Proceedings of the 33rd International Colloquium on Automata, Languages and Programming (ICALP)*, pages 1–12. Springer, 2006.
- [40] Peter Eigenschink, Thomas Reutterer, Stefan Vamosi, Ralf Vamosi, Chang Sun, and Klaudius Kalcher. Deep generative models for synthetic data: A survey. *IEEE Access*, 11:47304–47320, 2023.
- [41] Gabriel Eilertsen, Apostolia Tsirikoglou, Claes Lundström, and Jonas Unger. Ensembles of gans for synthetic training data generation. *CoRR*, abs/2104.11797, 2021.
- [42] Clément Elliker, Emeric Tonnelier, and Aymen Shabou. Coherent multi-table data synthesis for tabular and time-series data with gans. In *Proceedings of the Fourteenth ACM Conference on Data and Application Security and Privacy, CODASPY '24*, page 245–252, New York, NY, USA, 2024. Association for Computing Machinery.
- [43] Cristóbal Esteban, Stephanie L. Hyland, and Gunnar Rätsch. Real-valued (medical) time series generation with recurrent conditional gans. *CoRR*, abs/1706.02633, 2017.
- [44] Joao Fonseca and Fernando Bacao. Tabular and latent space synthetic data generation: a literature review. *Journal of Big Data*, 10(1):1–25, 2023.
- [45] David Foster. *Generative deep learning*. O'Reilly Media, Sebastopol, CA, 2 edition, May 2023.
- [46] Philippe Fournier-Viger, Jerry Chun-Wei Lin, Rage Uday Kiran, Yun Sing Koh, and Rincy Thomas. A survey of sequential pattern mining. *Data Science and Pattern Recognition*, 1(1):54–77, 2017.
- [47] Sarah Friedrich and Tim Friede. On the role of benchmarking data sets and simulations in method comparison studies. *Biometrical Journal*, 66(1):e2200212, 2024.
- [48] Jiahui Gao, Renjie Pi, Yong Lin, Hang Xu, Jiacheng Ye, Zhiyong Wu, Weizhong Zhang, Xiaodan Liang, Zhenguo Li, and Lingpeng Kong. Self-guided noise-free data generation for efficient zero-shot learning. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.
- [49] Josh Gardner, Juan C. Perdomo, and Ludwig Schmidt. Large scale transfer learning for tabular data via language modeling. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.
- [50] Chang Ge, Shubhankar Mohapatra, Xi He, and Ihab F. Ilyas. Kamino: Constraint-aware differentially private data synthesis. *Proc. VLDB Endow.*, 14(10):1886–1899, 2021.

- [51] Aurélien Géron. *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems*. O'Reilly Media, Inc., 2022.
- [52] Andre Goncalves, Priyadip Ray, Braden Soper, Jennifer Stevens, Linda Coyle, and Ana Paula Sales. Generation and evaluation of synthetic patient data. *BMC Med. Res. Methodol.*, 20(1):108, December 2020.
- [53] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron C. Courville, and Yoshua Bengio. Generative adversarial networks. *CoRR*, abs/1406.2661, 2014.
- [54] R. W. Hamming. Error detecting and error correcting codes. *The Bell System Technical Journal*, 29(2):147–160, 1950.
- [55] John T Hancock and Taghi M Khoshgoftaar. Survey on categorical data for neural networks. *Journal of big data*, 7(1):28, 2020.
- [56] Lasse Hansen, Nabeel Seedat, Mihaela van der Schaar, and Andrija Petrovic. Reimagining synthetic tabular data generation through data-centric ai: A comprehensive benchmark. *Advances in neural information processing systems*, 36:33781–33823, 2023.
- [57] Jan L. Harrington. *Relational Database Design and Implementation*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 4th edition, 2016.
- [58] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020.
- [59] Debapriya Hazra and Yung-Cheol Byun. Synsiggan: Generative adversarial networks for synthetic biomedical signal generation. *Biology*, 9(12):441, 2020.
- [60] Mikel Hernandez, Gorka Epelde, Ane Alberdi, Rodrigo Cilla, and Debbie Rankin. Synthetic data generation for tabular health records: A systematic review. *Neurocomputing*, 493:28–45, 2022.
- [61] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin, editors, *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, pages 6840–6851, 2020.

- [62] Yanqun Huang, Muyu Wang, Zhimin Zheng, Moxuan Ma, Xiaolu Fei, Lan Wei, and Hui Chen. Representation of time-varying and time-invariant emr data and its application in modeling outcome prediction for heart failure patients. *Journal of Biomedical Informatics*, 143:104427, 2023.
- [63] Valter Hudovernik, Martin Jurkovic, and Erik Strumbelj. Benchmarking the fidelity and utility of synthetic relational data. *CoRR*, abs/2410.03411, 2024.
- [64] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.
- [65] YData Labs Inc. Synthetic data quality report. https://docs.sdk.ydata.ai/latest/synthetic_data/synthetic_data_quality/report_pdf/. Accessed: 2025-04-30.
- [66] YData Labs Inc. Synthetic multivariate time series data. <https://ydata.ai/resources/synthetic-multivariate-time-series-data/>. Accessed: 2025-04-30.
- [67] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. *CoRR*, abs/1611.01144, 2016.
- [68] James Jordon, Lukasz Szpruch, Florimond Houssiau, Mirko Bottarelli, Giovanni Cherubin, Carsten Maple, Samuel N. Cohen, and Adrian Weller. Synthetic data - what, why and how? *CoRR*, abs/2205.03257, 2022.
- [69] Manu Joseph. *Modern Time Series Forecasting with Python*. Packt Publishing, Birmingham, England, November 2022.
- [70] Manu Joseph. *Modern Time Series Forecasting with Python: Explore industry-ready time series forecasting using modern machine learning and deep learning*. Packt Publishing Ltd, 2022.
- [71] João Marcelo Borovina Josko. A formal taxonomy of temporal data defects. In Hakim Hacid, Quan Z. Sheng, Tetsuya Yoshida, Azadeh Sarkheyli, and Rui Zhou, editors, *Data Quality and Trust in Big Data - 5th International Workshop, QUAT 2018, Held in Conjunction with WISE 2018, Dubai, UAE, November 12-15, 2018, Revised Selected Papers*, volume 11235 of *Lecture Notes in Computer Science*, pages 94–110. Springer, 2018.
- [72] K2View. What is synthetic data generation? <https://www.k2view.com/what-is-synthetic-data-generation/>. Accessed: 2025-05-08.
- [73] Arlind Kadra, Marius Lindauer, Frank Hutter, and Josif Grabocka. Well-tuned simple nets excel on tabular datasets. In Marc'Aurelio Ranzato, Alina Beygelzimer, Yann N.

Dauphin, Percy Liang, and Jennifer Wortman Vaughan, editors, *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pages 23928–23941, 2021.

- [74] Sanket Kamthe, Samuel Assefa, and Marc Peter Deisenroth. Copula flows for synthetic data generation. *CoRR*, abs/2101.00598, 2021.
- [75] Abdulrahman Kerim. *Synthetic Data for Machine Learning: Revolutionize your approach to machine learning with this comprehensive conceptual guide*. Packt Publishing, 2023.
- [76] Mona Khalil. *Effective data analysis*. Manning Publications, New York, NY, 2025.
- [77] Jayoung Kim, Chaejeong Lee, and Noseong Park. Stasy: Score-based tabular data synthesis. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.
- [78] Jayoung Kim, Chaejeong Lee, Yehjin Shin, Sewon Park, Minjung Kim, Noseong Park, and Jihoon Cho. SOS: score-based oversampling for tabular data. In Aidong Zhang and Huzefa Rangwala, editors, *KDD '22: The 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Washington, DC, USA, August 14 - 18, 2022*, pages 762–772. ACM, 2022.
- [79] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In Yoshua Bengio and Yann LeCun, editors, *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Conference Track Proceedings*, 2014.
- [80] Mario Köppen. The curse of dimensionality. In *5th online world conference on soft computing in industrial applications (WSC5)*, volume 1, pages 4–8, 2000.
- [81] Akim Kotelnikov, Dmitry Baranchuk, Ivan Rubachev, and Artem Babenko. Tabddpm: Modelling tabular data with diffusion models. In *Proceedings of the 40th International Conference on Machine Learning*, pages 17564–17579. PMLR, 2023.
- [82] Vaibhav Kulkarni, Natasa Tagasovska, Thibault Vatter, and Benoît Garbinato. Generative models for simulating mobility trajectories. *CoRR*, abs/1811.12801, 2018.
- [83] Tuomas Kynkäänniemi, Tero Karras, Samuli Laine, Jaakko Lehtinen, and Timo Aila. Improved precision and recall metric for assessing generative models. In Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pages 3929–3938, 2019.
- [84] Hoa La and Kinh La. Comparing model-centric and data-centric approaches to determine the efficiency of data-centric ai. *Journal of Emerging Investigators*, April 2023.

- [85] Gretel Labs. Gretel evaluate. <https://docs.gretel.ai/optimize-synthetic-data/evaluate>. Accessed 2025-04-30.
- [86] F. Lazzeri. *Machine Learning for Time Series Forecasting with Python*. Wiley, 2020.
- [87] Gael Lederrey, Tim Hillel, and Michel Bierlaire. DATGAN: integrating expert knowledge into deep learning for synthetic tabular data. *CoRR*, abs/2203.03489, 2022.
- [88] Chaejeong Lee, Jayoung Kim, and Noseong Park. CoDi: Co-evolving contrastive diffusion models for mixed-type tabular synthesis. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 18940–18956. PMLR, 23–29 Jul 2023.
- [89] Mark Leznik, Arne Lochner, Stefan Wesner, and Jörg Domaschka. [sok] the great GAN bake off, an extensive systematic evaluation of generative adversarial network architectures for time series synthesis. *J. Syst. Res.*, 2(1), 2022.
- [90] Xiaomin Li, Vangelis Metsis, Huangyingrui Wang, and Anne Hee Hiong Ngu. TTS-GAN: A transformer-based time-series generative adversarial network. In Martin Michalowski, Syed Sibte Raza Abidi, and Samina Abidi, editors, *Artificial Intelligence in Medicine - 20th International Conference on Artificial Intelligence in Medicine, AIME 2022, Halifax, NS, Canada, June 14-17, 2022, Proceedings*, volume 13263 of *Lecture Notes in Computer Science*, pages 133–143. Springer, 2022.
- [91] Weiqiang Lin, Mehmet A. Orgun, and Graham J. Williams. An overview of temporal data mining. In Simeon J. Simoff, Graham J. Williams, and Markus Hegland, editors, *The 15th Australian Joint Conference on Artificial Intelligence 2002, Proceedings Australasian Data Mining Workshop, Canberra, Australia*, pages 83–90. University of Technology Sydney, Australia, 2002.
- [92] Zinan Lin, Alankar Jain, Chen Wang, Giulia Fanti, and Vyas Sekar. Generating high-fidelity, synthetic time series datasets with doppelganger. *CoRR*, abs/1909.13403, 2019.
- [93] Zinan Lin, Alankar Jain, Chen Wang, Giulia Fanti, and Vyas Sekar. Using gans for sharing networked time series data: Challenges, initial promise, and open questions. In *IMC '20: ACM Internet Measurement Conference, Virtual Event, USA, October 27-29, 2020*, pages 464–483. ACM, 2020.
- [94] Changshuo Liu and Canyao Liu. Entity-based financial tabular data synthesis with diffusion models. In *Proceedings of the 5th ACM International Conference on AI in Finance, ICAIF 2024, Brooklyn, NY, USA, November 14-17, 2024*, pages 547–554. ACM, 2024.
- [95] Nikos Mamoulis. *Temporal Data Mining*, pages 3935–3940. Springer New York, New York, NY, 2018.

- [96] Qi Mao, Hsin-Ying Lee, Hung-Yu Tseng, Siwei Ma, and Ming-Hsuan Yang. Mode seeking generative adversarial networks for diverse image synthesis. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2019, Long Beach, CA, USA, June 16-20, 2019*, pages 1429–1437. Computer Vision Foundation / IEEE, 2019.
- [97] Javier Marin. Evaluating synthetic tabular data generated to augment small sample datasets. <https://arxiv.org/abs/2211.10760>, 2025.
- [98] B. M. Maweu, R. Shamsuddin, Sagnik Dakshit, and B. Prabhakaran. Generating healthcare time series data for improving diagnostic accuracy of deep neural networks. *IEEE Transactions on Instrumentation and Measurement*, 70:1–15, 2021.
- [99] Casey Meehan, Kamalika Chaudhuri, and Sanjoy Dasgupta. A three sample hypothesis test for evaluating generative models. In Silvia Chiappa and Roberto Calandra, editors, *The 23rd International Conference on Artificial Intelligence and Statistics, AISTATS 2020, 26-28 August 2020, Online [Palermo, Sicily, Italy]*, volume 108 of *Proceedings of Machine Learning Research*, pages 3546–3556. PMLR, 2020.
- [100] Ofer Mendelevitch and Michael D. Lesh. Fidelity and privacy of synthetic medical data. *CoRR*, abs/2101.08658, 2021.
- [101] Shervin Minaee, Tomás Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. Large language models: A survey. *CoRR*, abs/2402.06196, 2024.
- [102] Khanh-Hoi Le Minh and Kim-Hung Le. Airgen: Gan-based synthetic data generator for air monitoring in smart city. In *6th IEEE International Forum on Research and Technology for Society and Industry, RTSI 2021, Naples, Italy, September 6-9, 2021*, pages 317–322. IEEE, 2021.
- [103] Mehdi Mirza and Simon Osindero. Conditional generative adversarial nets. *CoRR*, abs/1411.1784, 2014.
- [104] Theophano Mitsa. *Temporal Data Mining*. Chapman & Hall/CRC, 1st edition, 2010.
- [105] Tinatin Mshvidobadze. Python for automating machine learning tasks. *JINAV: Journal of Information and Visualization*, 2(2):77–82, Mar. 2021.
- [106] Arvind Narayanan and Vitaly Shmatikov. Robust de-anonymization of large sparse datasets. In *2008 IEEE Symposium on Security and Privacy (SP 2008), 18-21 May 2008, Oakland, California, USA*, pages 111–125. IEEE Computer Society, 2008.
- [107] Marcel Neuhausen, Patrick Herbers, and Markus König. Using synthetic data to improve and evaluate the tracking performance of construction workers on site. *Applied Sciences*, 10(14):4948, 2020.
- [108] A. Nielsen. *Practical Time Series Analysis: Prediction with Statistics and Machine Learning*. Titolo collana. O'Reilly, 2019.

- [109] Alexander V. Nikitin, Letizia Iannucci, and Samuel Kaski. TSGM: A flexible framework for generative modeling of synthetic time series. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, pages 129042–129061, 2024.
- [110] Josep Noguer, Iván Contreras, Omer Mujahid, Aleix Beneyto, and Josep Vehí. Generation of individualized synthetic data for augmentation of the type 1 diabetes data sets using deep learning models. *Sensors*, 22(13):4944, 2022.
- [111] Skyler Norgaard, Ramyar Saeedi, Keyvan Sasani, and Assefaw H. Gebremedhin. Synthetic sensor data generation for health applications: A supervised deep learning approach. In *40th Annual International Conference of the IEEE Engineering in Medicine and Biology Society, EMBC 2018, Honolulu, HI, USA, July 18-21, 2018*, pages 1164–1167. IEEE, 2018.
- [112] Beata Nowok, Gillian M. Raab, and Chris Dibben. synthpop: Bespoke creation of synthetic data in r. *Journal of Statistical Software*, 74(11):1–26, 2016.
- [113] Fredrick Ochieng’Odhiambo. Comparative study of various methods of handling missing data. *Mathematical Modelling and Applications*, 5(2):87, 2020.
- [114] Oualid Ouarem, Farid Nouioua, and Philippe Fournier-Viger. A survey of episode mining. *WIREs Data. Mining. Knowl. Discov.*, 14(2):e1524, 2024.
- [115] Inkit Padhi, Yair Schiff, Igor Melnyk, Mattia Rigotti, Youssef Mroueh, Pierre L. Dognin, Jerret Ross, Ravi Nair, and Erik R. Altman. Tabular transformers for modeling multivariate time series. In *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2021, Toronto, ON, Canada, June 6-11, 2021*, pages 3565–3569. IEEE, 2021.
- [116] Noseong Park, Mahmoud Mohammadi, Kshitij Gorde, Sushil Jajodia, Hongkyu Park, and Youngmin Kim. Data synthesis based on generative adversarial networks. *Proc. VLDB Endow.*, 11(10):1071–1083, 2018.
- [117] Neha Patki, Roy Wedge, and Kalyan Veeramachaneni. The synthetic data vault. In *2016 IEEE International Conference on Data Science and Advanced Analytics, DSAA 2016, Montreal, QC, Canada, October 17-19, 2016*, pages 399–410. IEEE, 2016.
- [118] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake VanderPlas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Edouard Duchesnay. Scikit-learn: Machine learning in python. *J. Mach. Learn. Res.*, 12:2825–2830, 2011.

- [119] François Petitjean, Germain Forestier, Geoffrey I. Webb, Ann E. Nicholson, Yanping Chen, and Eamonn J. Keogh. Dynamic time warping averaging of time series allows faster and more accurate classification. In Ravi Kumar, Hannu Toivonen, Jian Pei, Joshua Zhexue Huang, and Xindong Wu, editors, *2014 IEEE International Conference on Data Mining, ICDM 2014, Shenzhen, China, December 14-17, 2014*, pages 470–479. IEEE Computer Society, 2014.
- [120] Jackson Antonio do Prado Lima. GSP-Py - Generalized Sequence Pattern algorithm in Python. <https://doi.org/10.5281/zenodo.3333987>, December 2024.
- [121] Liudmila Ostroumova Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. Catboost: unbiased boosting with categorical features. In Samy Bengio, Hanna M. Wallach, Hugo Larochelle, Kristen Grauman, Nicolò Cesa-Bianchi, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, pages 6639–6649, 2018.
- [122] Zhaozhi Qian, Bogdan-Constantin Cebere, and Mihaela van der Schaar. Synthcity: facilitating innovative use cases of synthetic data in different data modalities. *CoRR*, abs/2301.07573, 2023.
- [123] Sebastian Raschka. Mlxtend: Providing machine learning and data science utilities and extensions to python’s scientific computing stack. *The Journal of Open Source Software*, 3(24):638, April 2018.
- [124] Luc Rocher, Julien M. Hendrickx, and Yves-Alexandre de Montjoye. Estimating the success of re-identifications in incomplete datasets using generative models. *Nature Communications*, 10(1):3069, 2019.
- [125] Donald B Rubin. Statistical disclosure limitation. *Journal of official Statistics*, 9(2):461–468, 1993.
- [126] Mehdi S. M. Sajjadi, Olivier Bachem, Mario Lucic, Olivier Bousquet, and Sylvain Gelly. Assessing generative models via precision and recall. In Samy Bengio, Hanna M. Wallach, Hugo Larochelle, Kristen Grauman, Nicolò Cesa-Bianchi, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, pages 5234–5243, 2018.
- [127] Arthur L. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3):210–229, 1959.
- [128] Gabriele Santangelo, Giovanna Nicora, Riccardo Bellazzi, and Arianna Dagliati. How good is your synthetic data? synthro, a dashboard to evaluate and benchmark synthetic tabular data. *BMC Medical Informatics Decis. Mak.*, 25(1):89, 2025.
- [129] SDV. Deepecho. <https://github.com/sdv-dev/DeepEcho>. Accessed: 2025-04-30.

- [130] Prithviraj Sen and Amol Deshpande. Representing and querying correlated tuples in probabilistic databases. In Rada Chirkova, Asuman Dogac, M. Tamer Özsu, and Timos K. Sellis, editors, *Proceedings of the 23rd International Conference on Data Engineering, ICDE 2007, The Marmara Hotel, Istanbul, Turkey, April 15-20, 2007*, pages 596–605. IEEE Computer Society, 2007.
- [131] Satya Narayan Shukla and Benjamin M. Marlin. A survey on principles, models and methods for learning from irregularly sampled time series: From discretization to attention and invariance. *CoRR*, abs/2012.00168, 2020.
- [132] Joshua Snoke, Gillian M. Raab, Beata Nowok, Chris Dibben, and Aleksandra Slavkovic. General and specific utility measures for synthetic data. *Journal of the Royal Statistical Society Series A: Statistics in Society*, 181(3):663–688, 2018.
- [133] C Spearman. The proof and measurement of association between two things. *Am. J. Psychol.*, 15(1):72, January 1904.
- [134] Ramakrishnan Srikant and Rakesh Agrawal. Mining sequential patterns: Generalizations and performance improvements. In Peter Apers, Mokrane Bouzeghoub, and Georges Gardarin, editors, *Advances in Database Technology — EDBT '96*, pages 1–17. Springer Berlin Heidelberg, 1996.
- [135] Padmanaba Srinivasan and William J. Knottenbelt. Time-series transformer generative adversarial networks. *CoRR*, abs/2205.11164, 2022.
- [136] Akash Srivastava, Lazar Valkov, Chris Russell, Michael U. Gutmann, and Charles Sutton. VEEGAN: reducing mode collapse in gans using implicit variational learning. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pages 3308–3318, 2017.
- [137] Statista and IDC. Volume of data/information created, captured, copied, and consumed worldwide from 2010 to 2023, with forecasts from 2024 to 2028. <https://www.statista.com/statistics/871513/worldwide-data-created/>, 2024. Accessed: 2025-05-02.
- [138] Michael Stenger, André Bauer, Thomas Prantl, Robert Leppich, Nathaniel Hudson, Kyle Chard, Ian T. Foster, and Samuel Kounev. Thinking in categories: A survey on assessing the quality for time series synthesis. *ACM J. Data Inf. Qual.*, 16(2):14, 2024.
- [139] Michael Stenger, Robert Leppich, Ian T. Foster, Samuel Kounev, and André Bauer. Evaluation is key: a survey on evaluation measures for synthetic time series. *J. Big Data*, 11(1):66, 2024.
- [140] Namjoon Suh, Yuning Yang, Din-Yin Hsieh, Qitong Luan, Shirong Xu, Shixiang Zhu, and Guang Cheng. Timeautodiff: Combining autoencoder and diffusion model for time series tabular data synthesizing. *CoRR*, abs/2406.16028, 2024.

- [141] Dina Sukhobok, Nikolay Nikolov, and Dumitru Roman. Tabular data anomaly patterns. In Muhammad Younas, Irfan Awan, and Irena Holubová, editors, *International Conference on Big Data Innovations and Applications, Innovate-Data 2017, Prague, Czech Republic, August 21-23, 2017*, pages 25–34. IEEE, 2017.
- [142] Haoyuan Sun, Navid Azizan, Akash Srivastava, and Hao Wang. Private synthetic data meets ensemble learning. *CoRR*, abs/2310.09729, 2023.
- [143] Syed Mahir Tazwar, Max Knobbout, Enrique Hortal Quesada, and Mirela Popa. Tabvae: A novel VAE for generating synthetic tabular data. In Modesto Castrillón Santana, Maria De Marsico, and Ana Fred, editors, *Proceedings of the 13th International Conference on Pattern Recognition Applications and Methods, ICPRAM 2024, Rome, Italy, February 24-26, 2024*, pages 17–26. SCITEPRESS, 2024.
- [144] Hui Yie Teh, Andreas W. Kempa-Liehr, and Kevin I-Kai Wang. Sensor data quality: a systematic review. *J. Big Data*, 7(1):11, 2020.
- [145] Lucas Theis, Aäron van den Oord, and Matthias Bethge. A note on the evaluation of generative models. In Yoshua Bengio and Yann LeCun, editors, *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016.
- [146] Jonathan Tremblay, Aayush Prakash, David Acuna, Mark Brophy, Varun Jampani, Cem Anil, Thang To, Eric Cameracci, Shaad Boochoon, and Stan Birchfield. Training deep networks with synthetic data: Bridging the reality gap by domain randomization. In *2018 IEEE Conference on Computer Vision and Pattern Recognition Workshops, CVPR Workshops 2018, Salt Lake City, UT, USA, June 18-22, 2018*, pages 969–977. Computer Vision Foundation / IEEE Computer Society, 2018.
- [147] Jürgen Umbrich, Sebastian Neumaier, and Axel Polleres. Quality assessment and evolution of open data portals. In Irfan Awan, Muhammad Younas, and Massimo Mecella, editors, *3rd International Conference on Future Internet of Things and Cloud, FiCloud 2015, Rome, Italy, August 24-26, 2015*, pages 404–411. IEEE Computer Society, 2015.
- [148] vanderschaarlab. synthcity. <https://github.com/vanderschaarlab/synthcity>. Accessed: 2025-04-17.
- [149] Mario Villaizán-Valladolid, Matteo Salvatori, Carlos Segura, and Ioannis Arapakis. Diffusion models for tabular data imputation and synthetic data generation. *CoRR*, abs/2407.02549, 2024.
- [150] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M.

- Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020.
- [151] Alex X. Wang, Stefanka S. Chukova, Colin R. Simpson, and Binh P. Nguyen. Challenges and opportunities of generative models on tabular data. *Appl. Soft Comput.*, 166:112223, 2024.
 - [152] Alex Xing Wang. Data-Centric AI: Tabular Data Synthesis with Deep Generative Models. https://openaccess.wgtn.ac.nz/articles/thesis/Data-Centric_AI_Tabular_Data_Synthesis_with_Deep_Generative_Models/27014419, 2024.
 - [153] Yuxin Wang, Duanyu Feng, Yongfu Dai, Zhengyu Chen, Jimin Huang, Sophia Ananiadou, Qianqian Xie, and Hao Wang. HARMONIC: harnessing llms for tabular data synthesis and privacy protection. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, pages 100196–100212, 2024.
 - [154] Wes McKinney. Data Structures for Statistical Computing in Python. In Stéfan van der Walt and Jarrod Millman, editors, *Proceedings of the 9th Python in Science Conference*, pages 56 – 61, 2010.
 - [155] Magnus Wiese, Robert Knobloch, Ralf Korn, and Peter Kretschmer. Quant gans: deep generation of financial time series. *Quantitative Finance*, 20(9):1419–1440, April 2020.
 - [156] Kyle Wiggers. Waymo is using ai to simulate autonomous vehicle camera data. <https://venturebeat.com/ai/waymo-is-using-ai-to-simulate-autonomous-vehicle-camera-data/>. Accessed: 2025-05-04.
 - [157] Ian H. Witten, Eibe Frank, Mark A. Hall, and Christopher J. Pal. *Data Mining, Fourth Edition: Practical Machine Learning Tools and Techniques*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 4th edition, 2016.
 - [158] Chengshu Xie, Shaurya Jauhari, and Antonio Mora. Popularity and performance of bioinformatics software: the case of gene set analysis. *BMC Bioinform.*, 22(1):191, 2021.
 - [159] Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, and Kalyan Veeramachaneni. Modeling tabular data using conditional GAN. In Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pages 7333–7343, 2019.

- [160] Lei Xu and Kalyan Veeramachaneni. Synthesizing tabular data using generative adversarial networks. *CoRR*, abs/1811.11264, 2018.
- [161] Chao Yan, Yao Yan, Zhiyu Wan, Ziqi Zhang, Larsson Omberg, Justin Guinney, Sean D Mooney, and Bradley A Malin. A multifaceted benchmarking of synthetic electronic health record generation models. *Nat. Commun.*, 13(1):7609, December 2022.
- [162] Scott Cheng-Hsin Yang, Baxter Eaves, Michael Schmidt, Ken Swanson, and Patrick Shafto. Structured evaluation of synthetic tabular data. *CoRR*, abs/2403.10424, 2024.
- [163] Zeyu Yang, Peikun Guo, Khadija Zanna, and Akane Sano. Balanced mixed-type tabular data synthesis with diffusion models. *CoRR*, abs/2404.08254, 2024.
- [164] Jinsung Yoon, Daniel Jarrett, and Mihaela van der Schaar. Time-series generative adversarial networks. In Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d’Alché-Buc, Emily B. Fox, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pages 5509–5519, 2019.
- [165] Hengrui Zhang, Jiani Zhang, Zhengyuan Shen, Balasubramaniam Srinivasan, Xiao Qin, Christos Faloutsos, Huzefa Rangwala, and George Karypis. Mixed-type tabular data synthesis with score-based diffusion in latent space. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.
- [166] Kevin Alex Zhang, Neha Patki, and Kalyan Veeramachaneni. Sequential models in the synthetic data vault. *CoRR*, abs/2207.14406, 2022.
- [167] Yifan Zhang, Weiqi Chen, Zhaoyang Zhu, Dalin Qin, Liang Sun, Xue Wang, Qingsong Wen, Zhang Zhang, Liang Wang, and Rong Jin. Addressing concept shift in online time series forecasting: Detect-then-adapt. *CoRR*, abs/2403.14949, 2024.
- [168] Pu Zhao, Chuan Luo, Bo Qiao, Lu Wang, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. T-SMOTE: temporal-oriented synthetic minority oversampling technique for imbalanced time series classification. In Luc De Raedt, editor, *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI 2022, Vienna, Austria, 23-29 July 2022*, pages 2406–2412. ijcai.org, 2022.
- [169] Zilong Zhao, Robert Birke, and Lydia Y. Chen. Tabula: Harnessing language models for tabular data synthesis. *CoRR*, abs/2310.12746, 2023.
- [170] Zilong Zhao, Aditya Kunar, Robert Birke, and Lydia Y. Chen. CTAB-GAN: effective table data synthesizing. In Vineeth N. Balasubramanian and Ivor W. Tsang, editors, *Asian Conference on Machine Learning, ACML 2021, 17-19 November 2021, Virtual Event*, volume 157 of *Proceedings of Machine Learning Research*, pages 97–112. PMLR, 2021.

- [171] Zilong Zhao, Aditya Kunar, Robert Birke, Hiek Van der Scheer, and Lydia Y. Chen. CTAB-GAN+: enhancing tabular data synthesis. *Frontiers Big Data*, 6, 2024.
- [172] Shuhan Zheng and Nontawat Charoenphakdee. Diffusion models for missing value imputation in tabular data. *CoRR*, abs/2210.17128, 2022.
- [173] Xiangtian Zheng, Bin Wang, Dileep Kalathil, and Le Xie. Creation of synthetic networked pmu data: A generative adversarial network approach. *arXiv preprint arXiv:1908.08180*, 2019.
- [174] Youran Zhou, Sunil Aryal, and M Reda Bouadjenek. A comprehensive review of handling missing data: Exploring special missing mechanisms. *arXiv preprint arXiv:2404.04905*, 2024.

A. Appendix

A.1. Transfermarkt data set preprocessing steps

The Table A.1 is the complete list of preprocessing steps taken:

Table A.1.: Preprocessing steps and their descriptions

Preprocessing Step	Description
Entry validity	Start and end dates indicating the validity period of each entry were added based on the sampling timestamps. This makes the data set tuple-versioned.
Merge same-day entries	Entries sampled on the same date for the same entity were merged into a single record. For example, if a player sustained an injury and transferred to the Bundesliga on the same day, both events are represented within the same entry. This prevents illegal states within entities where overlapping validity dates occur.
Injury categorization	Due to the presence of over 300 distinct injury types, injuries were grouped into broader categories. The corresponding injury category was added as an additional column, adding a functional dependency to the data set. The functional dependency is $\{\text{injury}\} \rightarrow \{\text{injury_category}\}$

Continued on next page

Table A.1 – continued from previous page

Preprocessing Step	Description
Market value categorization	Market values were categorized into quantiles based on their distribution at the start of each respective season. The resulting category was added as a new feature. This adds a categorical feature and sets into relation the market values within a temporal dimension. A market value of 10 million might be very high in 2010 but not in 2019, thus reflecting this change.
English translation	Conversion of text fields to English, as the majority of entries in the Transfermarkt database are originally in German.
Removal of players with multiple injuries	Few players with multiple injuries occurring at the same time were removed from the data set. Otherwise this would have lead to challenges when analyzing the sequences of injuries when multiple can occur at the same time.
Removal of players with overlapping injuries	Few players with overlapping injury periods were excluded from the data set to ensure consistency in injury representation. Otherwise this would have lead to challenges when analyzing the sequences of injuries when multiple can occur at the same time.
Performance data integration	Performance metrics such as goals scored and minutes played were added based on the player's data until the time of sampling. This allows to set into relation the performance data over a temporal dimension.
Missed games calculation	The number of games missed by a player's team during the validity period of an injury entry was calculated and added as a feature. This took into account Bundesliga, Premier League, Champions League or Europa League. This reflects how severe an injury was in terms of games missed.

Continued on next page

Table A.1 – continued from previous page

Preprocessing Step	Description
Transfer fee cleaning	Unknown or missing transfer fees were standardized by setting their values to 0. A value is required to keep the feature numerical. Unknown or missing transfer fees were strings such as '?' or '-'
Age at time of entry	The player's age at the start of each entry's validity period was calculated and added as a feature. This adds another temporal relationship to the data set.
Last transfer fee propagation	The most recent transfer fee was propagated to all subsequent entries until the next transfer occurred. This allows for the information of the transfer fee not to be lost between other transfer-unrelated events.
Injury propagation	If a player remained injured during subsequent entries, the injury status was carried over and reflected in those entries, including cases such as market value updates.

A.2. Transfermarkt feature explanation

Table A.2 presents the complete list of features alongside the explanations of the respective feature.

Table A.2.: Features and their meaning

Name	Description
player_id	Unique identifier of the player in the Transfermarkt database.
first_name	The player's first name, as listed in the Transfermarkt database.
last_name	The player's last name, as listed in the Transfermarkt database.
reason	The reason for entry's creation, for instance "regular interval" or "transfer" as detailed in subsection 4.2.2. If multiple events occurred on the same day, their reasons are merged as a comma separated string.
validity_start	The date on which the event was sampled and the entry's validity period begins, formatted as YYYY-MM-DD.
validity_end	The date on which the entry's validity period ends, formatted as YYYY-MM-DD.
injury	The name of the injury the player is currently sustaining, if applicable. Otherwise, this field is set to null.
injury_category	The name of the injury category the player is currently sustaining, if applicable. Otherwise, this field is set to null.
citizenship	Citizenship of the player as recorded in the Transfermarkt database.

Continued on next page

Table A.2 – continued from previous page

Name	Description
international_competition	The highest international competition (Champions League or Europa League) the player's club participated in during the respective season. Set to an empty string if the club did not compete internationally.
market_value_category	A categorical representation of a player's market value based on its position within the distribution of all market values on July 1st of each season. Values are grouped into five quantiles, each covering 20% of the data: VERY LOW, LOW, MEDIUM, HIGH, and VERY HIGH.
age	The player's age on the date the entry's validity period begins.
coach_id	Unique identifier of the coach associated with the player's club. If the club is outside the Bundesliga and Premier League, the value is set to 1 as a placeholder for "not applicable".
club_id	The Transfermarkt identifier of the player's club. If the club is outside the Bundesliga or Premier League, the value is set to -1 as a placeholder for "not applicable".
league_id	The Transfermarkt identifier of the league in which the player's club competes.
season_id	Identifier of the season, ranging from 2010 to 2019, corresponding to the respective year the season started.
last_transfer_fee	The transfer fee paid for the player in their most recent transfer prior to the current entry.
pseudonym	Pseudonym of the player in the Transfermarkt database.

Continued on next page

Table A.2 – continued from previous page

Name	Description
position	The primary playing position of the player (e.g., goalkeeper, central midfielder, ...) as defined in the Transfermarkt database. As a possible change of primary position is not tracked in the Transfermarkt database, it is considered to be a static feature in this work.
foot	The player's dominant foot. Categories include right, left, both, or – if unknown.
height	The height of the player in meters.
date_of_birth	The player's date of birth, formatted as YYYY-MM-DD.
club	The name of the club the player plays for. This field is set to null if the player transfers to a club outside the Bundesliga and Premier League.
league	The name of the league in which the player's club competes. This field is set to null if the player transfers to a league outside the Bundesliga and Premier League.
coach	The name of the coach at the player's club. This field is set to null if the player transfers to a club outside the Bundesliga and Premier League.
market_value	The player's Transfermarkt market value at the start of the entry's validity period.
league_played_matches	Total number of matches the player had played in the Bundesliga and Premier League up to the start of the entry's validity period.
league_goals	Total number of goals the player had scored in the Bundesliga and Premier League up to the start of the entry's validity period.
league_minutes_played	Total number of minutes the player had played in the Bundesliga and Premier League up to the start of the entry's validity period.

Continued on next page

Table A.2 – continued from previous page

Name	Description
international_goals	Total number of goals the player had scored in the Champions League and Europa League up to the start of the entry's validity period.
international_minutes_played	Total number of minutes the player had played in the Champions League and Europa League up to the start of the entry's validity period.
international_played_matches	Total number of matches the player had played in the Champions League and Europa League up to the start of the entry's validity period.
missed_matches	Total number of matches the player missed during the entry's validity period across the Bundesliga, Premier League, Champions League, and Europa League due to injury.

A.3. TempoBench prototype interfaces

The Listing A.1 is the Python definition of the BenchmarkMetric interface, that all metrics implemented in the prototype satisfy and that custom metrics have to implement.

Listing A.1: Abstract base class for benchmark metrics

```
from abc import ABCMeta, abstractmethod
import pandas
class BenchmarkMetric(metaclass=ABCMeta):
    @abstractmethod
    def compute(self, X_real: pandas.DataFrame, X_syn:
        pandas.DataFrame) -> dict:
        """
        Compute the metric.
        :param X_real: The original dataset
        :param X_syn: The synthetic dataset
        :return: Dict that contains all values from this metric
        """
        ...

    @abstractmethod
    def name(self) -> str:
        """Name of the metric"""
        ...

    @abstractmethod
    def category(self) -> str:
        """Category of the metric"""
```

The Listing A.2 is the Python definition of the BenchmarkVisualization interface, that all visualizations implemented in the prototype satisfy and that custom visualizations have to implement.

Listing A.2: Abstract base class for benchmark visualizations

```
import pandas
class BenchmarkVisualization(metaclass=ABCMeta):
    @abstractmethod
    def create(self, X_real: pandas.DataFrame, X_syn:
        pandas.DataFrame, path: str):
        """
        Create the visualization and save it.
        :param X_real: The original dataset
        :param X_syn: The synthetic dataset
        :param path: The path to save the visualization
        """
        ...

    @abstractmethod
    def category(self) -> str:
        """Name of category (will be a subfolder)"""

    @abstractmethod
    def name(self) -> str:
        """Name of the visualization."""
        ...
```

A.4. Thresholds for metric comparison

This section defines the thresholds for each metric when evaluating how a synthesized data set performed on a metric. Table A.3 shows the thresholds while Table A.4 shows the used formula if a metric is made up of more than one component.

Table A.3.: Thresholds for evaluation

Metric	++	+	o	-	--
Duplicate rows	0	< 0.1	< 0.5	< 1	>= 1
Datatype mismatch	0	< 5	< 10	< 15	>= 15
Tuple validity overlap	0	< 0.5	< 1	< 2	>= 2
Validity end before start	0	< 1	< 5	< 10	>= 10
Monotonic increase	0	< 2	< 5	< 10	>= 10
Static data remains unchanged	0	< 1	< 5	< 10	>= 10
Timestamp min. and max.	0	< 5	< 10	< 15	>= 15
Maximum validity time	0	< 0.5	< 1	< 2	>= 2
Elementary statistics	<10	< 20	< 40	< 60	>= 60
Jensen-Shannon divergence	< 0.15	< 0.25	< 0.35	< 0.45	>= 0.45
Correlation	< 10	< 15	< 20	< 25	>= 25
Mean sequence length	< 10	< 20	< 30	< 40	>=40
Mean validity	< 5	< 15	< 25	< 35	>=35
Intra-class distance	< 5	< 15	< 25	< 50	>=50
Transition matrix distance	<0.1	<0.2	<0.3	< 0.4	>= 0.4
Train-on-Synthetic-Test-on-Real Regression	> 0.7	> 0.6	> 0.5	> 0.4	<= 0.4
GSP	>0.8	>0.65	> 0.5	> 0.35	<= 0.35

Table A.4.: Unit used for thresholds evaluation

Metric	Used unit
Duplicate rows	Proportion of duplicate rows in %
Datatype mismatch	Proportion of mismatched columns in %
Tuple validity overlap	Proportion of rows violated in %
Validity end before start	Proportion of rows violated in %
Monotonic increase	Proportion of rows violated in %
Static data remains unchanged	Proportion of entities violated in %
Timestamp min. and max.	Proportion of rows violated in %
Maximum validity time	Proportion of rows violated in %
Elementary statistics	Average deviation for entropy, mean, median, standard deviation minimum and maximum %
Jensen Shannon divergence	Jensen-Shannon divergence
Correlation	Normalized deviation of the correlation matrices in for the whole data set in %
Mean sequence length	Deviation from the ground truth mean sequence length in %
Mean validity	Deviation from the ground truth mean validity in %
Intra-class distance	Deviation from the ground truth intra-class distance in %
Transition matrix distance	Average distance of transition matrices over all categories
Train-on-Synthetic-Test-on-Real Regression	R^2 value
GSP	Average F1 score over the evaluated categories

Eidesstattliche Erklärung

Hiermit versichere ich an Eides statt, dass ich die vorliegende Arbeit im Masterstudiengang Informatik selbstständig verfasst und keine anderen als die angegebenen Hilfsmittel – insbesondere keine im Quellenverzeichnis nicht benannten Internet-Quellen – benutzt habe. Alle Stellen, die wörtlich oder sinngemäß aus Veröffentlichungen entnommen wurden, sind als solche kenntlich gemacht. Ich versichere weiterhin, dass ich die Arbeit vorher nicht in einem anderen Prüfungsverfahren eingereicht habe. Sofern im Zuge der Erstellung der vorliegenden Abschlussarbeit generative Künstliche Intelligenz (gKI) basierte elektronische Hilfsmittel verwendet wurden, versichere ich, dass meine eigene Leistung im Vordergrund stand und dass eine vollständige Dokumentation aller verwendeten Hilfsmittel gemäß der Guten Wissenschaftlichen Praxis vorliegt. Ich trage die Verantwortung für eventuell durch die gKI generierte fehlerhafte oder verzerrte Inhalte, fehlerhafte Referenzen, Verstöße gegen das Datenschutz- und Urheberrecht oder Plagiate.

Hamburg, den 29.06.2025

Benjamin Pöhlmann

Veröffentlichung

Ich stimme der Einstellung der Arbeit in die Bibliothek des Fachbereichs Informatik zu.

Hamburg, den 29.06.2025

Benjamin Pöhlmann